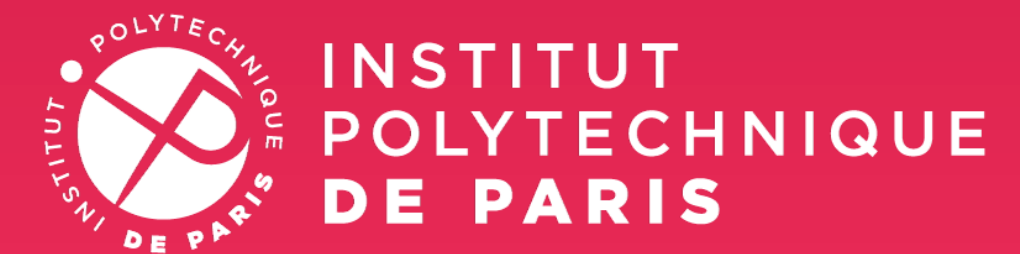


CSC_4CS02_TP

Cryptologie

Sébastien Canard
Weiqiang Wen
Année 2024-2025



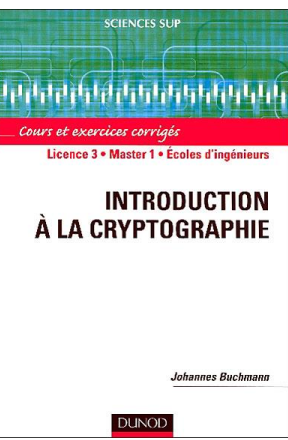
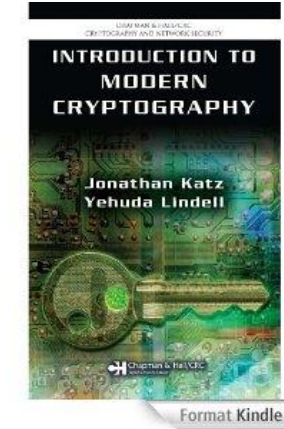
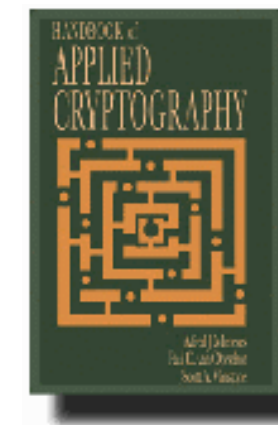
Objectifs du cours

- Savoir expliquer les principes généraux de la cryptographie, les méthodes de cryptographie à clé publique et à clé secrète, le concept de clé cryptographique, les algorithmes de chiffrement, de signature, d'authentification de message et de fonction de hachage.
- Savoir quelle taille de clé cryptographique utiliser en fonction des risques de cryptanalyse et des préconisations en cours.
- Savoir faire des recommandations sur le choix du type de cryptographie et le type d'algorithme à utiliser en fonction des exigences d'un produit ou d'un service.
- Avoir un regard critique sur une solution cryptographique déployée.

Contrôle de connaissance

- Un examen final écrit (CC) qui va comporter deux parties :
 - La première va permettre de tester les connaissances sur les principes généraux de la cryptographie (objectifs, types, méthodes, notion de clé, etc.), par le biais de questions plus ou moins proches des exercices vus en TD ou ceux à faire en autonomie
 - La seconde portera sur la capacité de l'étudiant à avoir un regard critique et faire des recommandations, par le biais d'une ou plusieurs études de cas. Jusqu'à 10% des questions posées lors de cette partie ne sera pas directement lié à ce qui a été vu en cours ou en TD et permettra de tester l'étudiant sur sa capacité à faire face à des situations nouvelles
- Une unique page recto-verso sera autorisée lors cet examen final

Références utiles



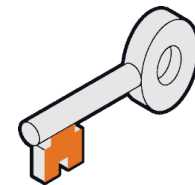
- Transparents du cours => Synapses / site pédagogique
- Pour aller plus loin
 - Introduction to Cryptography, Johannes Buchmann, (2000)
 - Introduction to Modern Cryptography, Katz , Lindell (2007)
 - Handbook of Applied Cryptography, Alfred Menezes, Paul van Oorschot, Scott Vanstone (1996)
<http://cacr.uwaterloo.ca/hac/>
- Référence des instances officielles
 - ANSSI en France : <https://www.ssi.gouv.fr/guide/mecanismes-cryptographiques/>
 - BSI en Allemagne : https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Standards-und-Zertifizierung/Technische-Richtlinien/TR-nach-Thema-sortiert/tr02102/tr02102_node.html
 - ENISA en Europe : <https://www.enisa.europa.eu/publications/algorithms-key-size-and-parameters-report-2014>
- Culture générale
 - L'histoire des codes secrets, Simon Singh, 1999

Définitions de base



Services de sécurité

Service de sécurité	Outil cryptographique	Usages
• Confidentialité • Protection des données • Combattre l'écoute en rendant inintelligible un document	Chiffrement	Protection des données militaires

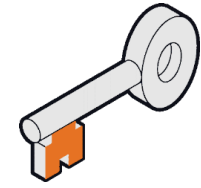


ATTENTION

- La cryptographie n'est qu'une composante de la sécurité
- Il en existe d'autres
 - Confiance dans les personnes, les organisations, les process
 - Confiance dans l'implémentation matérielle (hardware)
 - Confiance dans l'implémentation logicielle (software)

1. Cryptographie ancienne et ses limites

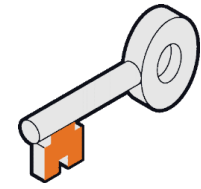
Reprenons l'histoire à ses débuts



Cryptographie = protéger un secret $\Rightarrow$ confidentialité $\Rightarrow$ chiffrement

- Autre moyen pour protéger un secret : **stéganographie**
 - Encre sympathique
 - Prendre la première/deuxième/... lettre de chaque mot
 - Piquer certaines lettres dans un message
 - ...
- Séparation de l'histoire en deux « âges »
 - **Âge artisanal** (Antiquité jusqu'à 1900) : systèmes basés sur des méthodes rudimentaires
 - **Âge technique** (1900 – 1970) : systèmes basés sur des machines (électro-)mécaniques
- Deux méthodes majeures
 - **Permutation** : intervertir les lettres/mots d'un message
 - **Substitution** : remplacer un-e lettre/mot par un-e autre

Méthode de permutation – Exemple de la Syctale



Principe : mélanger/permuter les lettres/mots d'un texte

- Principe de la **Scytale** (bâton de Plutarque)
 - Utilisé par les Spartiates (404 av. J.-C.)
 - Il faut pouvoir avoir un moyen facile d'exécuter la permutation inverse, sans qu'un ennemi ne puisse le faire facilement
- Exemple avec le texte suivant :
 - MPCAUEOEROVBRNLRENEOVOUCNTCOETTIDAFKNRIDRCAPDEOEETKORSN
- Et deux bâtons de tailles différentes
 - Première taille :
MVODPNPBUADCRCFEANNKOELTNEURCREEEEOITONEDKEETROROTCROVIAS
 - Seconde taille :
MERCIDEPRENDRECONTACTAVECFAKEBOOKPOURVENDRENOTRESOLUTION



Cryptanalyse de la Scytale



- Idée : attaque par force brute sur la « taille du bâton »
- Compter le nombre de lettre (56 dans notre exemple)
- Créer des grilles permettant de placer toutes les lettres (8×7 , 11×6 , ...)
- Placer les lettres

M	P	C	A	E	U	E	O	E	R	O
V	B	R	N	L	R	E	N	E	O	V
O	U	C	N	T	C	O	E	T	T	I
D	A	F	K	N	R	I	D	R	C	A
P	D	E	O	E	E	T	K	O	R	S
N										

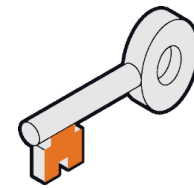
← taille du bâton →

M	P	C	A	E	U	E	O
E	R	O	V	B	R	N	L
R	E	N	E	O	V	O	U
C	N	T	C	O	E	T	T
I	D	A	F	K	N	R	I
D	R	C	A	P	D	E	O
E	E	T	K	O	R	S	N

← taille du bâton →

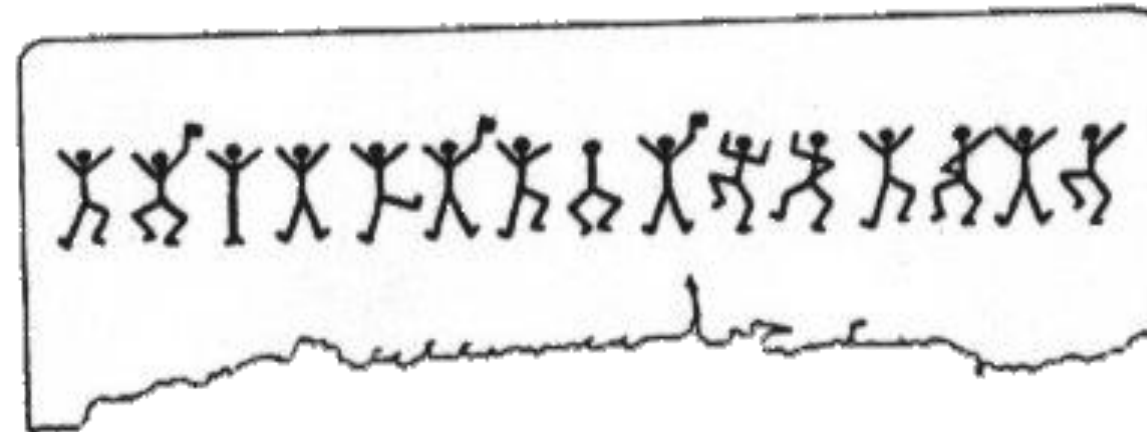
- Voir si un message apparaît...
- Très rapide avec les moyens de calcul actuels

Méthode de substitution



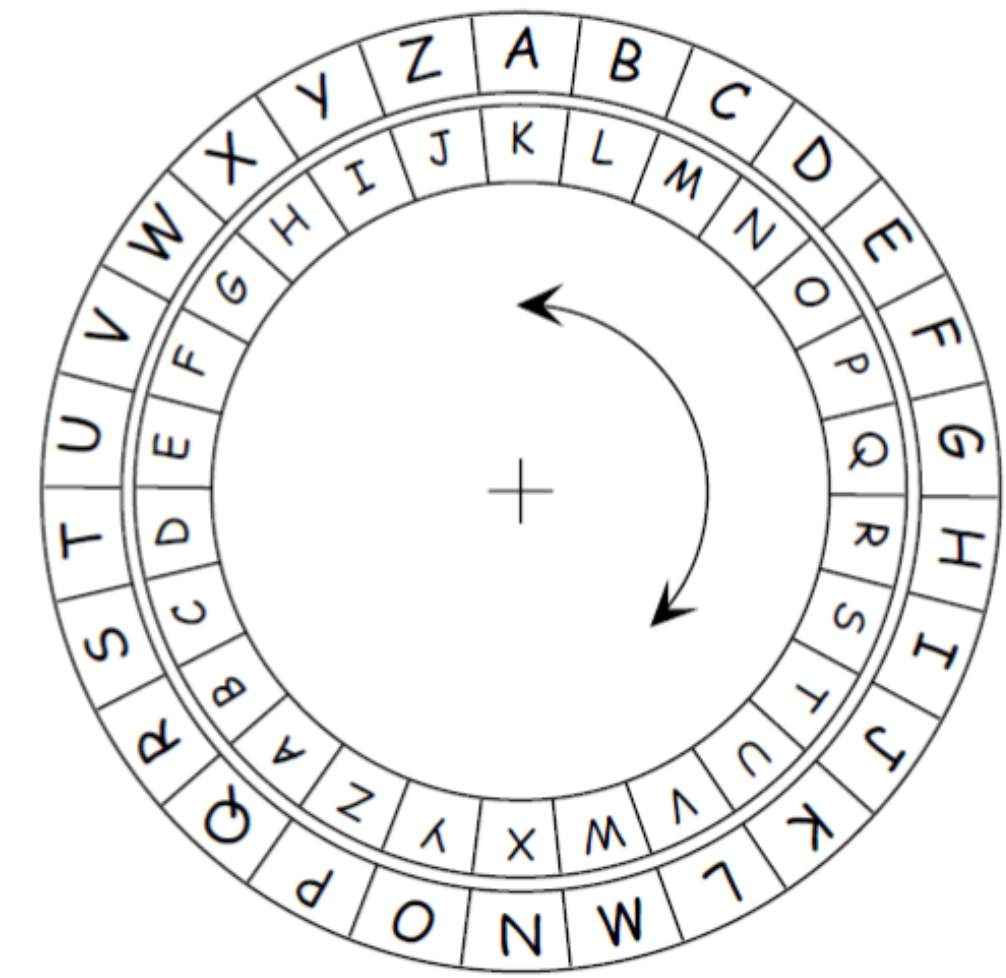
Principe : remplacer/substituer une lettre/mot par autre chose

- En utilisant un autre alphabet
 - Exemple des « Hommes dansants » (Sir Arthur Conan Doyle) avec Sherlock Holmes



- En utilisant le même alphabet
 - Exemples : systèmes de César, Vigenère, Vernam
 - Toujours le même principe : comment rapidement appliquer la substitution inverse sans donner de moyen facile pour l'ennemi de retrouver le texte initial ?

Système de César



- Utilisé au 1^{er} siècle (par Jules César ?)
- Principe : simplifier la mémorisation de la substitution
 - Rotation de l'alphabet d'une certaine valeur ($10 \Rightarrow A = K$)
 - Remplacer chaque lettre par la valeur correspondante
 - Déchiffrement : lire la roue dans l'autre sens

- Exemple de texte clair

C E M E S S A G E E S T C H I F F R E A V E C L E S Y S T E M E D E C E S A R

- Et du texte chiffré correspondant

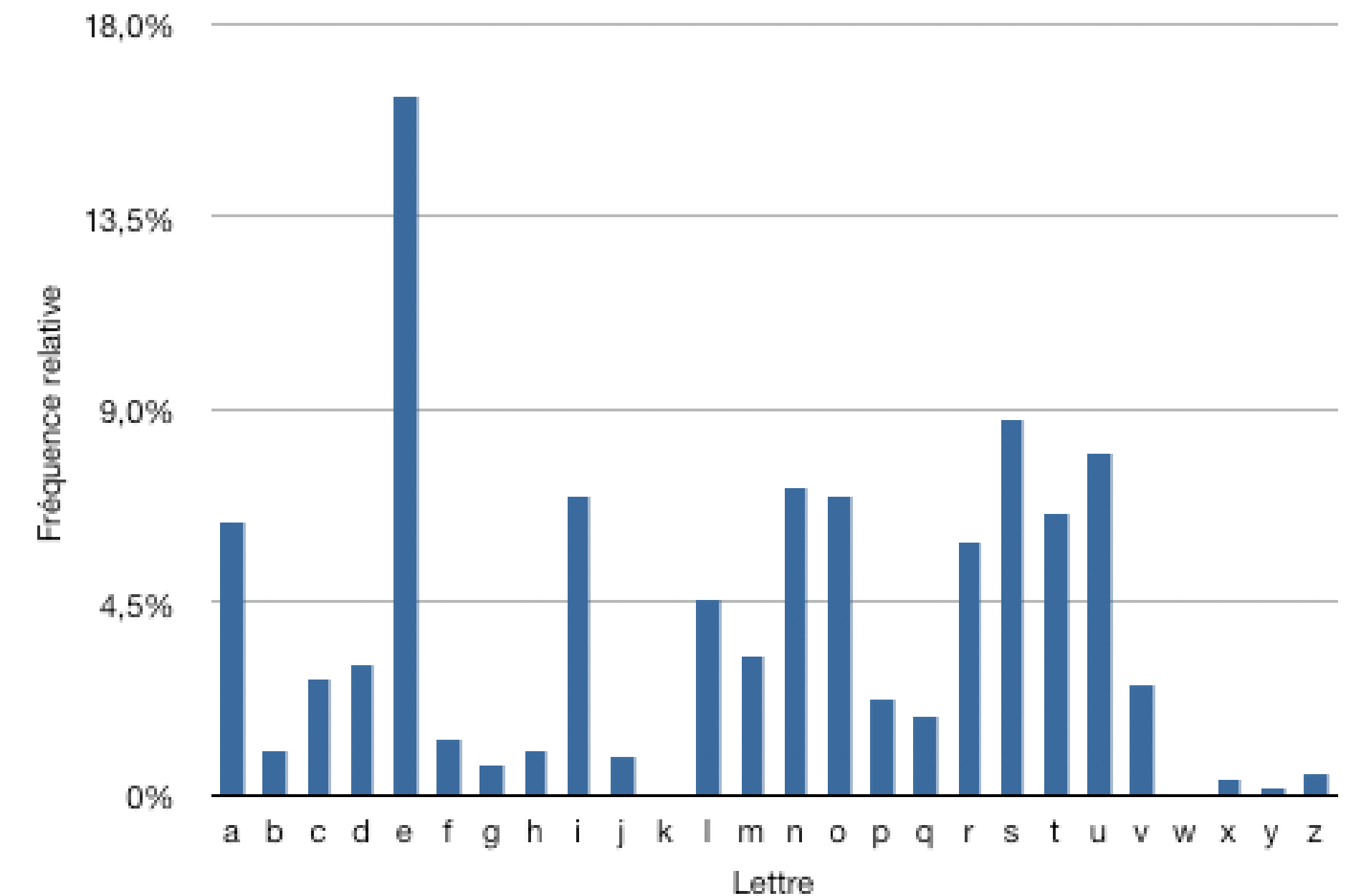
M O W O C C K Q O O C D M R S P P B O K F O M V O C I C D O W O N O M O C K B

Cryptanalyse de César

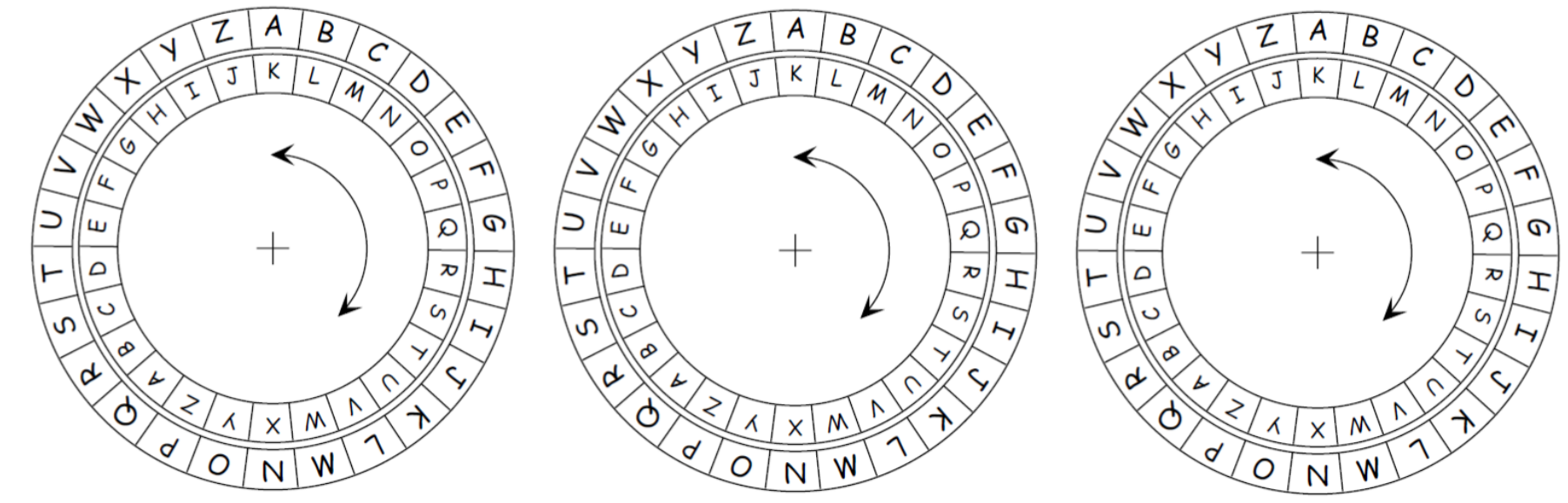


- Nombre de possibilités assez faible : 26
 - Tester toutes les combinaisons possible...
- **Meilleure méthode : utiliser l'analyse fréquentielle**
 - Formalisée par Al-Kindi au IXème siècle
- **Principe**
 - Chaque lettre est toujours chiffrée de la même façon
 - Compter le nombre d'occurrence des lettres dans le texte chiffré
 - Celle qui apparaît le plus correspond, avec une bonne probabilité, à la lettre E (la plus utilisée en Français)
 - On obtient le décalage utilisé

Source : <http://www.etaletaculture.fr/>



Système de Vigenère



- Proposé au XVIème siècle par Blaise de Vigenère
- Principe : utiliser successivement **plusieurs systèmes de César (plusieurs substitutions)** avec des décalages différents
- Exemple avec la clé **KEY** $\Rightarrow$ **A = K**, **A = E**, **A = Y**, **A = K**, **A = E**, **A = Y**, ...
- Exemple de texte clair

C E M E S S A G E E S T C H I F F R E A V E C L E S Y S T E M E D E C E S A R

- Et du chiffré correspondant

M I K O W Q K K C O W R M L G P J P O E T O G J O W W C X C W I B O G C C E P

Source : <http://fr.wikipedia.org/>

		Lettre en clair																									
		A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
C i é	A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
	B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
	C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
	D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
	E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
	F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
	G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
	H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
	I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
	J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
	K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
	L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
	M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
	N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
	O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
	P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
	R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
	S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
	T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
	U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
	V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
	W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
	X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
	Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
	Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

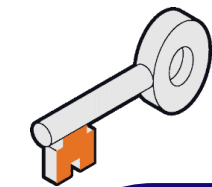
Cryptanalyse de Vigenère



- Cryptanalyse indépendamment proposée par Babbage (1854) et Kasiki (1863)
- Une même lettre est toujours chiffrée de la même façon sur chaque substitution

1. Trouver le nombre de substitutions utilisées

- Chercher des répétitions de séquences (par exemple de 3 lettres), en faisant le pari qu'elle correspond à un même texte clair chiffré avec la même clé
- Regarder la distance entre ces répétitions en faisant le pari que c'est un multiple du nombre de substitutions
- Prendre le diviseur commun et avec une bonne probabilité, c'est le nombre de substitutions



Technique de
cryptanalyse
toujours
utilisée
aujourd'hui

2. Utiliser la cryptanalyse de César

- En faire autant qu'il y a de substitutions, en utilisant le résultat de l'étape précédente

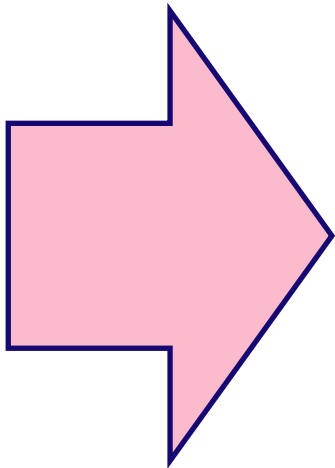
Un exemple



- Texte chiffré avec Vigenère :

X	Z	M	E	T	Y	P	P	F	G	W	P	G	N	M	V	L	X
K	Z	R	U	F	V	N	P	X	J	P	Q	G	O	I	U	D	G
K	P	R	E	P	W	S	F	M	U	P	V	C	T	X	C	D	W
G	K	X	G	X	I	T	L	M	T	P	T	Q	F	V	C	Q	J
K	C	Q	G	C	U	W	P	R	Q	F	W	E	Z	R	P	L	T
U	D	S	P	D	I	V	A	I	T	N	I	X	Z	R	U	E	S
W	E	I	U	W	I	U	Q	S	T	N	I	U	E	S	W	E	I
U	W	I	U	Z	R	F	P	W	G	E	X	Q	F	W	N	P	W
O	Z	C	G	Y	W	F	P	G	Q	X	Q	W	Y	M	E	L	X
K	Z	R	U	S	Y	D	P	V	V	C	I	G	G	I	U	L	W
V	C	S	R	S	C	U	T	G	K	P	R						

Séquence	Position	Distance	Décomposition
EIU	110-125	15	5.3
TNI	100-118	18	3.3.2
GKP	36-189	153	17.3.3
UES	106-121	15	5.3
KZR	19-163	144	3.3.2.2.2.2
QFW	82-139	144	19.3

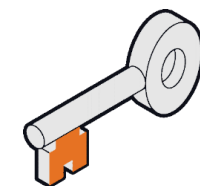


- PGCD = 3
- Taille probable de la clé : 3
- Cryptanalyse fréquentielle possible sur les 3 tranches

Système de Vernam



- Proposé en 1917 par Gilbert Vernam
- Même principe que Vigenère, dans lequel le **nombre de substitutions est la longueur du message**
- Ce n'est cependant pas suffisant !
 - Il faut que chaque substitution soit **parfaitement aléatoire**
 - Il faut que chaque ensemble de substitutions **ne soit utilisé qu'une seule fois**



Notion de masque jetable : One-Time Pad OTP

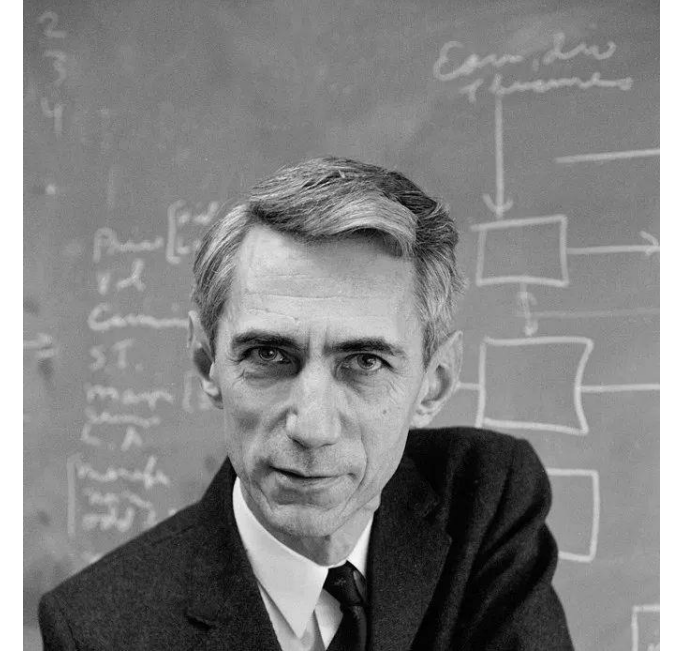
Utilisation pratique de l'OTP

$\oplus$	0	1
0	0	1
1	1	0

- En binaire, utilisation de l'opération de « ou exclusif » : $\oplus$
- Soit M un message à chiffrer et soit K un élément de la taille de M (**appelée la clé**)
- Le chiffré est alors $C = M \oplus K$
- Pour retrouver M connaissant C et K, il suffit de calculer $M = C \oplus K (= M \oplus K \oplus K = M)$



Sécurité inconditionnelle du OTP

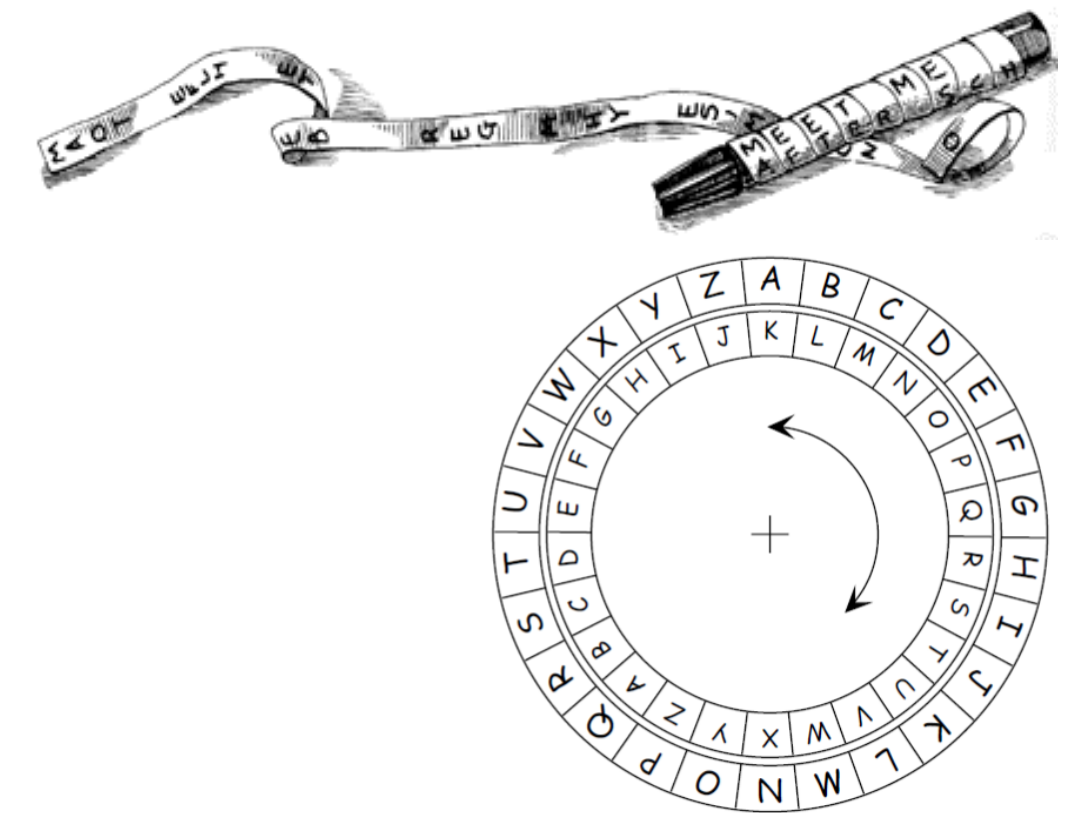


- Formulée et prouvée par **Claude Shannon** en 1949
- **Notion de sécurité inconditionnelle**
 - La connaissance du chiffré ne fournit aucune information sur le clair
 - $Pr[M = m_0 | C = c_0] = Pr[M = m_0]$ pour tout message m_0 et tout chiffré c_0
- Utilisations réelles... (?)
 - Un trésor caché...
 - Espions russes pendant la guerre froide avec des carnets rouges pas très aléatoires
 - Téléphone rouge entre Américains et Russes pendant la guerre froide

Optimalité et défauts du OTP

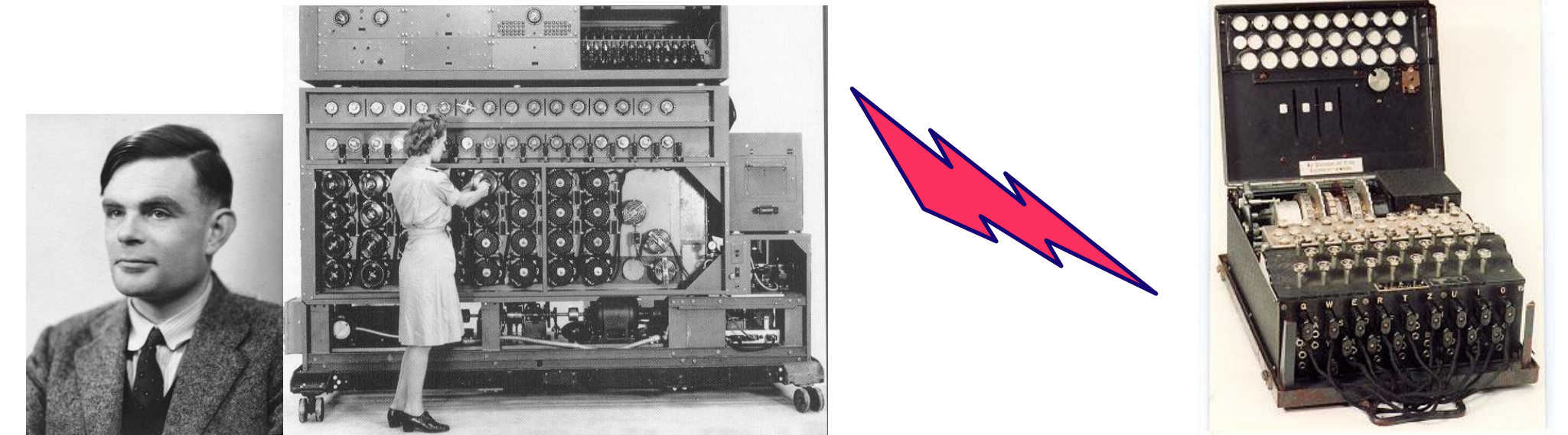
- **Optimalité**
 - Pour tout système cryptographique satisfaisant la condition de perfection de Shannon, **l'espace des clés a au moins le même cardinal que l'espace des clairs**
- **Défauts**
 - Très contraignant à utiliser
 - Il faut utiliser K pour chiffrer et pour déchiffrer
 - La valeur K doit être aléatoire, aussi longue que le message, et utilisée une seule fois
 - Ne peut pas assurer l'intégrité du message envoyé

Systèmes composés

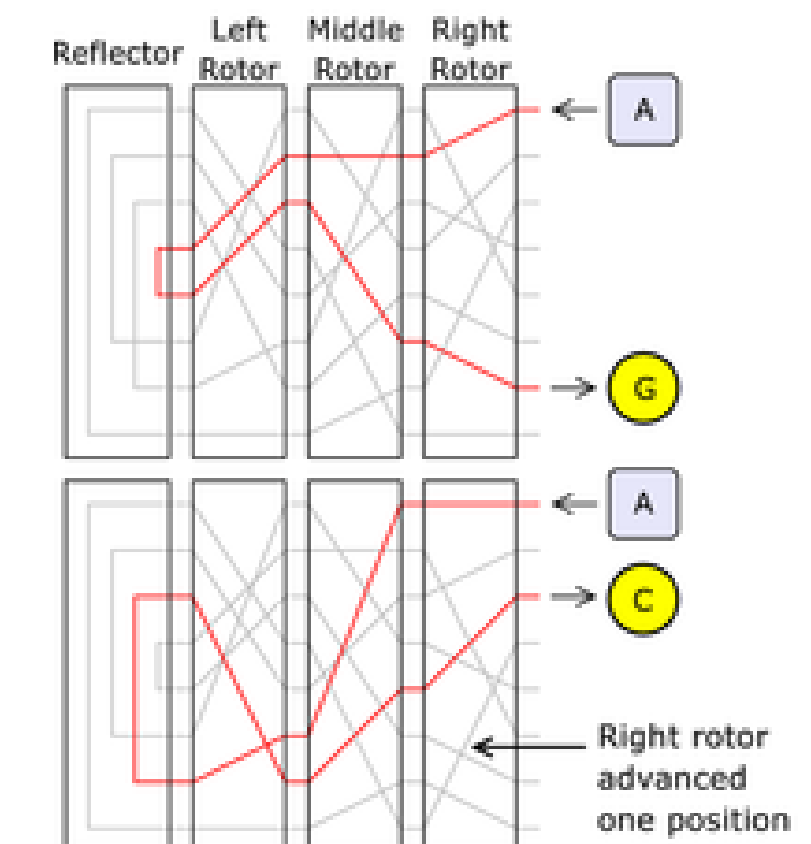
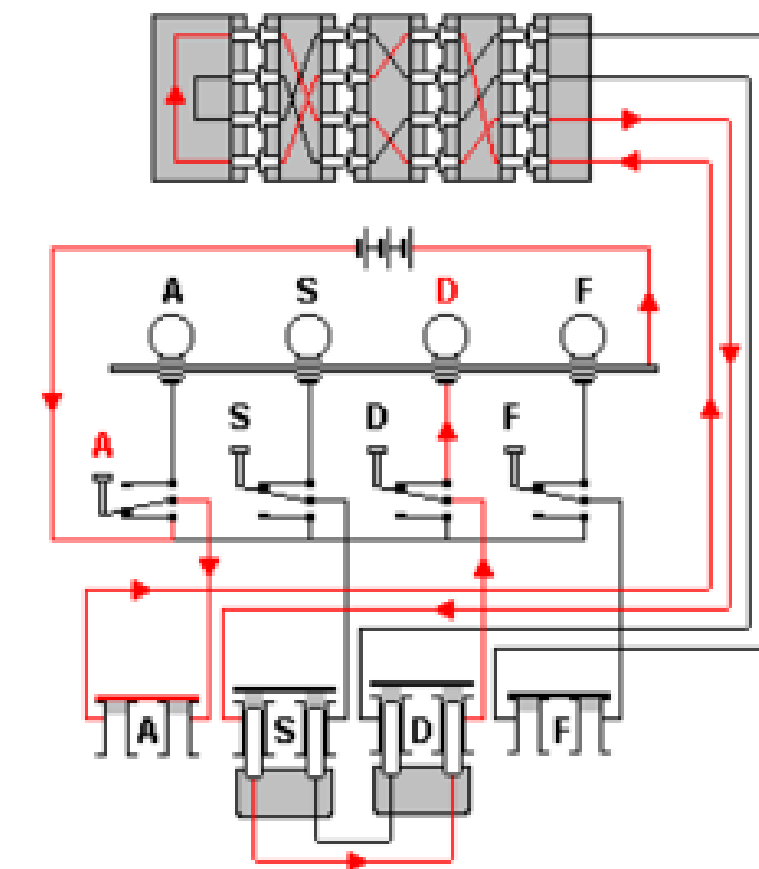


- Début du XXème siècle : utilisation majeure de systèmes composés
- Objectif double
 1. Augmenter la difficulté de la cryptanalyse en utilisant **plusieurs opérations secrètes de chiffrement** en série
 2. Ne pas trop augmenter la difficulté pour les personnes autorisées de chiffrer/déchiffrer
- Exemples : double substitution, double transposition, substitution suivie par une transposition, etc.
- Démarrage aussi de **l'âge technique**
 - Objectif : les systèmes devenant plus complexes, simplifier l'utilisation pour les humains
 - Utilisation de machines (électro) mécaniques...

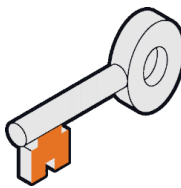
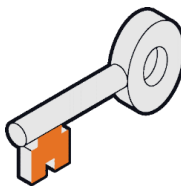
La machine Enigma



- Enigma = système **entre Vigenère et Vernam** avec une clé « aléatoire » générée mécaniquement pas une machine
 - La clé n'est pas parfaitement aléatoire
 - Le système n'est pas parfaitement sûr
- La complexité de la machine provient notamment de l'utilisation de plusieurs rotors en série (3, 4 ou 5)
- **Système cassé** par les Polonais et les Anglais avant la fin de la guerre
 - Cf. les « Bombes » d'Alan Turing
 - **Taille de la clé + redondances** de certains messages ($\neq$ cryptanalyse sur Vigenère)



Bilan de la cryptographie historique

- Quels algorithmes ?
 - OTP trop compliqué à utiliser, mais on peut l'adapter $\Rightarrow$ perte de la sécurité parfaite, mais suffisante **si c'est bien fait**
 -  Notion de chiffrement par flot
 - Principes de permutation et de substitution pas assez sûrs si utilisés seuls, mais on peut aussi les adapter $\Rightarrow$ idée de les utiliser ensemble sur des petits messages (**et de la bonne façon**)
 -  Notion de chiffrement par bloc
- Problématique de **l'échange de clé** à traiter
- Problématiques **d'authentification, de signature et de non-répudiation** à traiter
- **Démocratisation de la cryptographie** qui devient plus qu'un outil militaire

2. Concepts de la cryptographie moderne

Prémisses à la notion de clé

Système	Algorithme	Clé
Scytale	Enrouler un papier autour d'un bâton et écrire/lire le texte	Diamètre du bâton
César	Substituer chaque lettre par une autre	La substitution de A utilisée
Vigenère	Alterner successivement k substitutions différentes	Les k substitutions de A utilisées
Vernam	Utiliser Vigenère avec autant de substitutions qu'il y a de lettre dans le message à chiffrer	L'ensemble des substitutions
Enigma	Voir la description de la machine	Paramétrage initial de la machine (rotors et connecteurs)

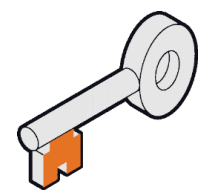
Remarque : la même clé est toujours utilisée pour chiffrer et pour déchiffrer

Notion de clé – Principes de Kerckhoffs

- Sortir de la cryptographie par l'obscurité : principes de Kerckhoffs
- Enoncé de six desiderata de la « cryptographie militaire » en 1883 !
 - Trois portent sur la facilité d'usage, un sur l'algorithme de chiffrement
 - Deux portent sur la notion de clé

« La sécurité d'un système ne doit pas être fondée sur son caractère secret »

« Seule une donnée de petite taille (clé) doit assurer la sécurité »



On préfère un système dont l'algorithme est publié qu'un système dont l'algorithme est secret

- La sécurité de la clé devient essentielle
 - Ne pas utiliser la clé trop longtemps, mais elle est moins coûteuse à changer
 - Problème de l'échange de clé
 - Petite taille $\Rightarrow$ attaque par force brute sur la clé

Quelle taille de clé pour contrer la force brute ?

- Combien d'opérations peut effectuer un ou plusieurs ordinateurs en un temps fini ?
- Un ordinateur cadencé à 1Ghz peut effectuer en 1 seconde 2^{30} opérations élémentaires
- $2^{90} = 10^{27} = 4.10^{11}$ années à 1Ghz = nombre d'opérations qu'aurait pu effectuer un ordinateur depuis le début de l'univers
- On estime aujourd'hui que l'on peut effectuer 2^{80} opérations, mais que 2^{112} , et a fortiori 2^{128} opérations ne sont pas atteignables en un temps raisonnable (moins de 100 ans)



RègleCléSym

1. Pour les clés symétriques utilisées jusqu'à la fin de l'année 2025, la taille minimale est de 112 bits.
2. La taille minimale des clés symétriques devant être utilisées à partir de 2026 est de 128 bits.



RecommandationCléSym

La taille minimale recommandée des clés symétriques est de 128 bits.

Recommandations
actuelles de l'ANSSI



Une cryptographie moderne basée sur les standards

- Le **premier standard** en cryptographie : algorithme DES (Data Encryption Standard) en 1970
- Principe général de la standardisation de la cryptographie aujourd'hui

Algorithmes cryptographiques

NIST

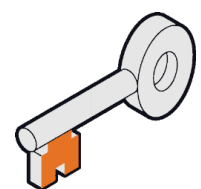


Usages de ces algorithmes



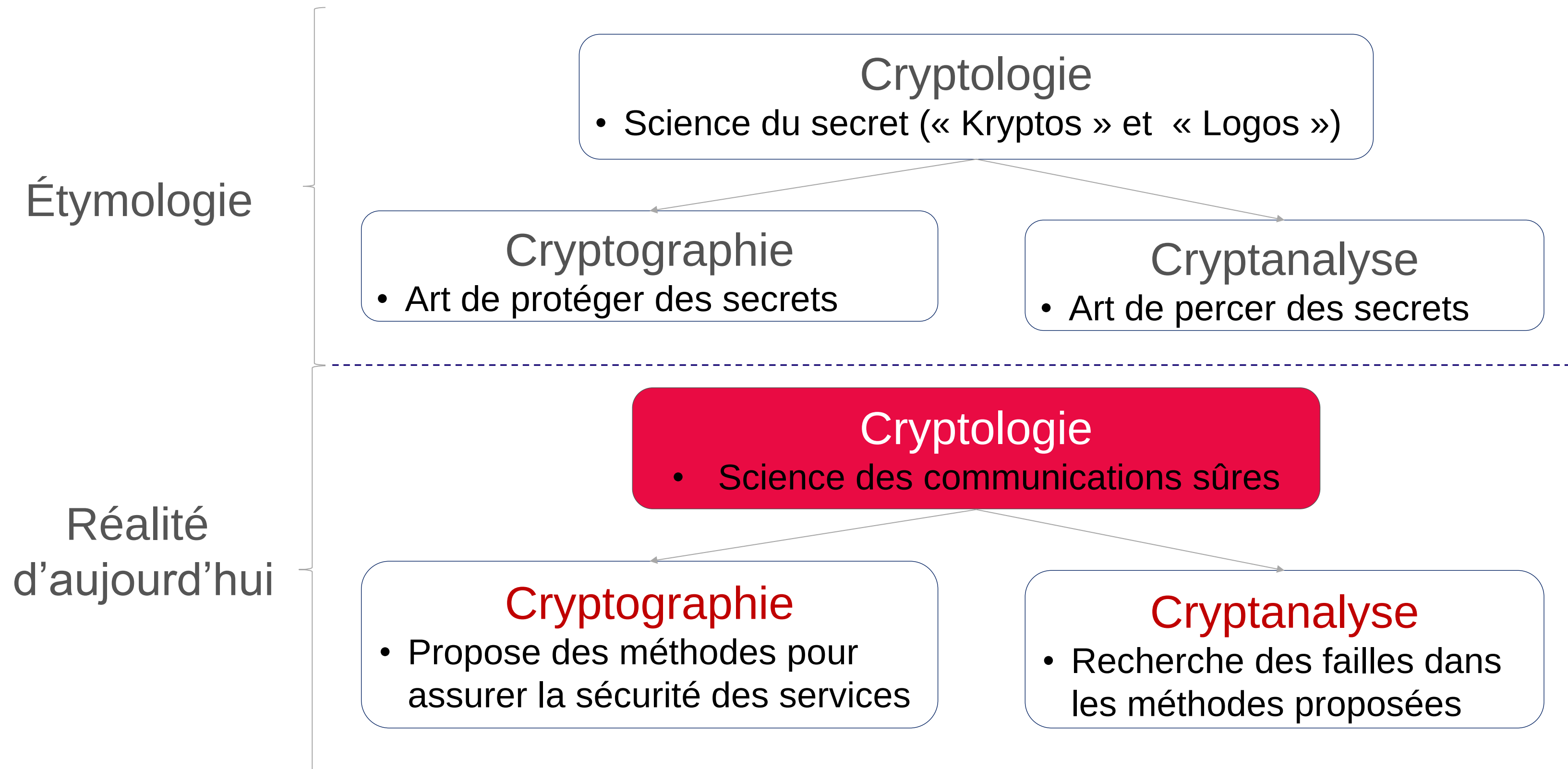
GSMA™

- Une procédure aujourd'hui assez classique...
 - **Compétition organisée par le NIST** sur plusieurs années pour définir un standard
 - Adaptation, **nouvelles propositions** et usage par les autres (ISO/IEC, IEEE, ...)
 - Il faut au moins 5 ans avant qu'un **algorithme** cryptographique ne soit **standardisé**



Toujours se référer aux standards et recommandations des instances officielles
(ANSSI, BSI, ENISA, NCSC, NSA, ...)

Nouvelles définitions



- La cryptographie est un ensemble de techniques applicables à l'échange et au traitement de l'information, permettant de garantir la **sécurité des services**
- Depuis les années 1970: discipline scientifique principalement basée sur les **mathématiques** (problèmes difficiles, analyse) et **l'informatique** (complexité, implémentation)

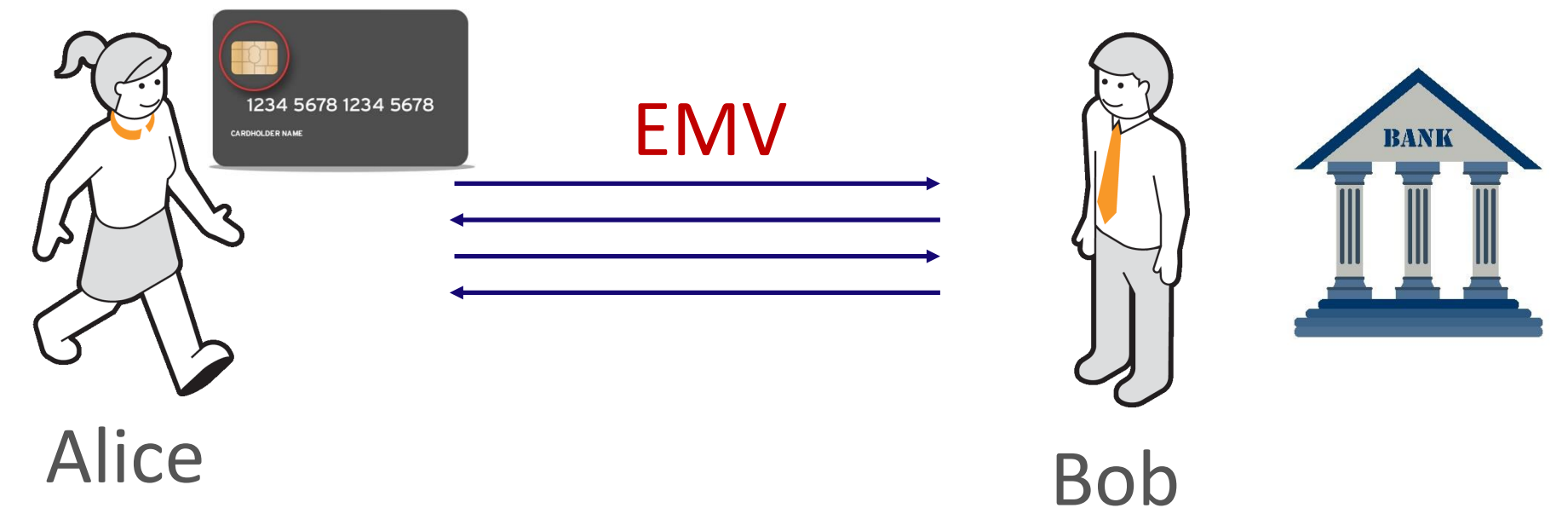
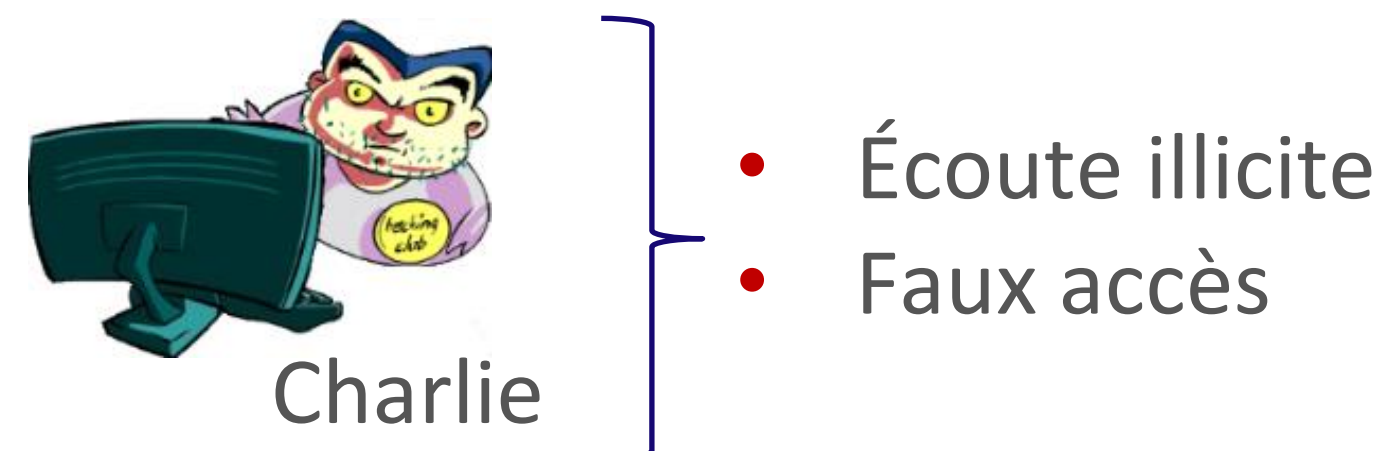
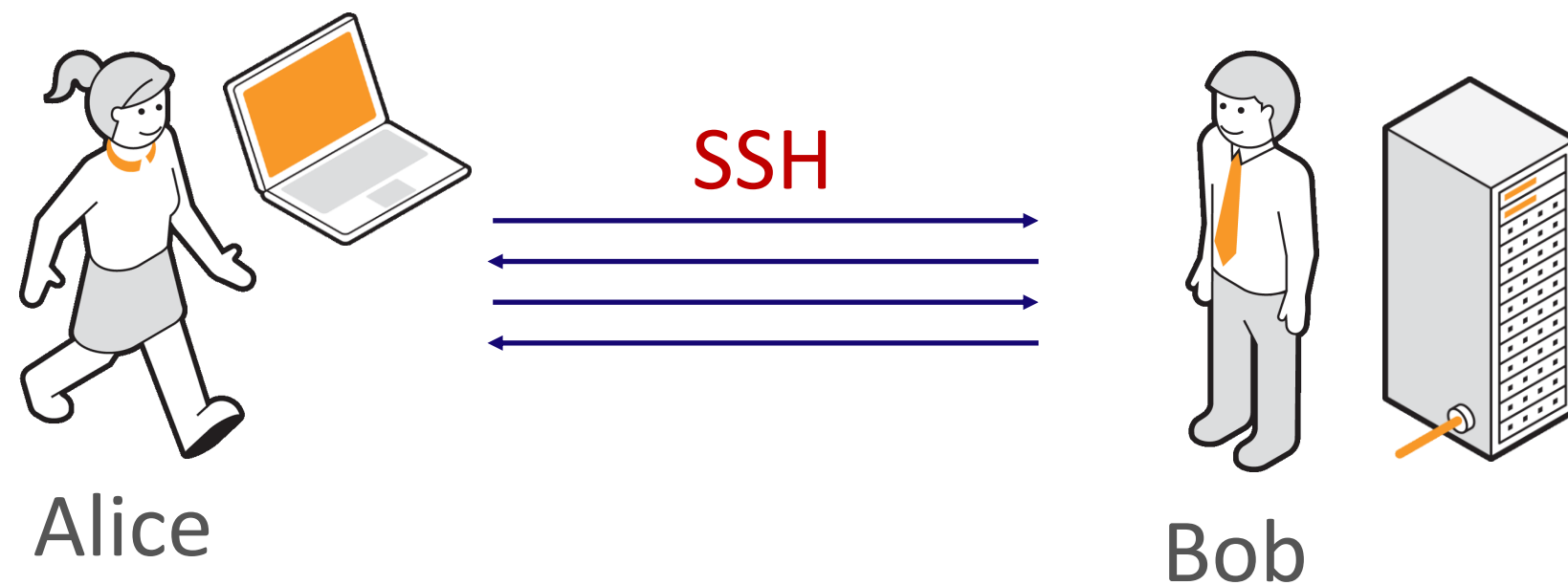
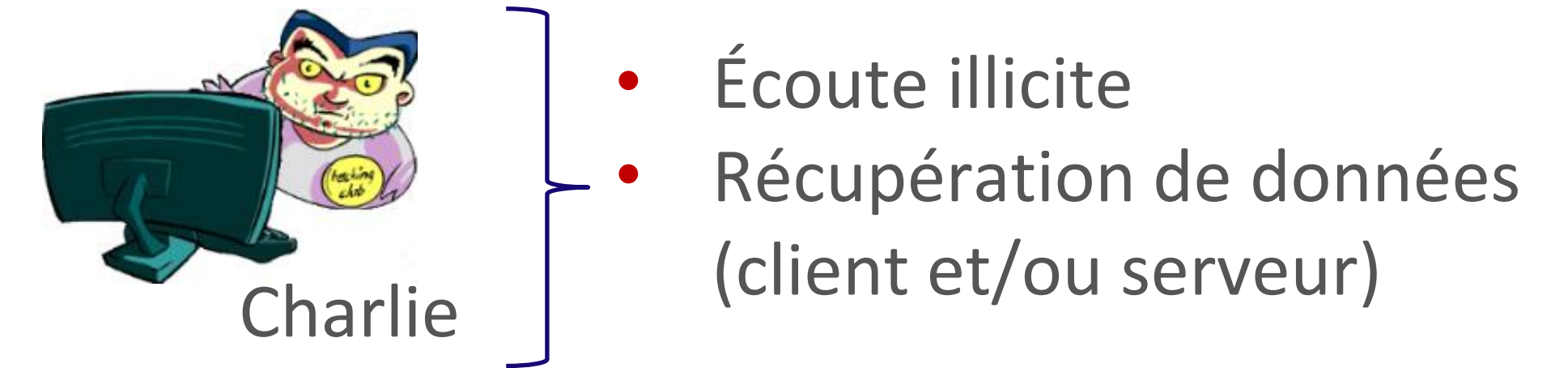
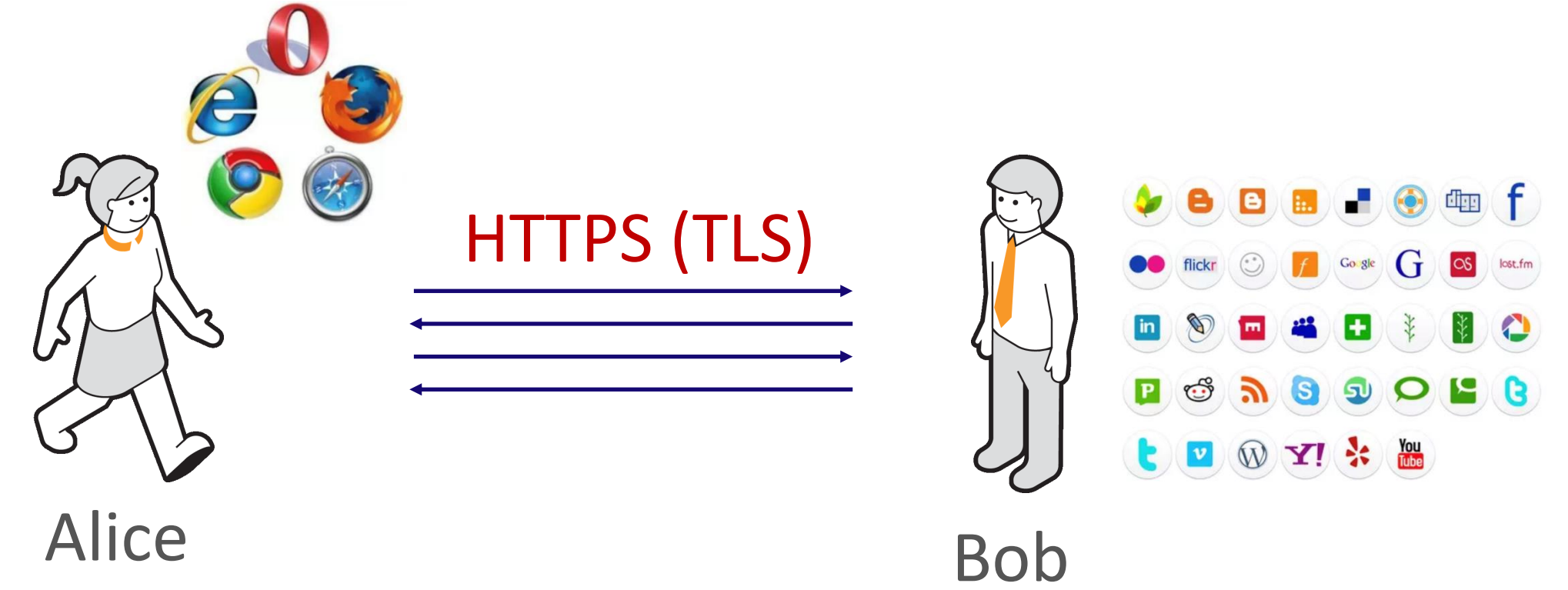
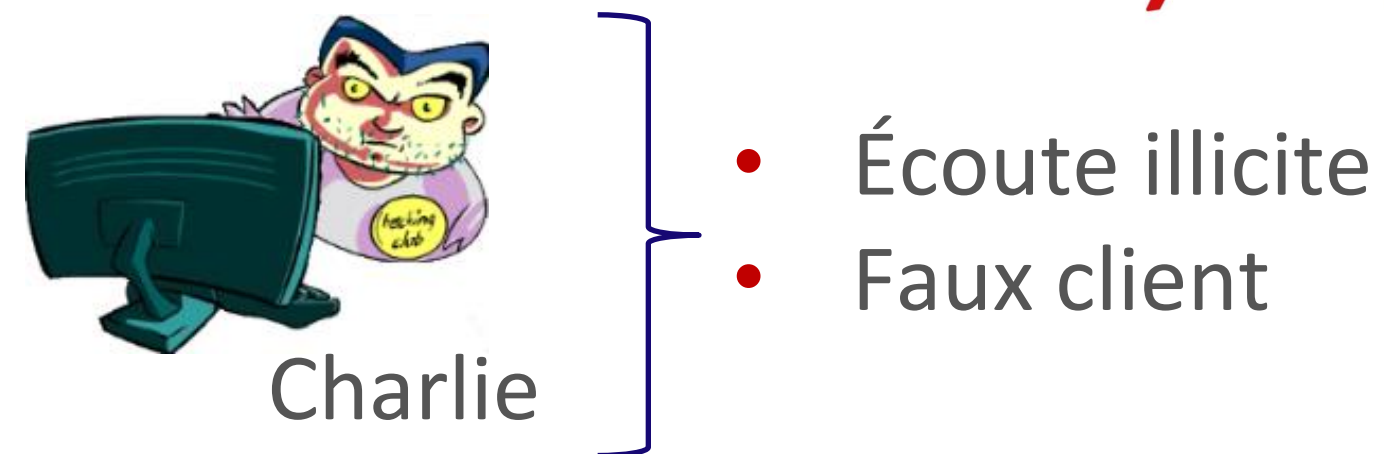
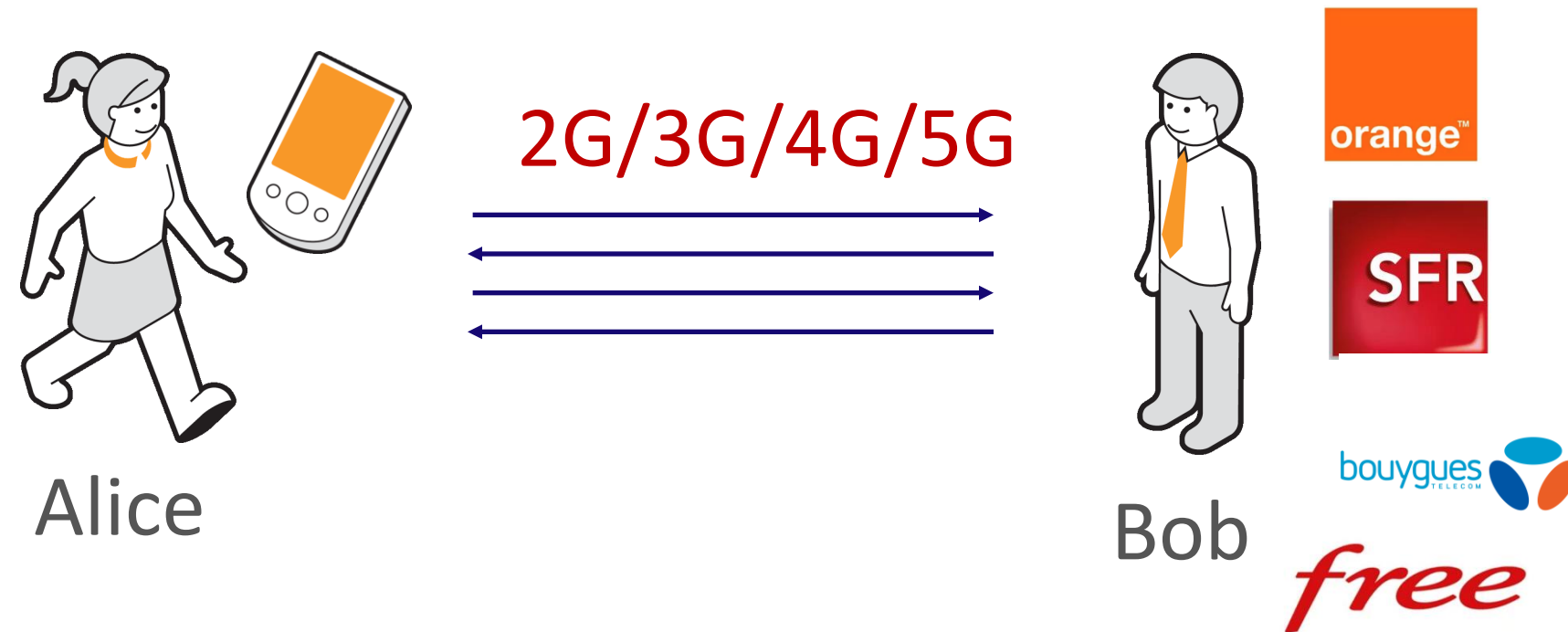
Nouveaux services de sécurité

Service de sécurité	Outil cryptographique	Usage moderne
• Confidentialité • Protection des données • Combattre l'écoute en rendant inintelligible un document	Chiffrement à clé secrète/publique	Navigateur Internet, WhatsApp,
• Intégrité • Détection de la modification d'un document par une tierce personne	Signature, fonction de hachage, code d'authentification	Proof of Work, empreinte d'un fichier téléchargé
• Non-répudiation • Une entité ne doit pas, après coup, être capable d'affirmer qu'elle n'a pas envoyé une certaine information	Signature	Carte bancaire,
• Authentification • Se convaincre que la personne que l'on a en face de soi est bien celle qu'elle prétend être • Se convaincre de la provenance d'une information reçue	Signature, code d'authentification	Sur un site Internet, carte SIM par opérateur



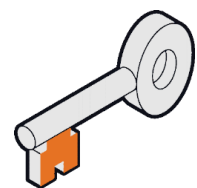
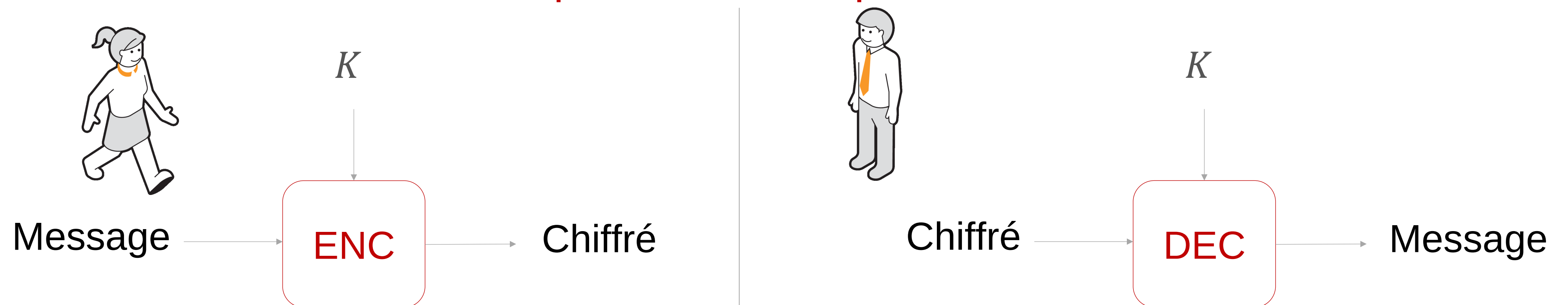
- Tout le monde ne peut pas utiliser la cryptologie totalement librement
- Il existe une réglementation en vigueur pour la commercialisation de produits intégrant des moyens de cryptologie
- Voir <https://www.ssi.gouv.fr/entreprise/reglementation/controle-reglementaire-sur-la-cryptographie/>

Quelques contextes



Concept de cryptographie à clé secrète

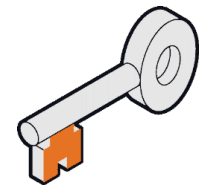
- Transformation d'un message en clair M en un message chiffré C en utilisant une clé secrète K
- Notion de cryptographie à clé secrète
 - La fonction de chiffrement doit être inversible
 - La même clé est utilisée pour chiffrer et pour déchiffrer



C'est le concept de cryptographie à clé secrète (ou cryptographie symétrique)

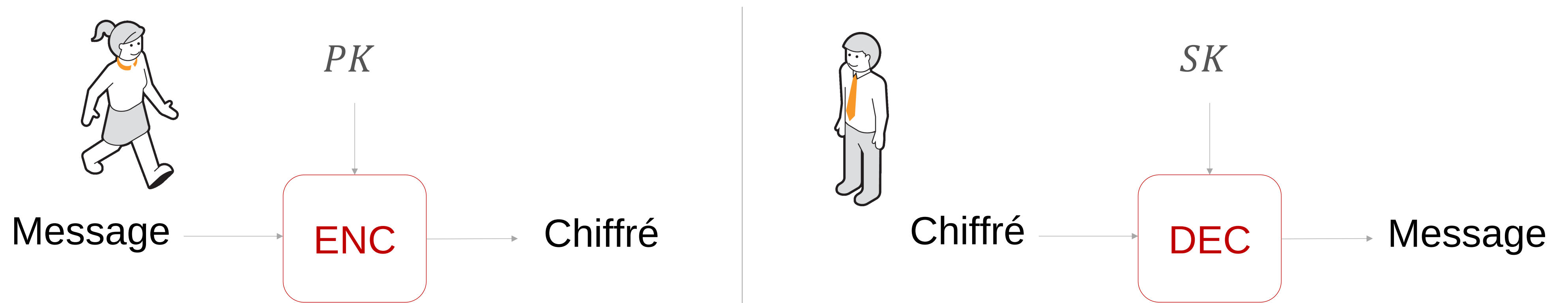
Concept de cryptographie à clé publique

- L'étape de chiffrement n'implique a priori aucune connaissance particulière
- Diffie et Hellman (1976) : « Rendre la clé de chiffrement publique »



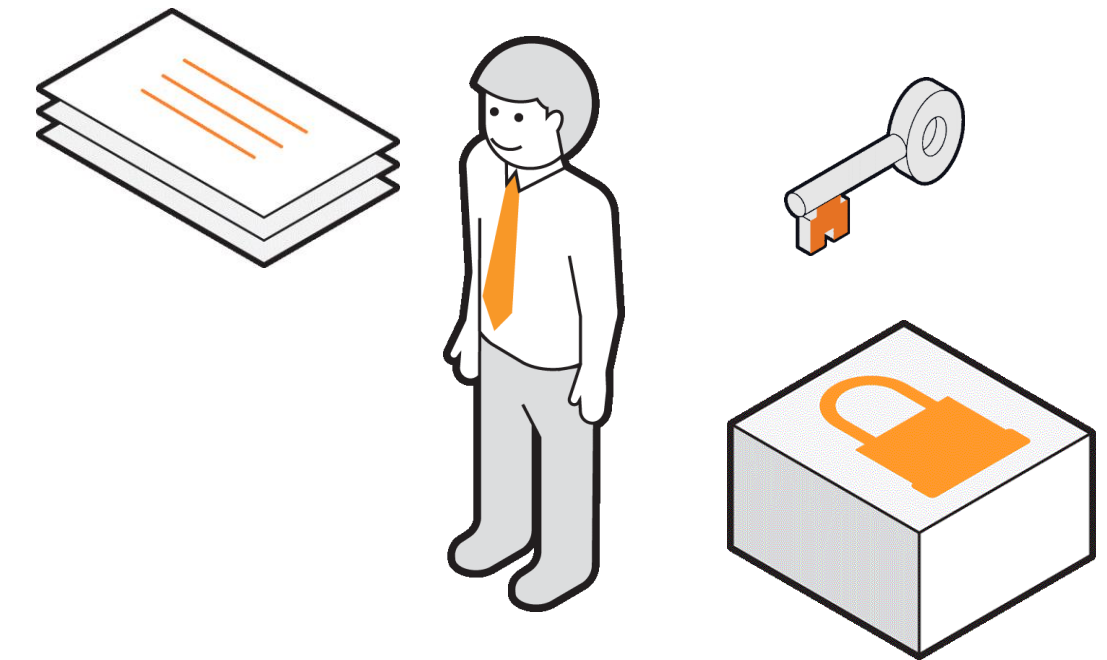
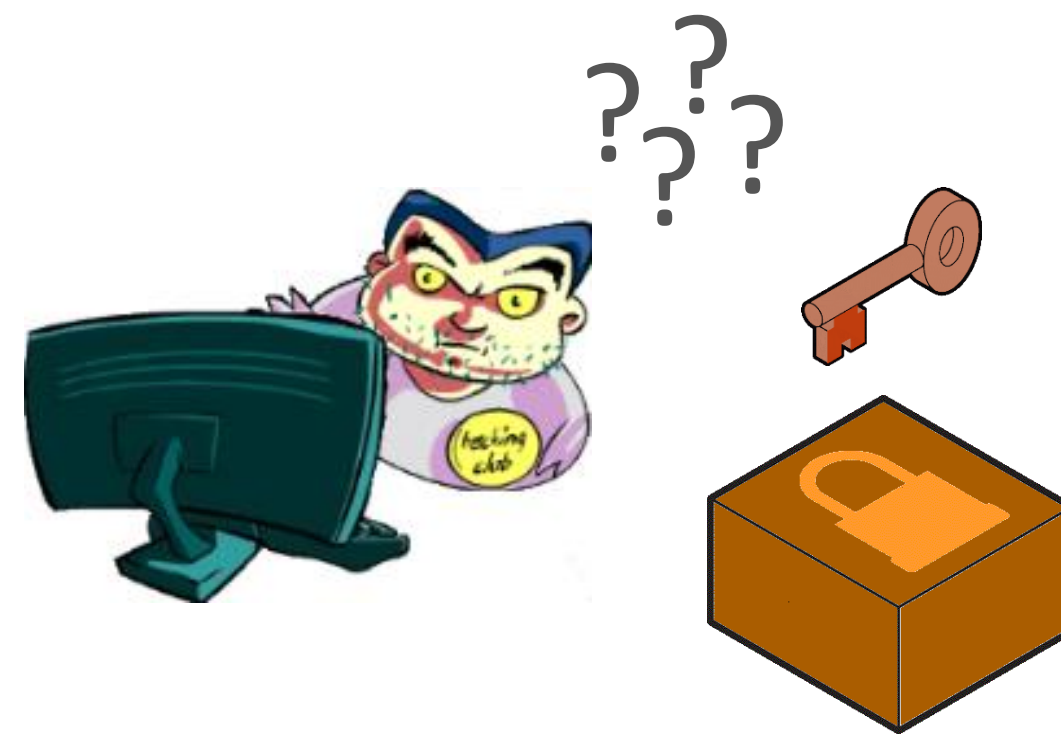
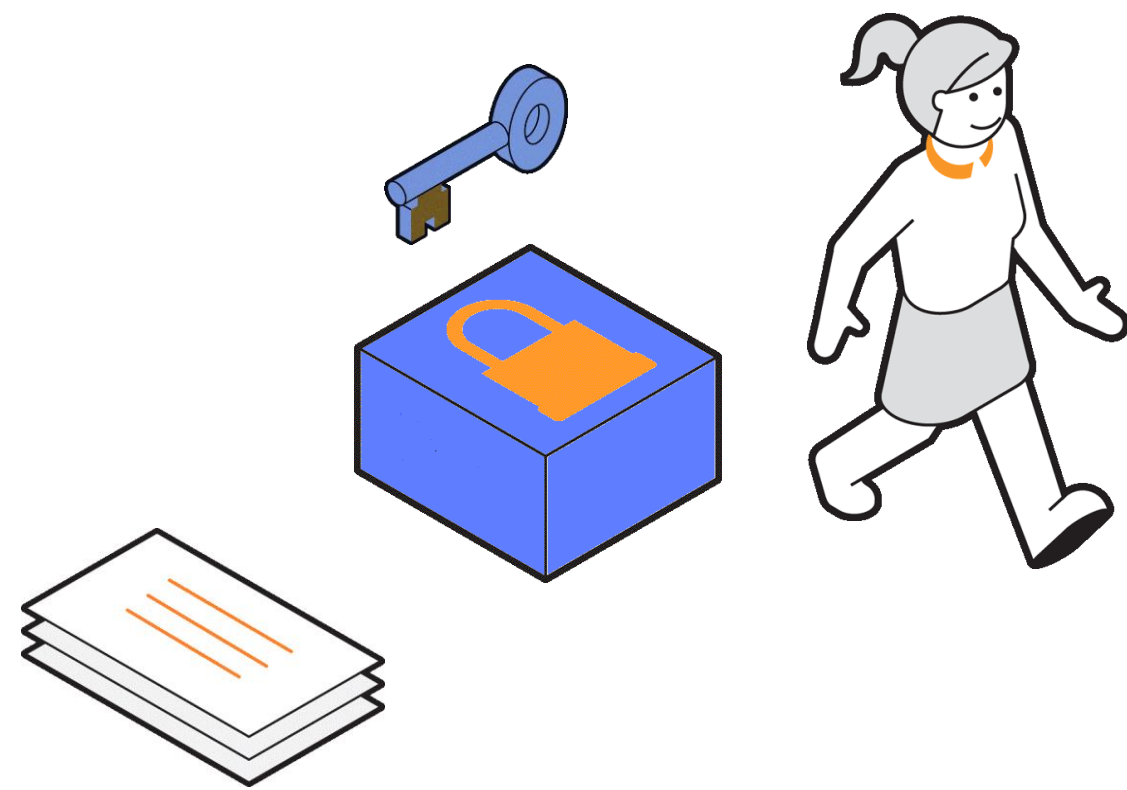
C'est le concept de cryptographie à clé publique (ou cryptographie asymétrique)

- Une clé est **publique** PK peut être utilisée par n'importe qui $\Rightarrow$ étape non sensible
- Une clé reste **secrète** SK (ou **privée**) $\Rightarrow$ étape sensible



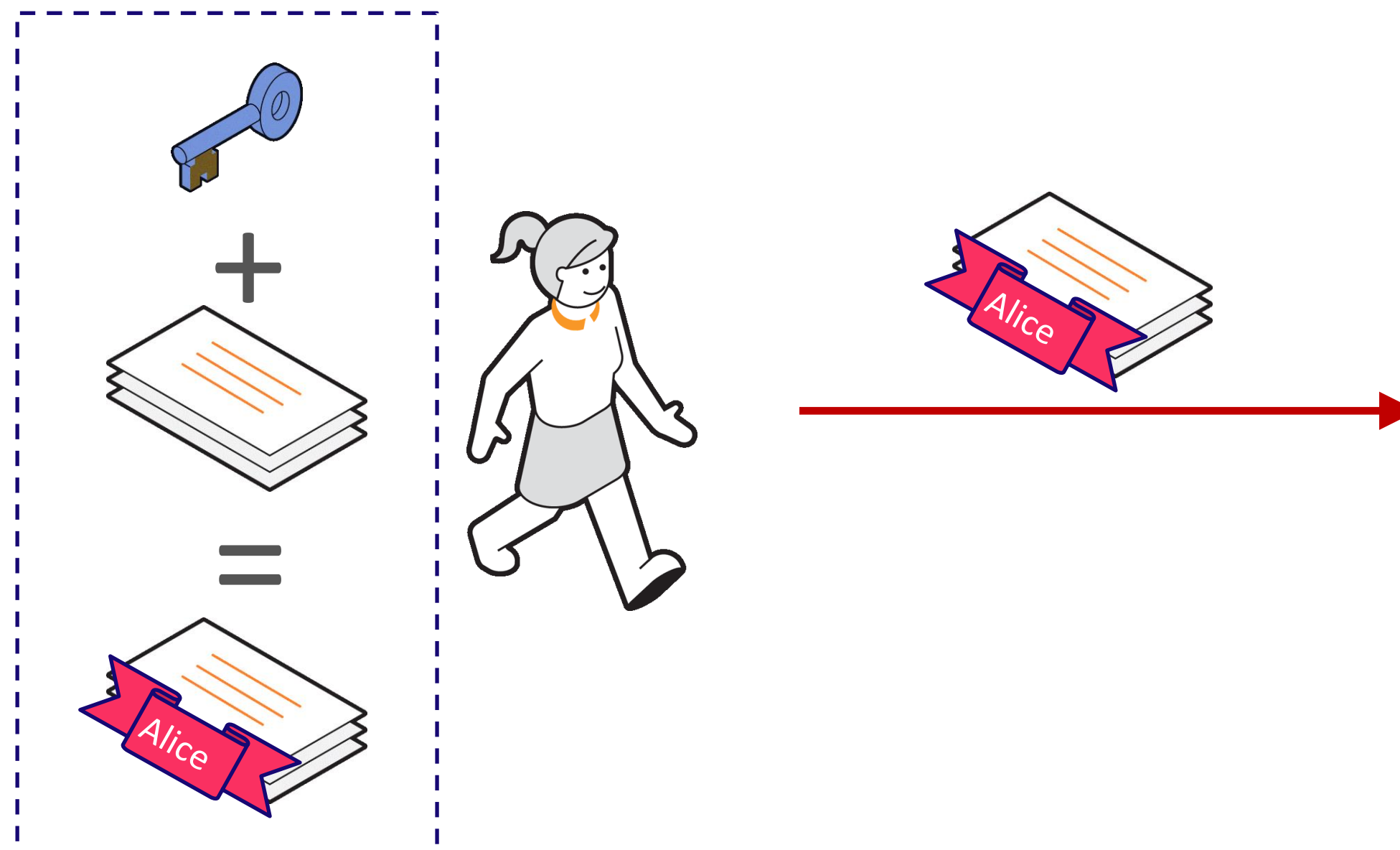
- Le fait qu'une unique entité connaisse la clé privée, et que l'autre clé devienne publique, rend possible le concept de signature numérique

Cryptographie à clé publique



C'est le concept de **cryptographie à clé publique** : la clé privée ne sort jamais de son environnement initial.

Signature numérique



- Le message provient bien d'Alice $\Rightarrow$ **Authentification**
- Le message n'a pas été modifié $\Rightarrow$ **Intégrité**
- Alice ne pourra pas le renier $\Rightarrow$ **Non répudiation**

Cryptographie à clé secrète vs. à clé publique

	Propriétés	Cryptanalyse	Avantages	Inconvénients	Usages
Cryptographie à clé secrète	• Une clé secrète pour chaque paire d'entités • Même clé pour chiffrer et déchiffrer	• Pas d'attaque meilleur que l'attaque par force brute • Cela définit la taille de clé (voir au-dessus)	• Rapide	• Beaucoup de clés à gérer • Pas de service de non-répudiation	• Chiffrement de grands volumes de données • Contrôle d'intégrité de grands volumes de données
Cryptographie à clé publique	• Une paire de clé (l'une publique et l'autre privée/secrète) pour chaque entité • Clé publique pour chiffrer, clé privée pour déchiffrer • Clé privée pour signer, clé publique pour vérifier	• Difficulté de casser un tel système repose sur un problème mathématique • Taille des clés dépend du problème mathématique	• Peu de clés à gérer • Service de non-répudiation fourni • Bonne structure mathématique	• Lent	• Distribution de clés secrètes • Signatures numériques • Cryptographie avancée

Concept de chiffrement

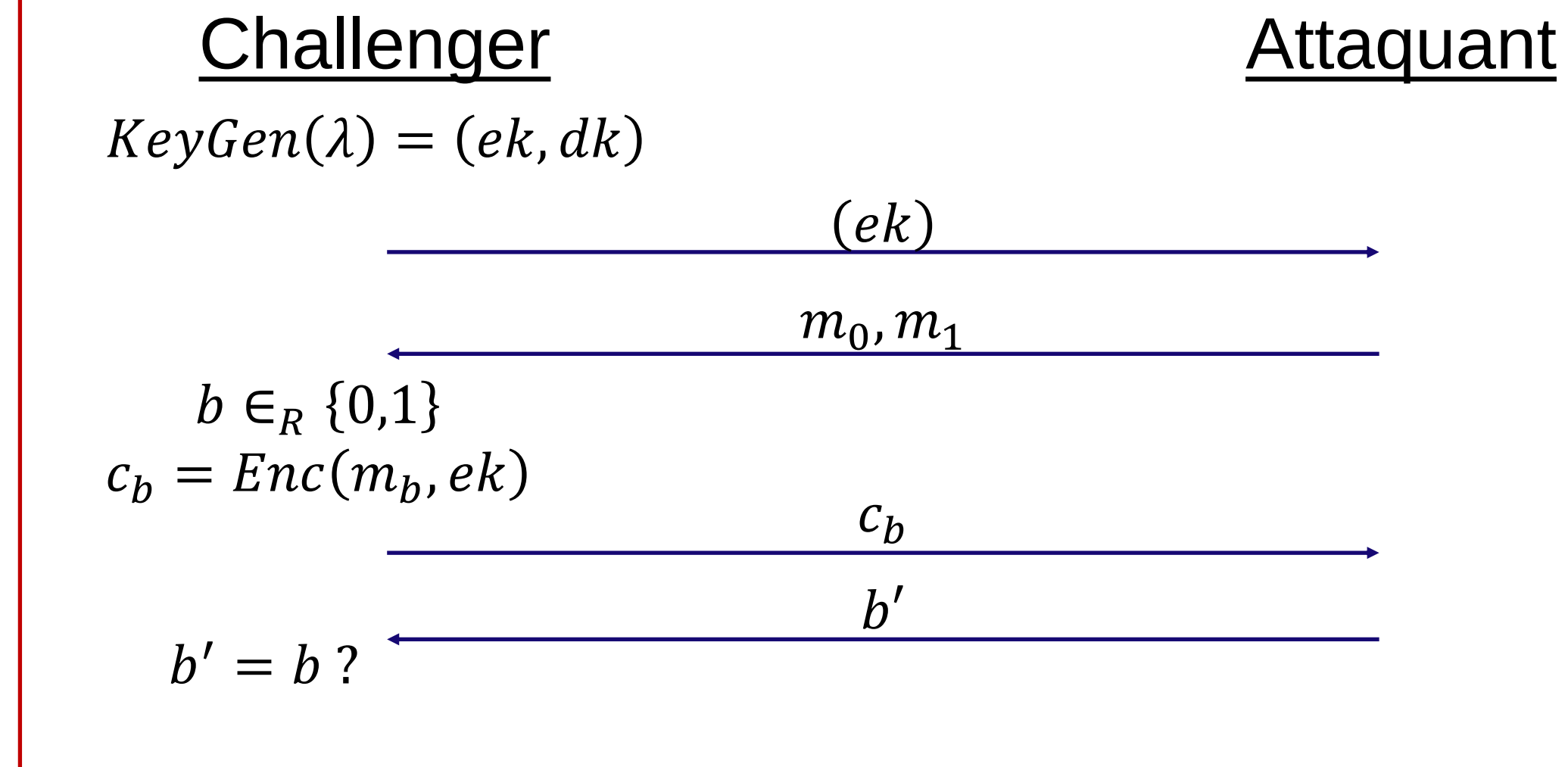
Chiffrement

- Génération des clés **KeyGen**
 - Entrée : paramètre de sécurité λ
 - Sorties : clé de chiffrement **ek** et clé de déchiffrement **dk**
- Chiffrement **Enc**
 - Entrées : message **m** et clé de chiffrement **ek**
 - Sortie : chiffré **c**
- Déchiffrement **Dec**
 - Entrées : chiffré **c** et clé de déchiffrement **dk**
 - Sortie : message **m**

- Assure la **confidentialité** des données
- Type de cryptographie
 - Chiffrement à clé secrète : $ek = dk$
 - Chiffrement à clé publique : $ek \neq dk$
- **Probabiliste** vs. Déterministe
 - Probabiliste si le chiffrement prend en entrée un aléa r
 - Cet aléa n'est pas forcément nécessaire lors de la phase de déchiffrement
- **Sécurité**
 - Chiffrement à sens unique
 - Chiffrement indistinguable

Notion de sécurité pour le chiffrement

Notion d'indistinguishabilité



- Moyens de l'attaquant
 - Texte clair choisi ($m \rightarrow c$) : uniquement en cryptographie à clé publique
 - Notion d'oracle
 - Texte clair choisi ($m \rightarrow c$) en cryptographie à clé secrète
 - Texte chiffré choisi ($c \rightarrow m$) dans les deux cas



Urgence

Concept de signature

Signature

- Génération des clés *KeyGen*
 - Entrée : paramètre de sécurité λ
 - Sorties : clé de signature sk et clé de vérification vk
 - Signature *Sign*
 - Entrées : message m et clé de signature sk
 - Sortie : signature σ
 - Vérification *Verif*
 - Entrées : message m , signature σ et clé de vérification vk
 - Sortie : bit $0/1$
- Assure **authentification** de la source, **intégrité** et **non-répudiation**
 - Uniquement possible en **cryptographie à clé publique** : $sk \neq vk$
 - Toujours probabiliste
 - **Sécurité**
 - Non falsification

Notion de sécurité pour la signature

- Notion de non-falsification
 - Connaissant vk , un attaquant ne doit pas être capable de sortir un couple (m, σ) tel que
$$\text{Verif}(m, \sigma, vk) = 1$$
- Moyens de l'attaquant
 - Texte choisi ($m \rightarrow \sigma$)

Concept de code d'authentification

Code d'authentification

- Génération des clés *KeyGen*
 - Entrée : paramètre de sécurité λ
 - Sortie : clé partagée *sk*
- Génération du code *MAC*
 - Entrées : message *m* et clé partagée *sk*
 - Sortie : code σ
- Vérification du code *MACVerif*
 - Entrées : message *m*, code σ et clé partagée *sk*
 - Sortie : bit *0/1*

- Assure **authentification** de la source et **intégrité**
- **N'assure pas la non-répudiation**
- Uniquement pertinent pour la cryptographie à clé secrète
- Toujours probabiliste
- **Sécurité**
 - Non falsification

Concept de fonction de hachage

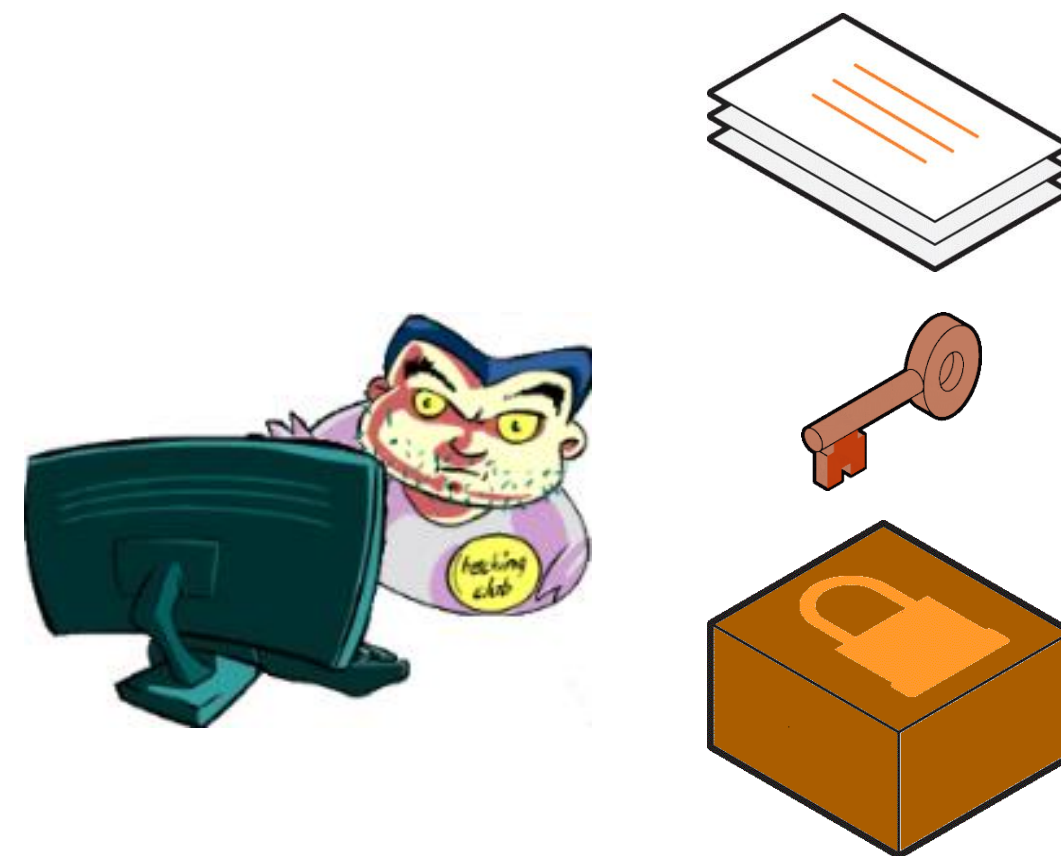
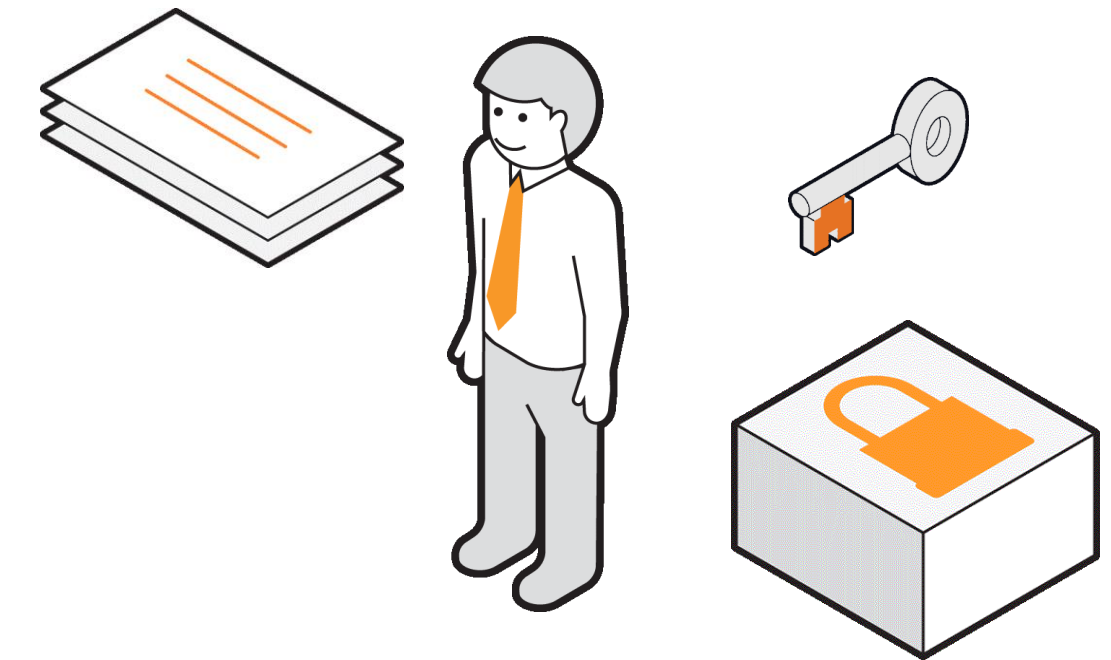
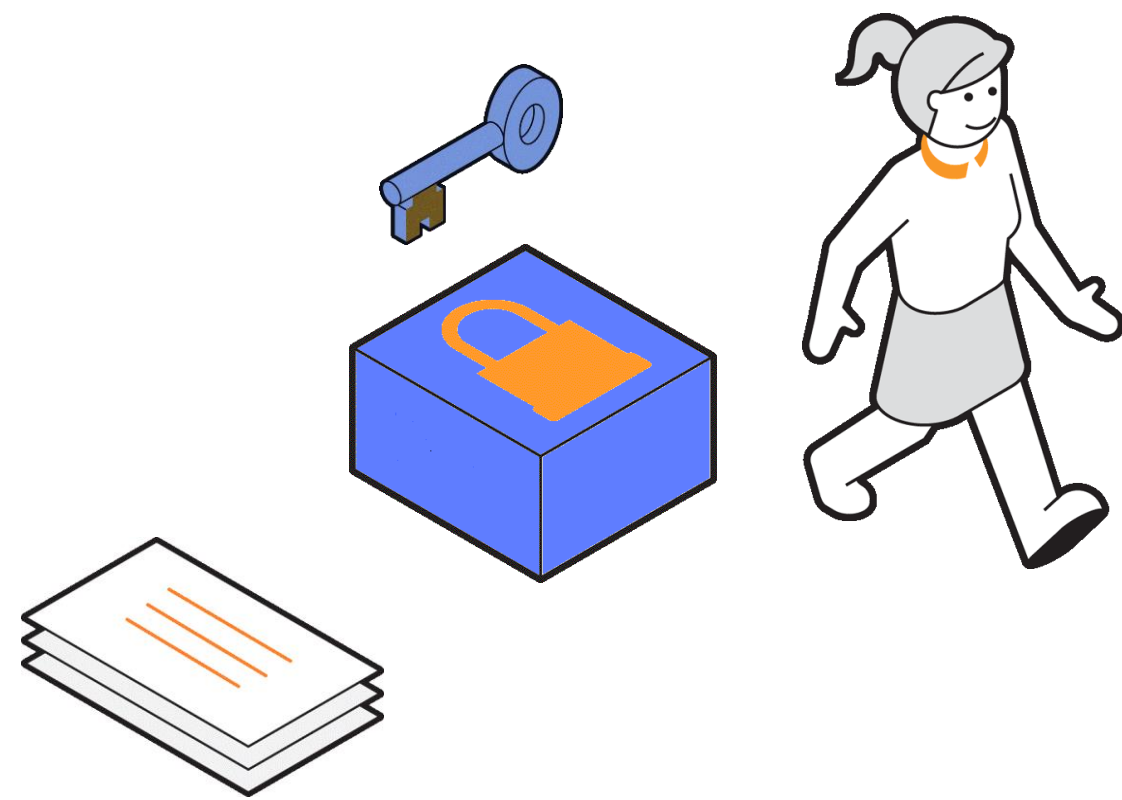
Fonction de hachage

- Génération du condensé *Hash*
 - Entrées : message *m*
 - Sortie : condensé *h*
- Vérification du condensé *HashVerif*
 - Entrées : message *m*, condensé *h*
 - Sortie : bit *0/1*

- Assure l'intégrité
- N'assure pas l'authentification de la source
- Utilisé dans les signatures numériques
- Toujours déterministe
- Sécurité
 - Résistance à la pré-image
 - Résistance à la seconde pré-image
 - Résistance aux collisions

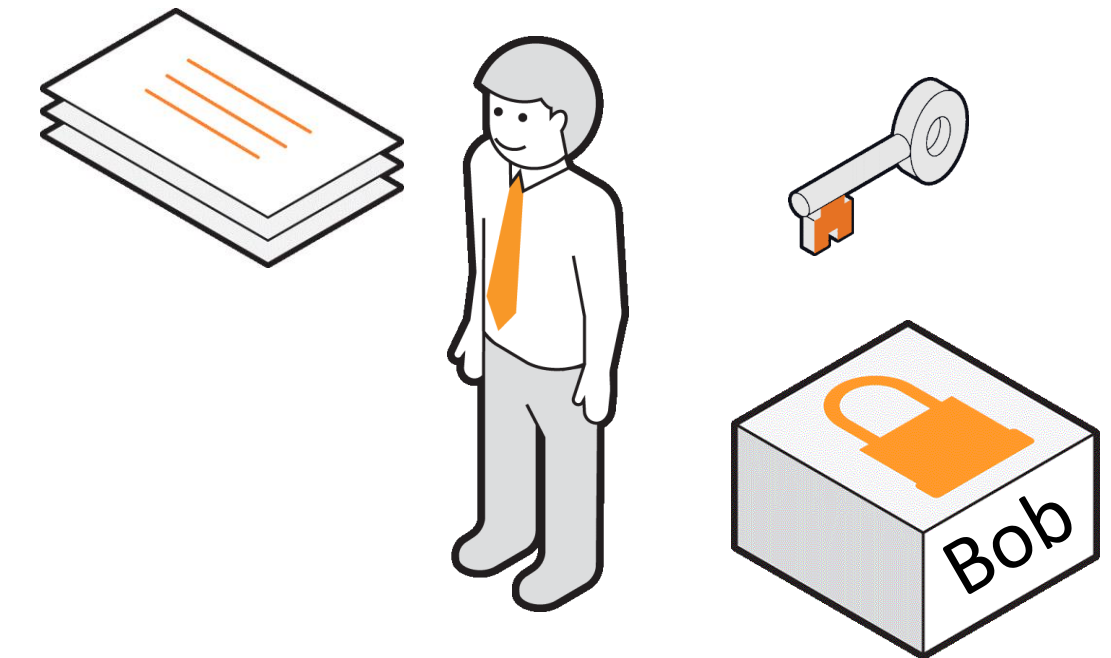
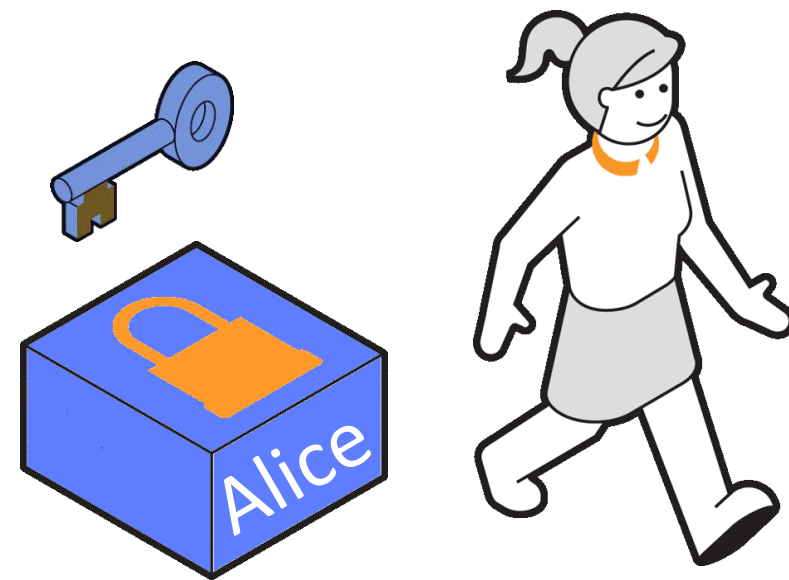
3. Certification numérique

Cryptographie à clé publique



Attaque de
l'homme du
milieu

Cryptographie à clé publique

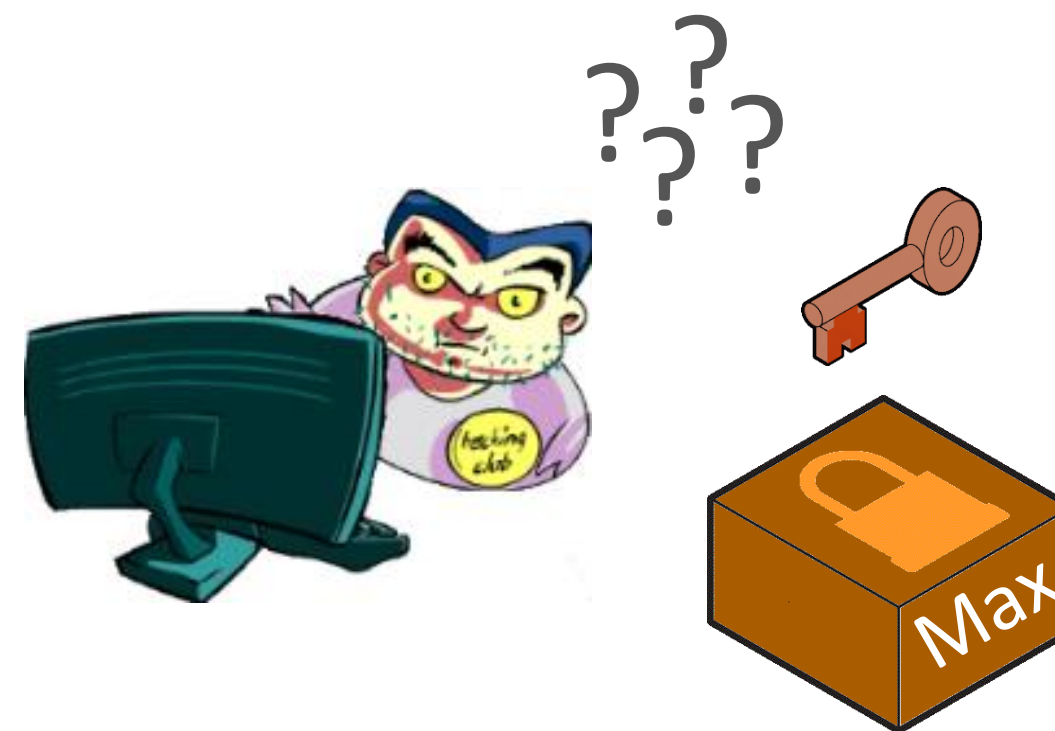


- Une autorité **certifie** le fait que la clé publique appartient bien à la bonne personne



La clé
publique
d'Alice est
 pk_A

Certified
By Authority



Concept de certification

- La **cryptographie à clé publique** ne peut fonctionner que si les interlocuteurs d'Alice sont certains que la clé publique appartient bien à Alice
- Problème de l'**intégrité d'une clé publique**
- Faute d'une telle certitude, un usurpateur peut se faire passer pour Alice.
- Problème : Comment fournir la preuve infalsifiable de la légitimité de cette clé ?

Notion de certificat numérique

- Associer un individu à une clé publique, avec un niveau de preuve satisfaisant
- Vérification hors-ligne de la validité d'un certificat : il n'y a personne en ligne qui peut
- Interaction minimum avec l'utilisateur
- Gestion simple en cas de mouvement de personnel

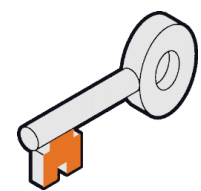
Principe du Certificat

- Une autorité **certifie** le fait que la clé publique appartient bien à Alice



La clé
publique
d'Alice est
 pk_A

Certified
By Authority



Notion d'infrastructure à clé
publique (*Public Key
Infrastructure PKI*)

- **Signature numérique**

- La signature est calculée avec la clé privée
- La clé publique associée est certifiée par une autorité
- Bob doit vérifier la validité du certificat, puis la validité de la signature

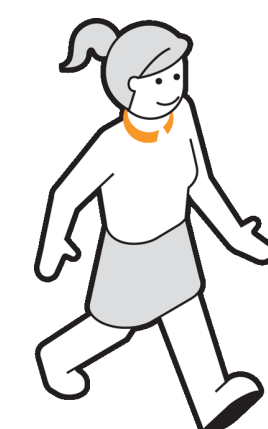


Signature + Certificat



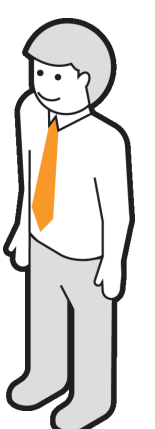
- **Chiffrement**

- Le certificat contient la clé publique et est certifiée par une autorité. Il doit être vérifié par Bob avant usage
- Le chiffrement est calculé avec la clé publique
- Alice déchiffre en utilisant la clé privée associée



Certificat

Chiffré



Un standard de certification : X.509

- **Présentation**

- Format standard de certificat
- Défini par l'Union Internationale des Télécommunications (UIT)

- **Principe d'un certificat X.509**

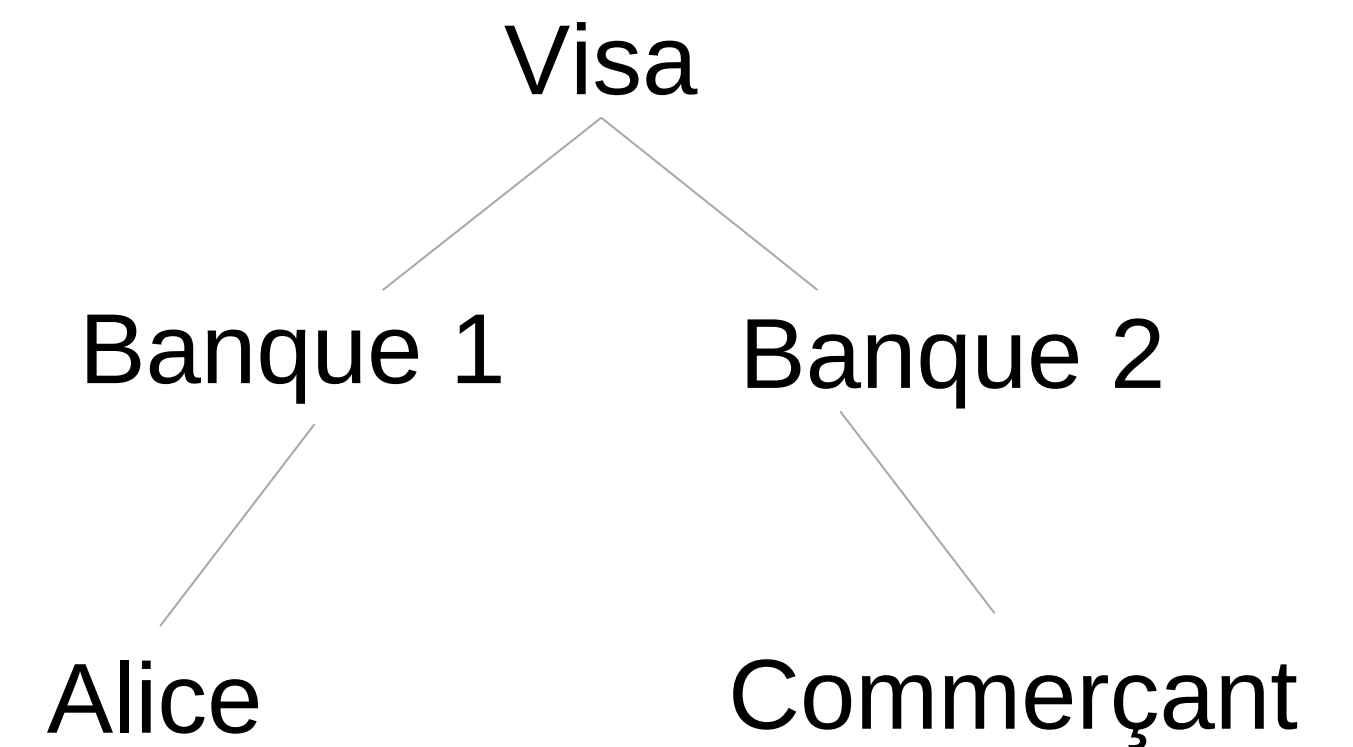
- Contient une **liste de données**
- Contient **la clé publique**
- Est **signé par une autorité de confiance**

- Exemple avec https...

- Numéro de version X509
- Numéro de série
- Algorithme de signature
- **Émetteur**
- **Sujet**
- **Validité**
- Infos sur la clé publique du sujet
- Algorithme utilisé par le sujet
- **Valeur de la clé publique**
- Identification unique émetteur
- Identification unique sujet
- Extensions

Autres éléments de la norme X.509

- Les chaînes de certificat
 - Problème : il faut faire confiance à l'autorité
 - Solution :
 - On fait forcément confiance à quelqu'un
 - Chaîne de certification
 - Mise en place d'une hiérarchie
- La révocation
 - Cause de révocation: décès, faute, etc.
 - Principe : liste des clés publiques révoquées, vérification par Bob



A retenir jusque-là



- La notion de clé cryptographique associée aux principes de Kerckhoffs et la taille de la clé par rapport à l'attaque par force brute
- Les similitudes et différences, les avantages et les inconvénients des deux types de cryptographie
- Les différents outils cryptographiques et le service de sécurité qu'ils proposent
- La nécessité de s'appuyer sur des standards et de regarder les préconisations des agences de sécurité
- Le secret parfait et les défauts du système de Vernam (OTP)
- La nécessité de s'appuyer sur une certification pour que la cryptographie à publique soit considérée comme sûre

4. Chiffrement par blocs

Deux grandes catégories

Chiffrement par blocs

- *Block cipher* en anglais
 - Idée : mélanger les permutations et les substitutions sur des petites données
- Le message est traité par blocs de données de taille fixe
 - Taille 64 ou 128 bits
 - Exemples : AES, Camellia, 3DES, Kasumi, Blowfish, ...
- Débit moyen

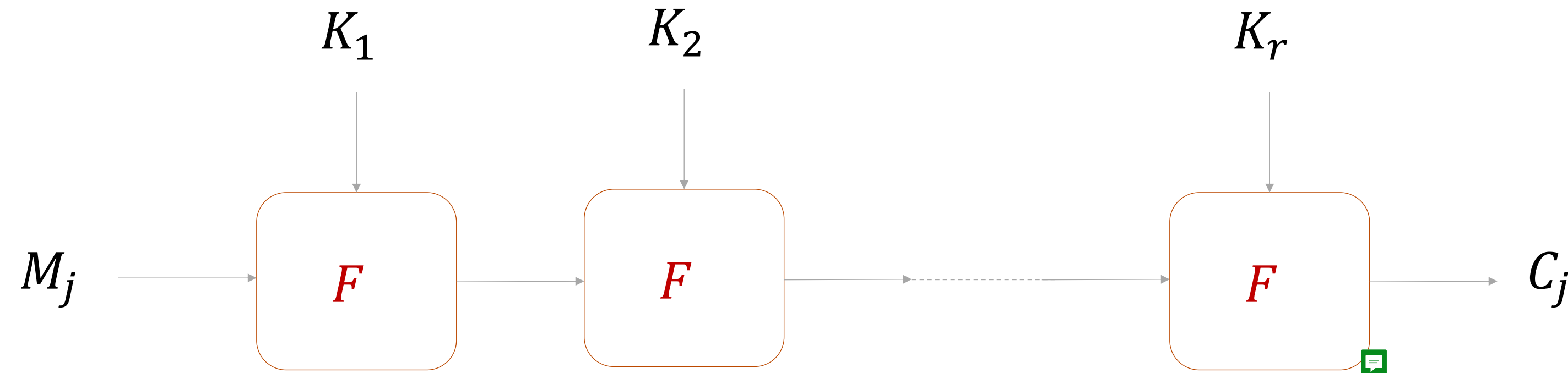
Chiffrement par flot

- *Stream cipher* en anglais
 - Idée : générer une clé aussi longue que le message de façon pseudo-aléatoire et utiliser le One-time-pad
- Le message est traité bit par bit
 - Exemples : HC-128, Salsa20/20, ChaCha, SNOW 2.0/3G, SOSEMANUK, ...
- Débit élevé

Principe général du chiffrement par blocs



- Fonction $E: \{0,1\}^n \times \{0,1\}^k \rightarrow \{0,1\}^n$ où k est la taille de la clé K , n est la taille du bloc
- Message $M = M_1 \parallel M_2 \parallel \dots \parallel M_N$
- Chiffré final C composé des $C_j = E(M_j, K)$ ($j \in [1, N]$)
- Construction classique : **fonction E itérative** avec une **fonction interne F** , r tours, chacun utilisant une **sous-clé K_i** ($i \in [1, r]$) **dérivée de K**



- La fonction F est parfois notée F_{K_i}
- Pour déchiffrer, le chiffrement doit être inversible, et les fonctions F_{K_i} bijectives

Principaux paramètres de sécurité

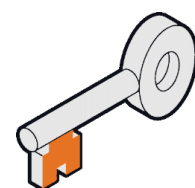
- Taille n du bloc
 - Une taille trop petite rend des attaques élémentaires (constitution de dictionnaires) plus efficaces en pratique que la recherche de la clé secrète
- 

RègleBlocSym

 1. Pour une utilisation jusqu'à la fin de l'année 2025, la taille minimale des blocs des mécanismes de chiffrement par bloc est de 64 bits.
 2. À partir de l'année 2026, la taille minimale des blocs des mécanismes de chiffrement par bloc est de 128 bits.
- Systèmes utilisés via des modes opératoires permettant de chiffrer des messages de taille quelconque ou bien de calculer des codes d'authentification de message (nous verrons ça plus tard)
- Taille k de la clé
 - Un algorithme sera considéré comme bon si la meilleure attaque doit coûter 2^k opérations (pas mieux que l'attaque par force brute)

Principes fondamentaux de la cryptographie à clé secrète moderne

- Proposés par Claude **Shannon**, et largement utilisés aujourd'hui
- La **confusion** vise à cacher n'importe quelle structure algébrique dans le système
 - Assurée par une **substitution non-linéaire** (on parle souvent de boîte-S)
 - Il doit s'agir de la seule étape non-linéaire, afin de gagner en efficacité
- La **diffusion** permet à chaque bit du message d'influer une grande partie du texte chiffré
 - La modification d'un bit en entrée modifie de nombreux bits en sortie
 - Assurée par une **permutation linéaire**

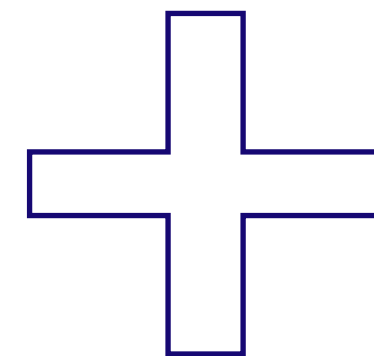


Toujours utilisé aujourd'hui dans le design de systèmes cryptographiques

Construction générique

Construction : combiner substitutions et permutations sur plusieurs tours

1. OTP avec une clé dépendante du tour : $M \oplus K_i$
2. Découper le résultat de n bits en plusieurs sous-chaînes de longueur l et exécuter la substitution non-linéaire $\pi_S: \{0,1\}^l \rightarrow \{0,1\}^l$
3. Recoller les sous-chaînes et appliquer la permutation $\pi_P: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$
4. Recommencer avec le message résultant, et une nouvelle clé de tour



Déchiffrement : opérations inverses, en utilisant les mêmes clés de tour

Historique rapide du chiffrement par blocs

- 1970-2000 : DES (*Data Encryption Standard*) et quelques autres algorithmes (FEAL, IDEA, RC5, ...)
- D'autres algorithmes parfois dédiés à des contextes spécifiques (RC6, CAMELLIA, KASUMI, ...)
- Depuis 2000 : AES (*Advanced Encryption Standard*)

Historique du DES

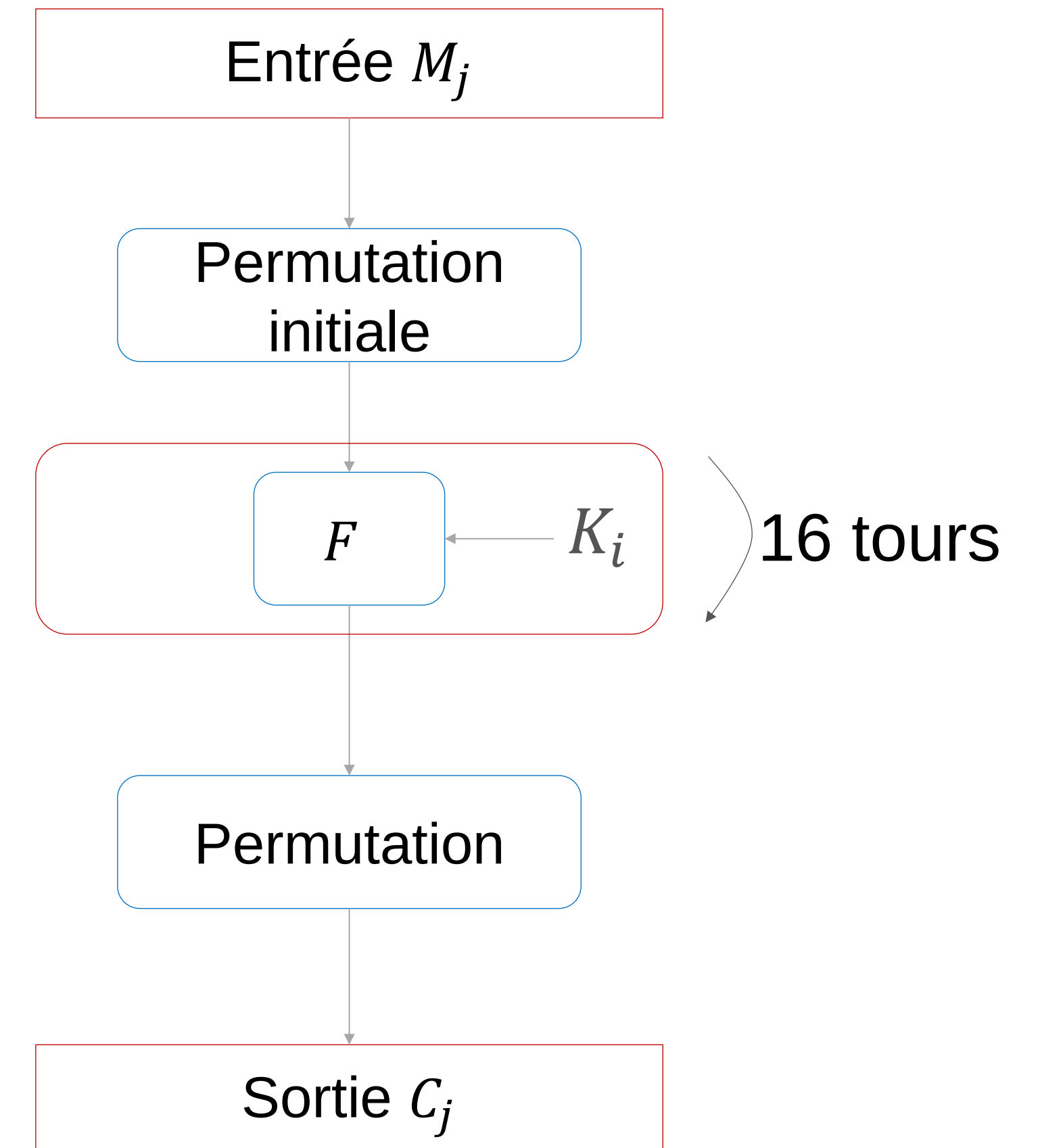
- 1971 : Création de l'algorithme LUCIFER par Horst Feistel (IBM)
- 1973 : Appel d'offre du National Bureau of Standards (NBS) aux USA
- 1974 : LUCIFER est proposé, mais la NSA demande à faire des modifications
 - Suspensions... Mais finalement pas si clair
- 1975 : Proposition publiée pour commentaire
- 1977 : DES est adopté norme FIPS PUB 46
- 1981 : DES adopté comme norme ANSI
- 1990 : Attaques théoriques de Biham-Shamir
- 2000 : DES en fin de vie, remplacement par AES

Caractéristiques de DES

- Taille des blocs $n = 64$
- Taille de la clé $k = 56$ (+8 bits de parité), les clés intermédiaires (notées K_i) font 48 bits
- Nombre de tours $r = 16$

permutation
initiale =

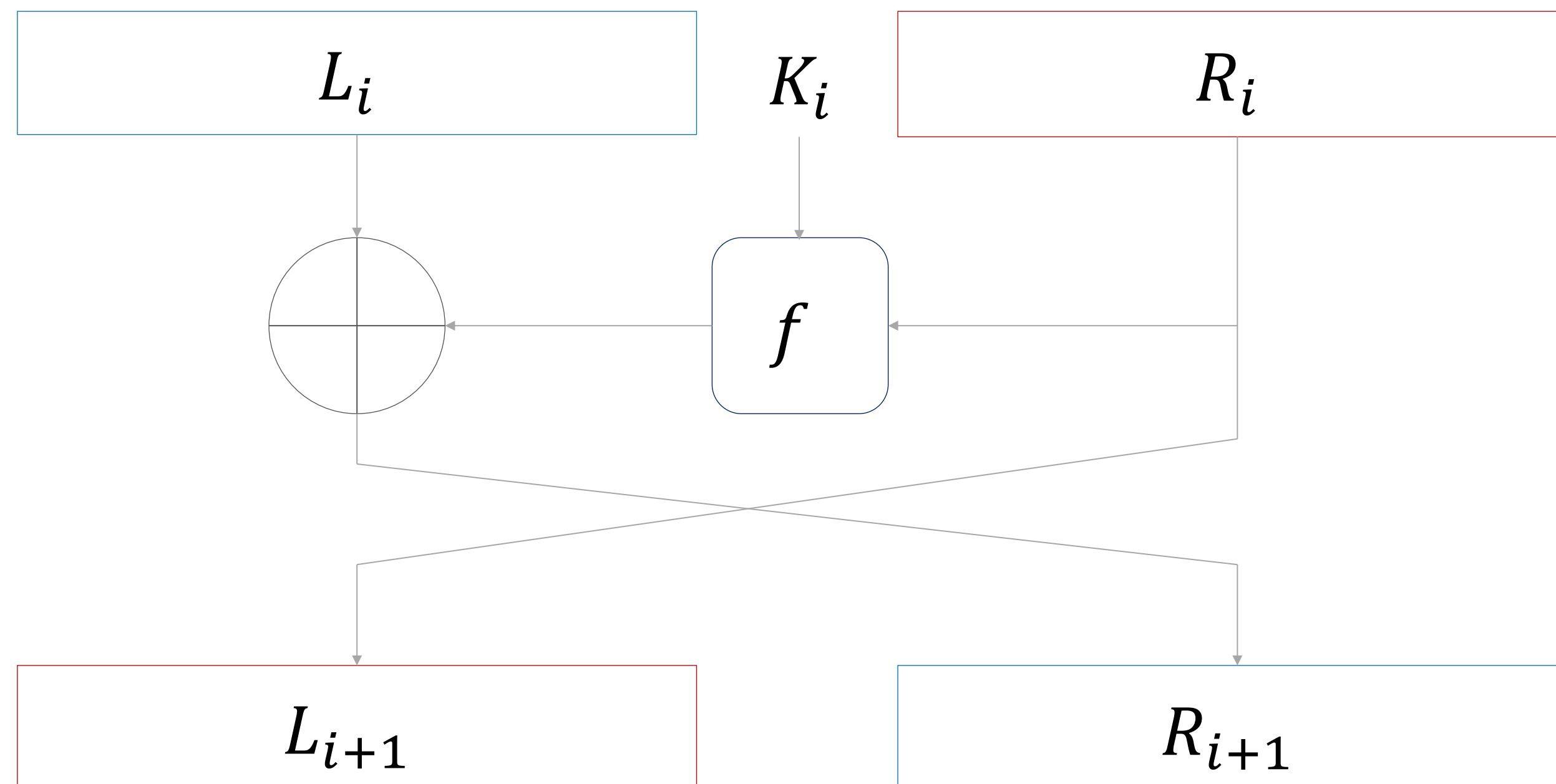
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7



Structure principale de la fonction F

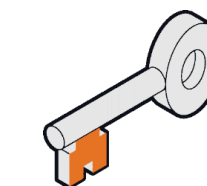


- Une telle structure est nécessairement inversible, quelle que soit la fonction f
- Déchiffrement : même algorithme en inversant l'ordre des clés



$$L_{i+1} = R_i$$

$$R_{i+1} = f(K_i, R_i) \oplus L_i$$



On parle aujourd'hui du schéma de Feistel

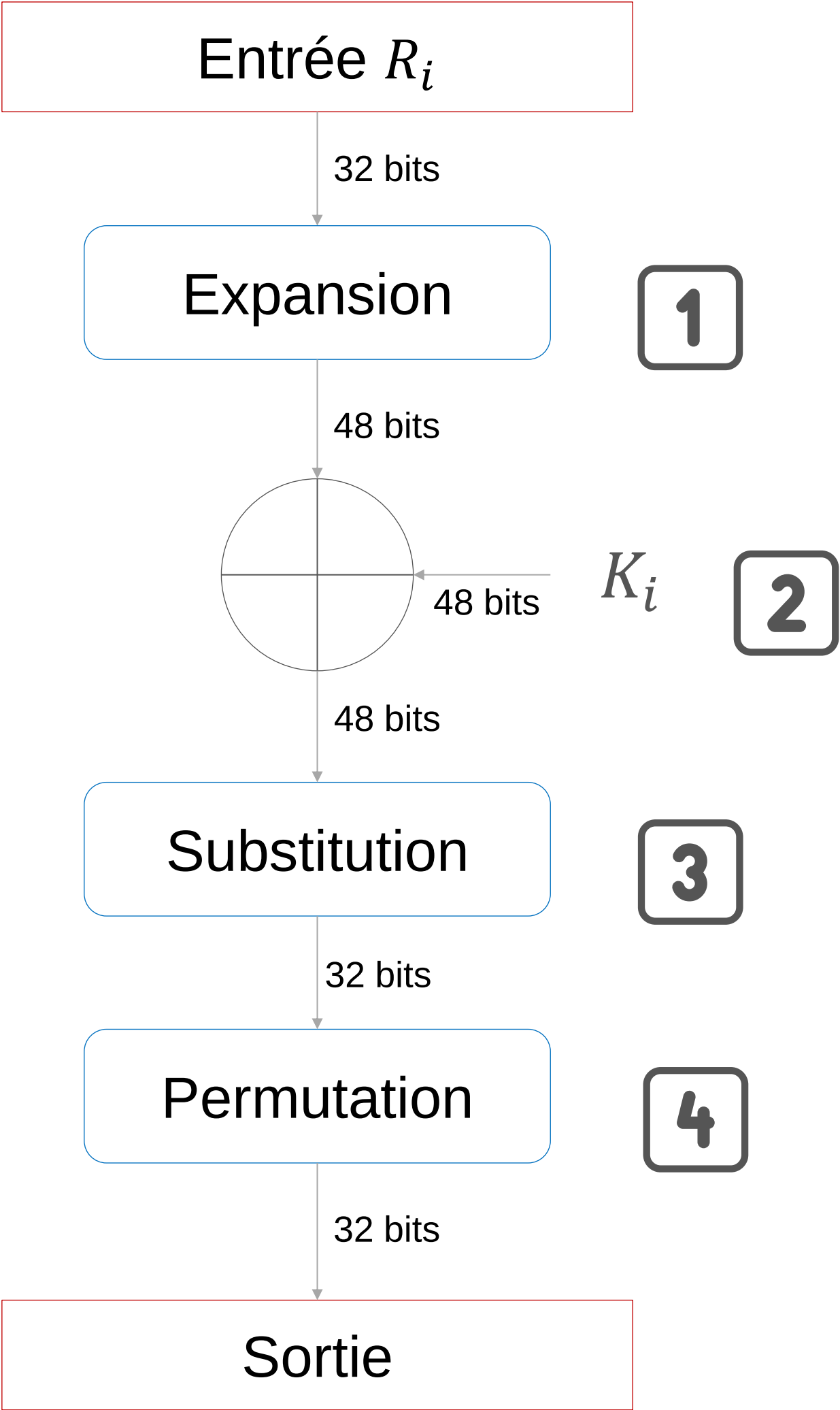
Détails sur la fonction f

- Fonction $f: \{0,1\}^{32} \times \{0,1\}^{48} \rightarrow \{0,1\}^{32}$ composée des étapes suivantes
 - Exécution d'une fonction E **d'expansion** pour faire un bloc de 48 bits à partir 32 bits (16 bits apparaissent ainsi deux fois)
 - Calcul de $E(R_i) \oplus K_i$
 - Découpage du résultat en 8 sous-chaînes de 6 bits, puis transformation de chacune des sous-chaîne en une sous-chaîne de 4 bits via une **fonction de substitution** non linéaire
 - Ordonnancement des 8 sous-chaînes de 4 bits via une **permutation**

$E =$

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1


 Urgence



Fonctions de permutation et de substitution

- Permutation

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Le 8^{ème} bit de la sortie provient du 17^{ème} bit de l'entrée

- Substitution

- Application en parallèle de 8 fonctions fixes (boîtes) de 6 bits vers 4 bits
 - Entrée B sur 6 bits $\{b_1, b_2, b_3, b_4, b_5, b_6\}$
 - Sortie $\tilde{B} = S_j[a, b]$ avec $a = 2 \cdot b_1 + b_6$ et $b = b_2 | b_3 | b_4 | b_5$
- Chaque boîte est codée par un tableau avec $2^6 = 64$ entrées (exemple de S_1)
- Fonction non linéaire

14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
00	15	07	04	14	02	13	01	10	06	12	11	09	05	03	08
04	01	14	08	13	06	02	11	15	12	09	07	03	10	05	00
15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

L'entier $02 = 0 \parallel 0 \parallel 1 \parallel 0$ est sorti si $a = 1$ et $b = 5$

Calcul des clés intermédiaires K_i

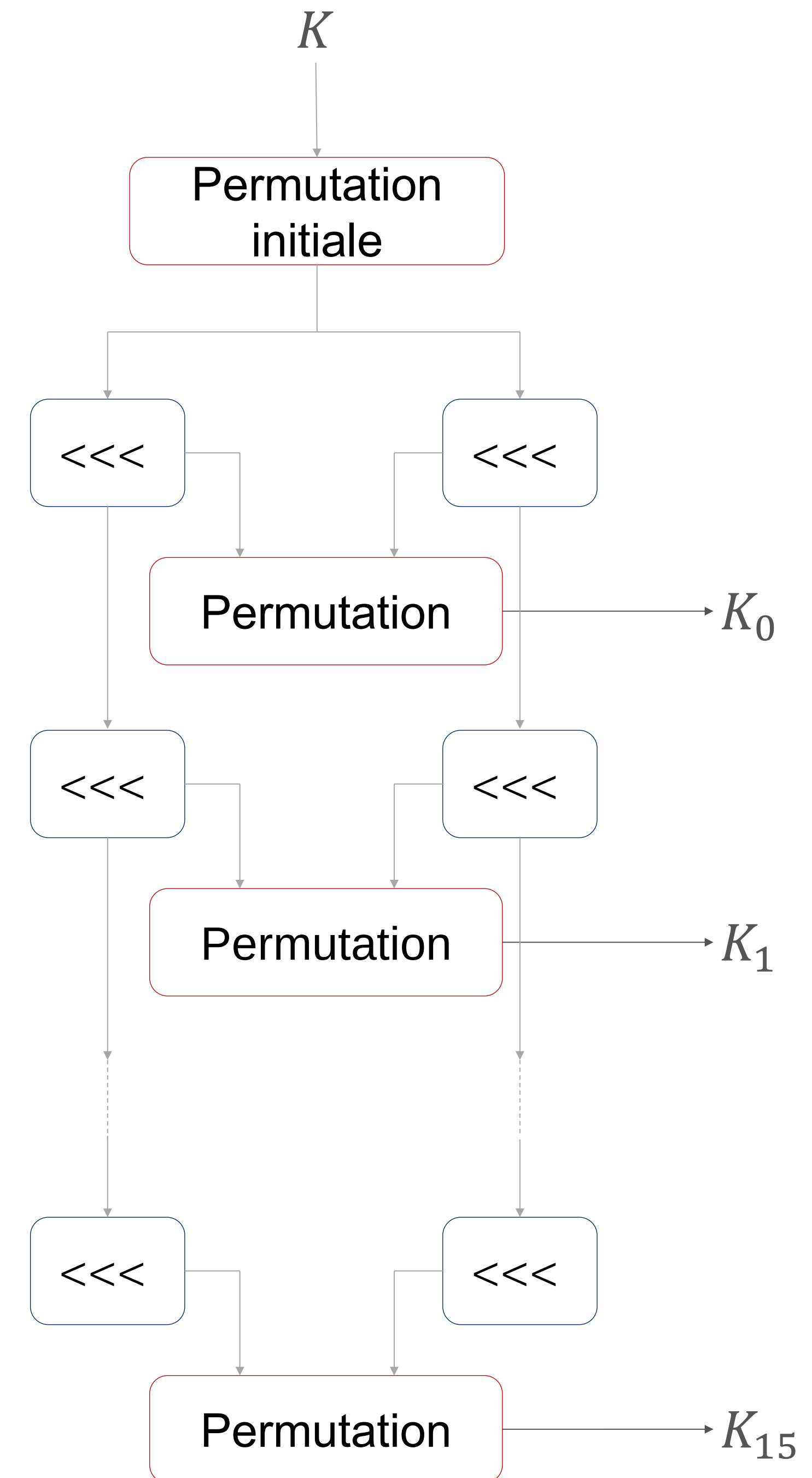
- Fonction $I: \{0,1\}^{64} \rightarrow (\{0,1\}^{48})^{16}$
- Le décalage circulaire ($\lll$) est fait
 - sur 1 bit pour K_0, K_1, K_8, K_{15} , sur 2 bits sinon
- Même processus pour le déchiffrement
 - Les clés seront utilisées dans l'autre sens
- Permutation initiale

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36

63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

- Permutation

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32



Commentaires sur le design

- Ces choix peuvent paraître arbitraires, mais ce n'est pas tout à fait le cas
 - Basé sur les principes de Shannon
 - Schéma de Feistel
 - La non-linéarité de la fonction de substitution a été largement étudiée
 - Expansions et permutations garantissent une diffusion rapide
- Toutes les briques sont très simples à coder et efficaces en hardware

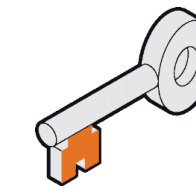
Quelques propriétés de DES

- Pour toute clé K et tout message M , $\overline{DES_K(M)} = DES_{\bar{K}}(\bar{M})$
 - Où pour une chaîne de bits B , $\bar{B}$ est son complément à 1
 - Cette propriété peut être utilisée pour **diviser par deux (i.e., un bit) la recherche exhaustive**
- **Il existe des clés à ne pas utiliser**
 - « **Faibles** » telles que pour tout message M , $DES_K(M) = DES_K^{-1}(M)$
 - « **Semi-faibles** » telles que pour tout message M , $DES_{K_1}(M) = DES_{K_2}^{-1}(M)$



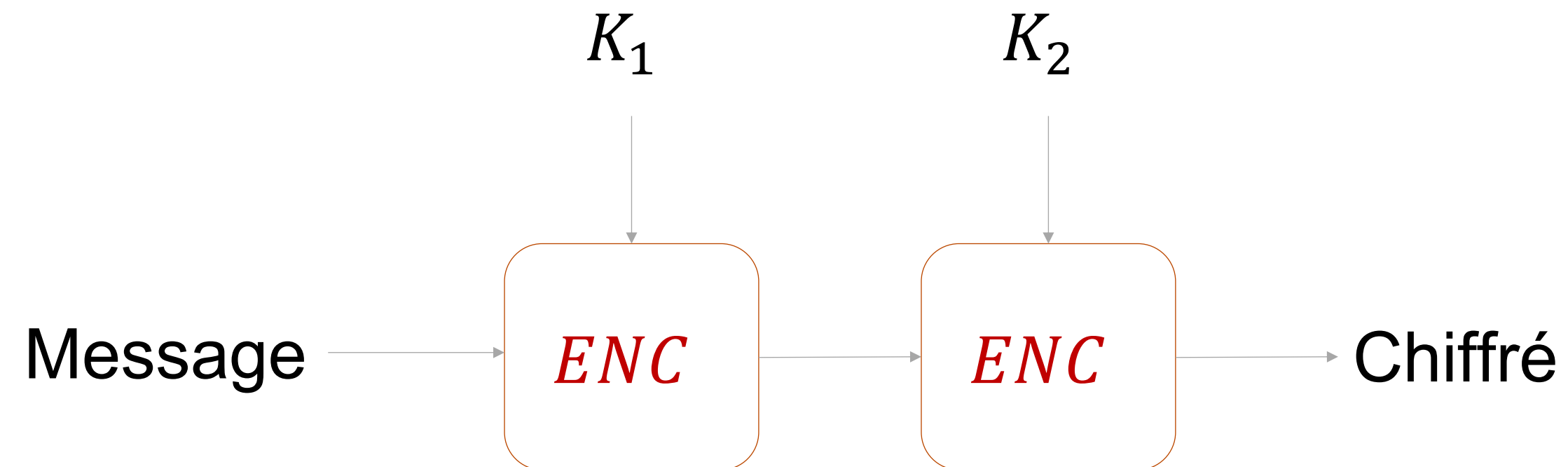
Attaques existantes sur DES

- Deux attaques génériques
 - **Cryptanalyse différentielle** (années 1980) par Biham et Shamir, demandant 2^{47} textes clairs choisis !
 - **Cryptanalyse linéaire** (1993) par Matsui, demandant 2^{41} textes clairs connus, et 2^{41} exécutions de DES
- Une attaque propre à DES
 - Attaque de Davies (années 1980) demandant 2^{50} textes clairs connus, et 2^{50} exécutions de DES, pour un taux de succès de 50%
- Mais un autre problème de DES est bien plus important...



Bases de la
cryptanalyse
actuelle

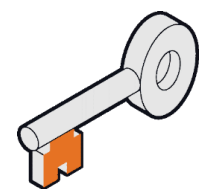
Double-DES



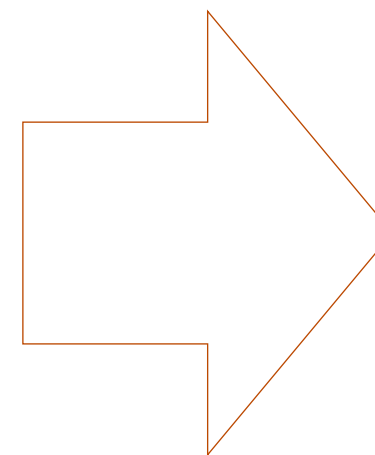
- On obtient un algorithme avec
 - Blocs de taille $n = 64$ bits
 - Clé de taille $k = 112$ bits

! Urgence

- Mais guère plus sûr que le DES
 - Attaque par le milieu avec un couple clair-chiffré



C'est une attaque
générique dit « *meet-
in-the-middle* »



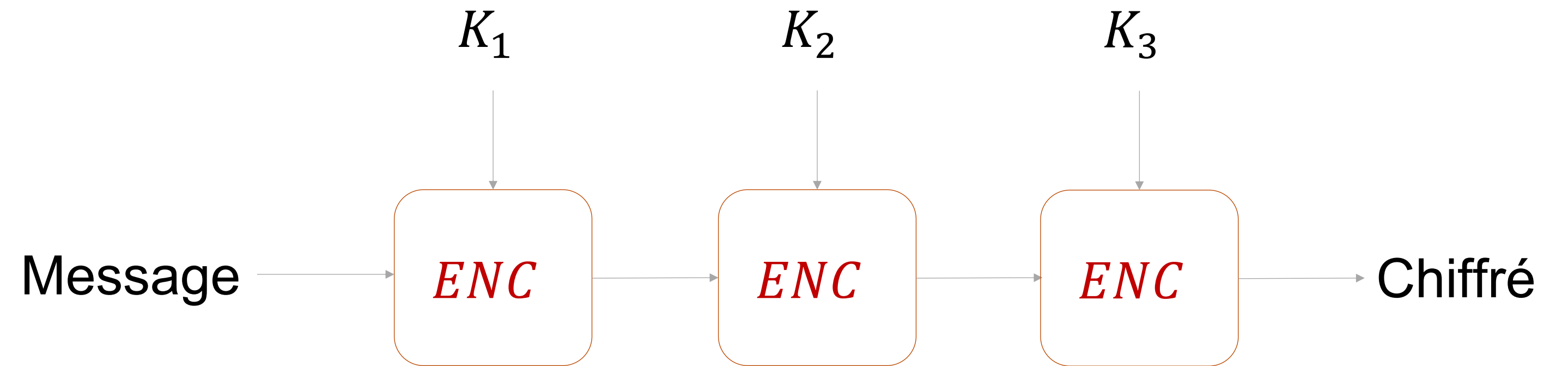
On a 2^{56} « chiffrés » et 2^{56} « déchiffrés »
Chacun fait 64 bits

$\Rightarrow \frac{(2^{56} \times 2^{56})}{2^{64}} = 2^{48}$ collisions en moyenne

$\Rightarrow$ Il existe 2^{48} paires de clés (K_1, K_2)
possibles !

Un second couple clair-chiffré nous donne la
bonne clé

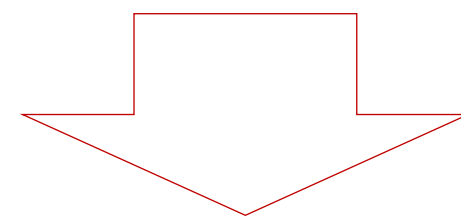
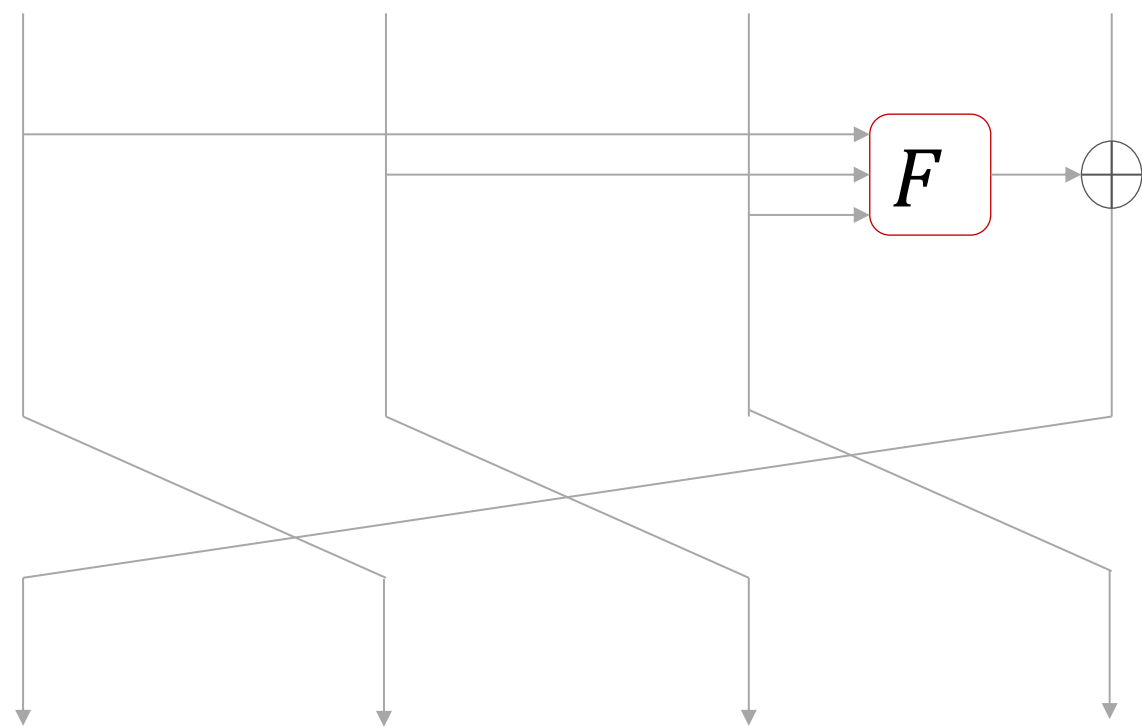
Triple-DES



- Plusieurs possibilités
 - Utiliser *DEC* ou *ENC* pour la seconde itération
 - Utiliser 3 clés ou 2 clés ($K_1 = K_3$)
- Mais aussi plusieurs problèmes
 - Même attaque « *meet-in-the-middle* » que contre Double-DES
 - Version avec 2 clés sujette à une attaque de complexité 2^{80} (resp. 2^{100}) pour 2^{40} (resp. 2^{20}) clairs-chiffrés connus $\Rightarrow$ **utiliser la version avec 3 clés**
 - **Taille des blocs insuffisante**
 - **Manque de rapidité**
- Défini dans plusieurs standards (ANS X9.52, FIPS PUB 46-3, NIST 800-67, ISO/IEC 18033-3) mais aucune agence de sécurité ne le recommande aujourd'hui

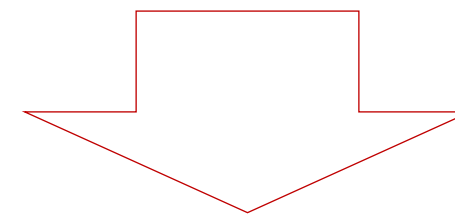
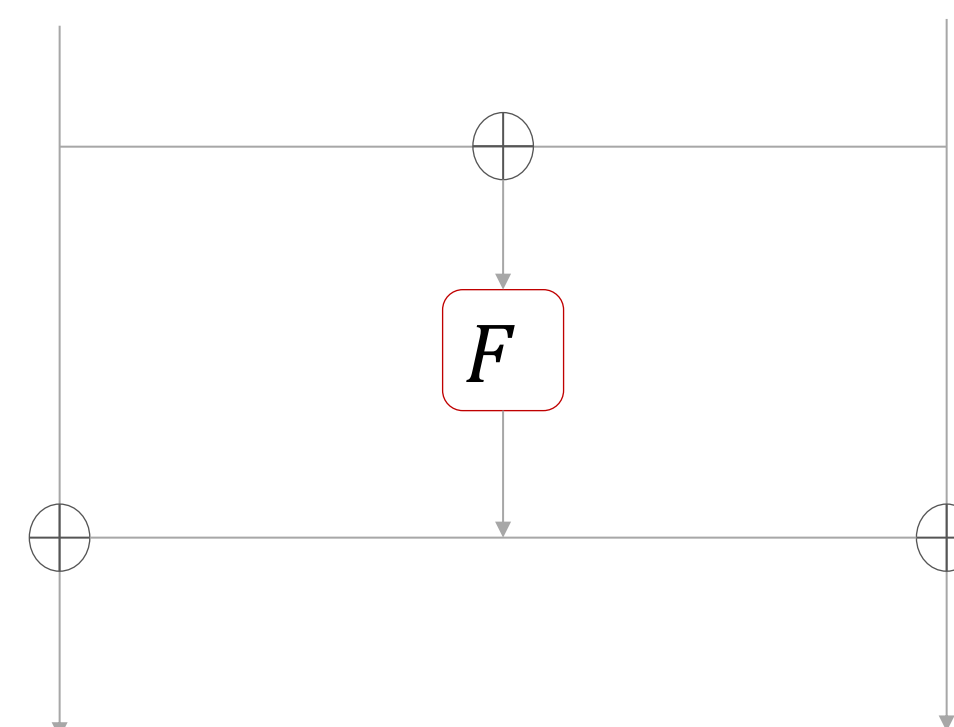
Autres constructions similaires aujourd'hui classiques

Feistel généralisé



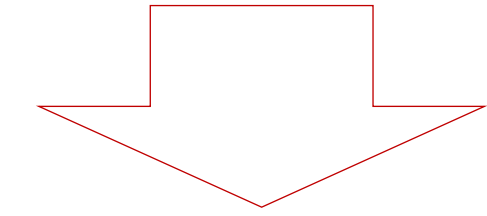
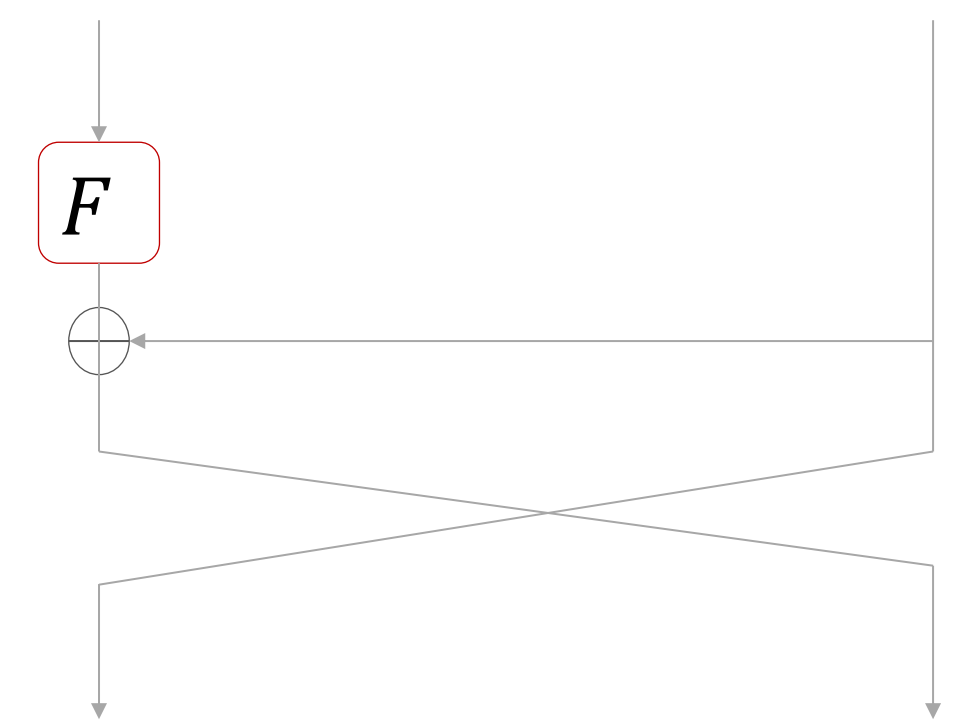
Utilisé dans le système RC6
Utilisé dans certaines
fonctions de hachage (à
suivre...

Lai-Massey



Utilisé dans le système IDEA
(système utilisé dans PGP)

Misty



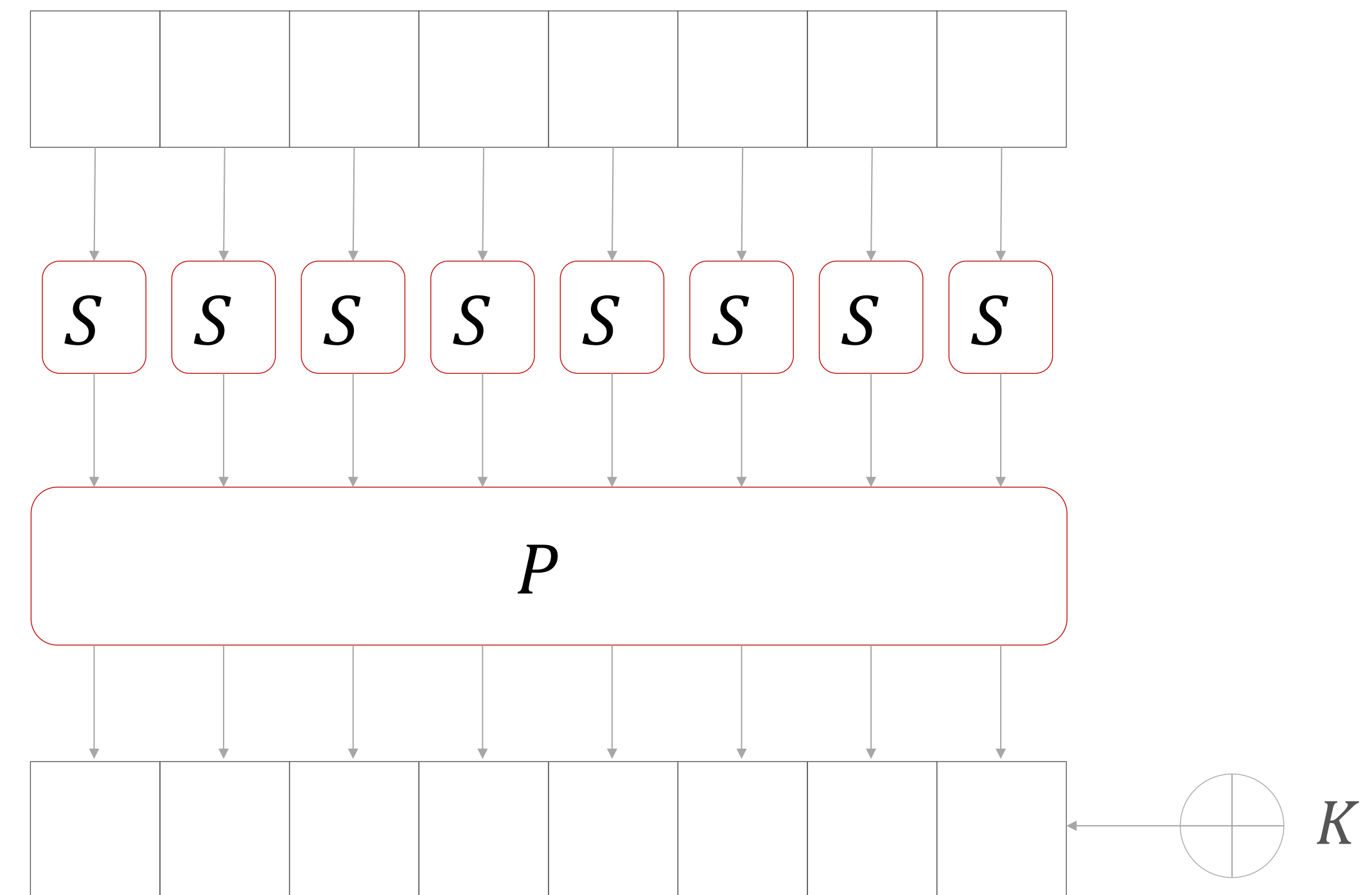
Utilisé avec Feistel dans le
système Kasumi (chiffrement
et intégrité des réseaux 3G)

La compétition AES

- Cahier des charges du NIST (1998)
 - Clé de 128 bits (et même 192 et 256)
 - Blocs de 128 bits
 - Pas d'attaque meilleure que la recherche exhaustive
 - Performant sur toutes les plateformes
 - Algorithme transparent
- Propositions
 - Août 1998 : 16 candidats, 2 folkloriques, 2 inconsciemment folkloriques, il en reste 12
 - Mars 1999 : première passe, 7 sont enlevés pour des petites raisons, il en reste 5 : MARS, RC6, Rijndael, Serpent, TwoFish
 - Octobre 2000 : vainqueur = Rijndael (algorithme belge)

AES est un schéma de substitution/permutation

- De tels schémas sont construits de la façon suivante
 - L'entrée est découpée en sous-blocs d'égale longueur
 - On applique une **substitution** S à chaque sous-blocs (éventuellement un différent par sous-bloc)
 - On concatène l'ensemble et on applique une **permutation** P
 - La dernière étape est **l'ajout de la clé** K
- Le premier tour est précédé de l'ajout d'une clé supplémentaire K_0 pour éviter que l'attaquant déchiffre le premier tour jusqu'à l'ajout de K_1



Caractéristiques de l'AES

- Taille des blocs $n = 128$
- Structure générale
 - Les données sont stockées dans des « matrices » de $4 \times 4 = 16$ cases
 - Chaque case contient un octet
 - On parle d'**état** qui va être modifié au fur et à mesure de l'exécution de l'algorithme
- Taille de la clé $k = 128, 192$ ou 256
- Le nombre de tours va dépendre de la taille de la clé

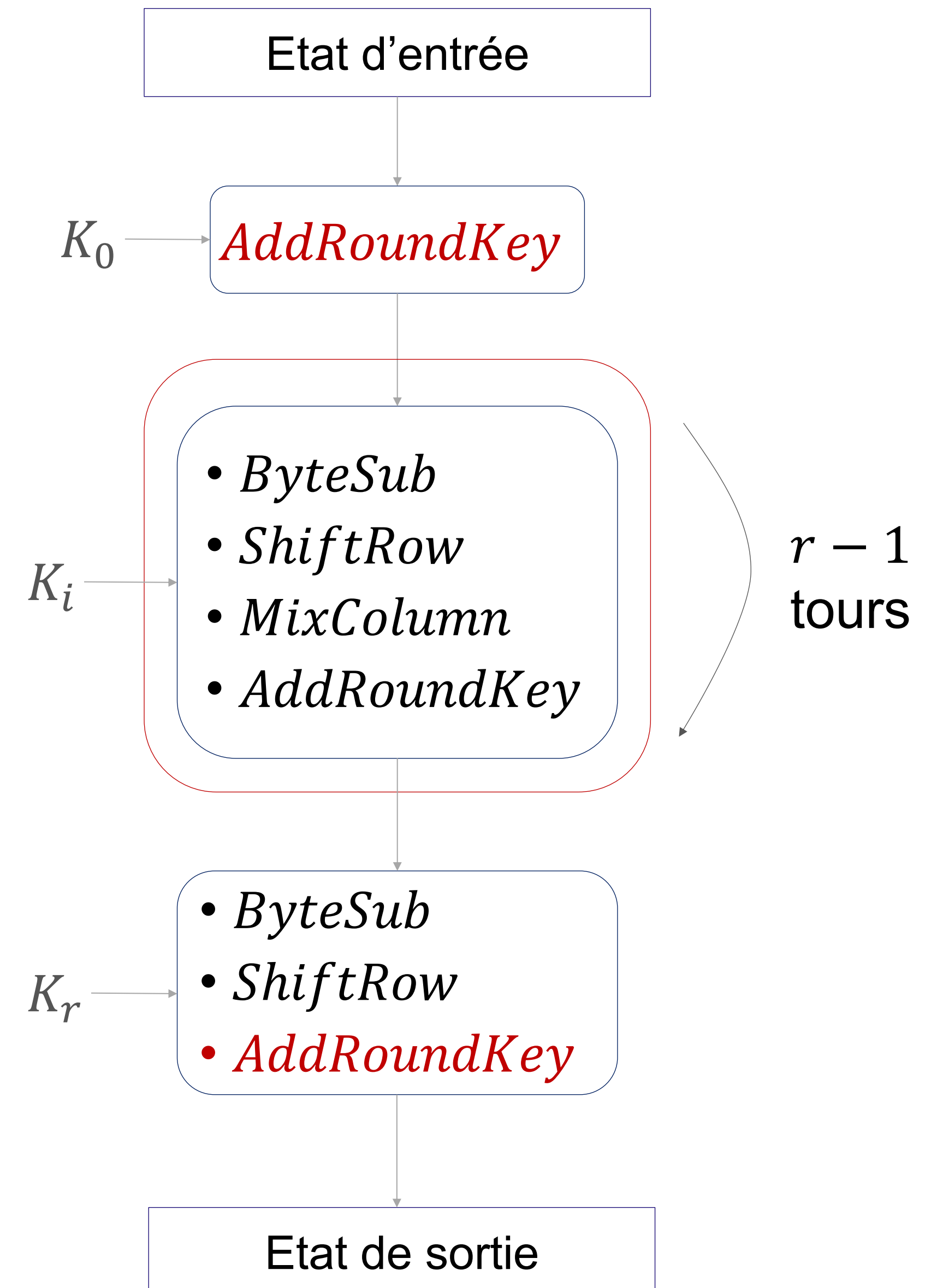
État AES :

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$

	$k = 128$	$k = 192$	$k = 256$
$n = 128$	$r = 10$	$r = 12$	$r = 14$

Structure générale de l'AES

- Calcul d'une clé étendue
- Quatre fonctions de base
 - *ByteSub* : substitution par octet
 - *ShiftRow* : décalage par ligne
 - *MixColumn* : mélange par colonne
 - *AddRoundKey* : addition de la sous-clé
- *ByteSub* est l'opération de **confusion** non linéaire
- *ShiftRow* et *MixColumn* sont des opérations de **diffusion** des bits de l'état



Fonction *ByteSub*

- Fonction fixe de 8 bits vers 8 bits
 - Exécutée sur chaque octet de l'état, avec la même fonction
- Basée sur une opération algébrique dans $GF(2^8) = F_2[X]/P$ avec $P(X) = X^8 + X^4 + X^3 + X + 1$
 - Chaque octet va être représenté par un élément de $GF(2^8)$ (réduits modulo P , de degré < 8)
 - Exemple : $a = 01010100 \rightarrow X^6 + X^4 + X^2$ (**Attention : les coefficients sont toujours dans $\{0,1\}$!**)
- Construction de *ByteSub*
 - Calculer l'inverse de l'octet a dans $GF(2^8) = F_2[X]/P$
 - Calculer l'image du résultat précédent par la fonction $G: GF(2^8) \rightarrow GF(2^8)$ telle que

$$G(X) = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \cdot X + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \text{ où } X = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix}$$

Utilisation d'une table de correspondance

- En pratique, il est souvent plus simple d'utiliser une table de correspondance pour la fonction *SubByte*

poids faible →

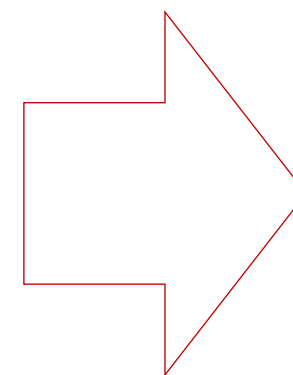
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

poids fort ↓

Fonction *ShiftRow*

- Cette transformation effectue un décalage cyclique des lignes de l'état selon différents offsets
 - La ligne 0 n'est pas décalée
 - La ligne 1 est décalée de 1 octets
 - La ligne 2 est décalée de 2 octets
 - La ligne 3 est décalée de 3 octets
- Il s'agit d'une simple permutation $\Rightarrow$ diffusion au niveau des octets

$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,0}$	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$
$a_{2,0}$	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$
$a_{3,0}$	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$



$a_{0,0}$	$a_{0,1}$	$a_{0,2}$	$a_{0,3}$
$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,0}$
$a_{2,2}$	$a_{2,3}$	$a_{2,0}$	$a_{2,1}$
$a_{3,3}$	$a_{3,0}$	$a_{3,1}$	$a_{3,2}$

Fonction *MixColumn*

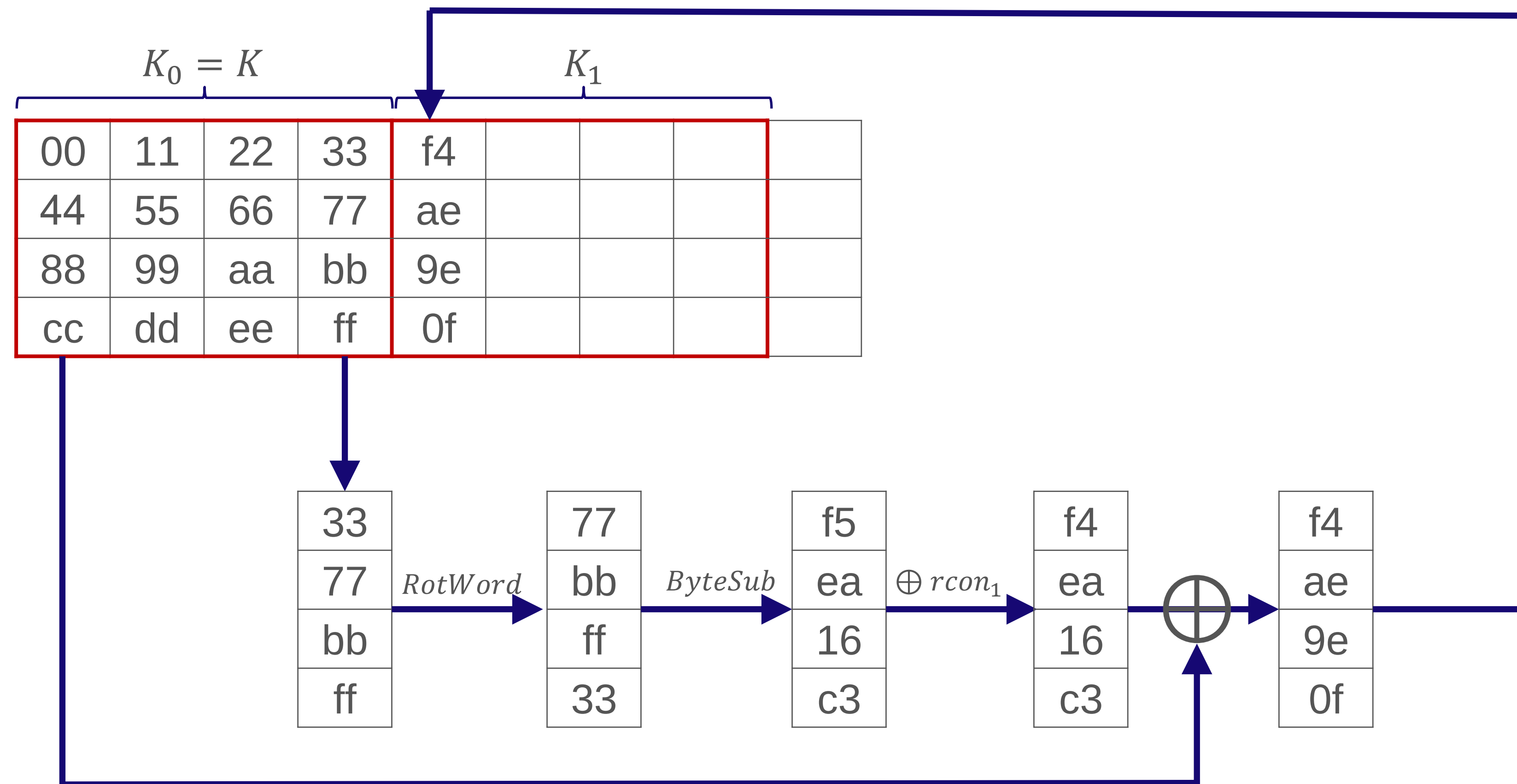
- Cette transformation consiste à prendre chaque colonne de l'état et à exécuter la fonction $H: (GF(2^8))^4 \rightarrow (GF(2^8))^4$ telle que

$$H(X) = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 00 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \cdot X$$

- Nous devons à nouveau utiliser la multiplication dans $GF(2^8)$ avec $P(X) = X^8 + X^4 + X^3 + X + 1$
- Cette étape est linéaire et participe aussi à la diffusion (cette fois-ci des bits)

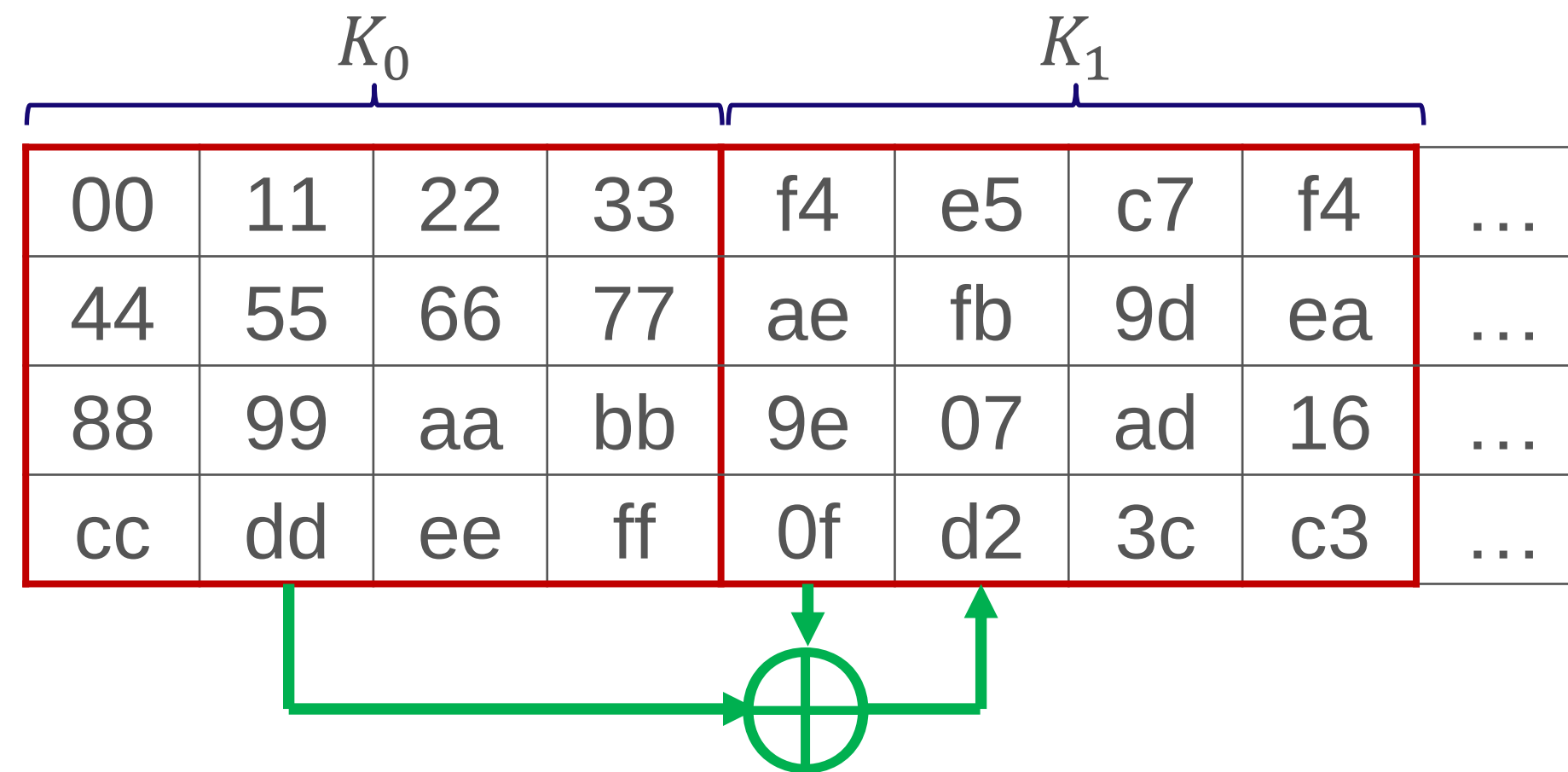
AddRoundKey et calcul de la clé étendue

- La fonction *AddRoundKey* sur chaque octet de l'état : $\forall(i, j), a_{i,j} = a_{i,j} \oplus k_{i,j}$
- Exemple de calcul de la clé étendue : $K \rightarrow \{K_i\}$



AddRoundKey et calcul de la clé étendue

- La fonction *AddRoundKey* sur chaque octet de l'état : $\forall(i, j), a_{i,j} = a_{i,j} \oplus k_{i,j}$
- Exemple de calcul de la clé étendue : $K \rightarrow \{K_i\}$



Sécurité par l'absence d'attaque



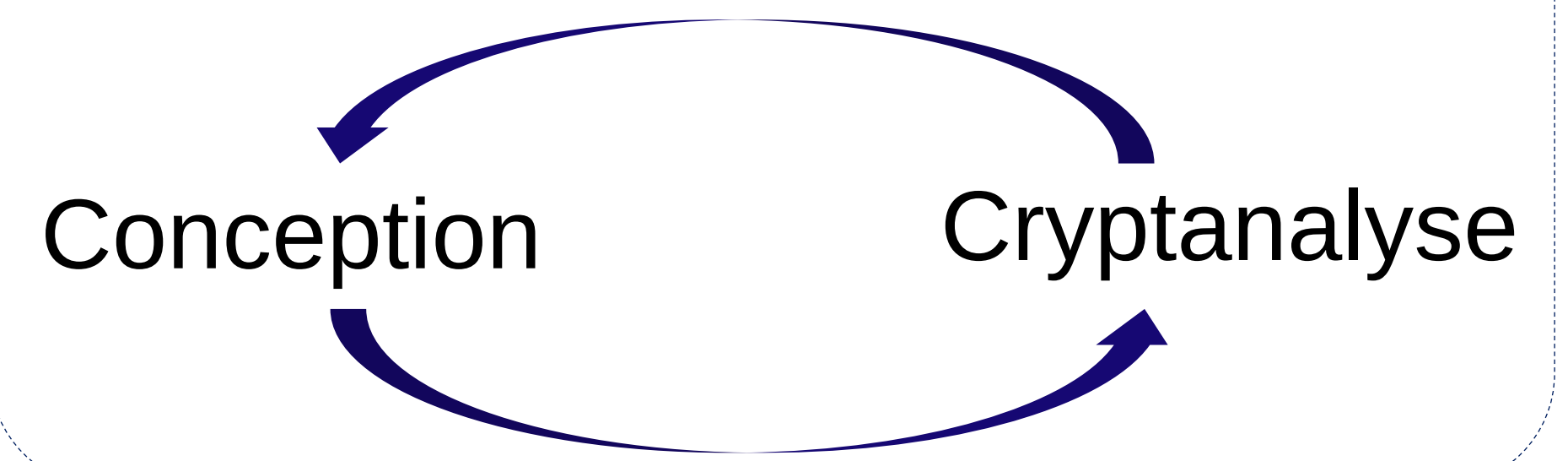
Objectifs

- ~~Un algorithme secret permet-il moins d'attaques... ?~~ **NON**
- Attaques ciblées contre un algorithme particulier
- Attaques génériques (ne dépendent pas de l'algorithme)
 - Principes de Kerckhoffs $\Rightarrow$ **attaque par force brute**
- Techniques d'attaques classiques : **différentielle**, **linéaire**

Moyens

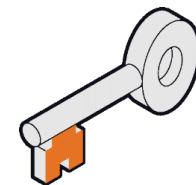
- Obtenir des couples (entrée, sortie) pour une même clé
- Plus il a de couples, plus fort il sera

Cryptographie à clé secrète



Conclusion sur AES

- AES est aujourd'hui un standard utilisé de partout
 - Internet, téléphones, IoT, ...
 - Standards de l'algorithme : **FIPS PUB 197, ISO/IEC 18033-3**
- **Il n'a pour l'instant pas été cassé**, même théoriquement
 - Pas d'attaque significativement plus efficace que la recherche exhaustive
 - Quelques attaques sur des versions simplifiées

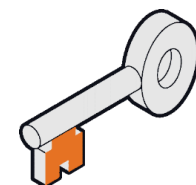


Aujourd'hui, quasiment tout le monde utilise AES !

5. Agencement des blocs et chiffrement par flot

Chiffrement par blocs et agencement

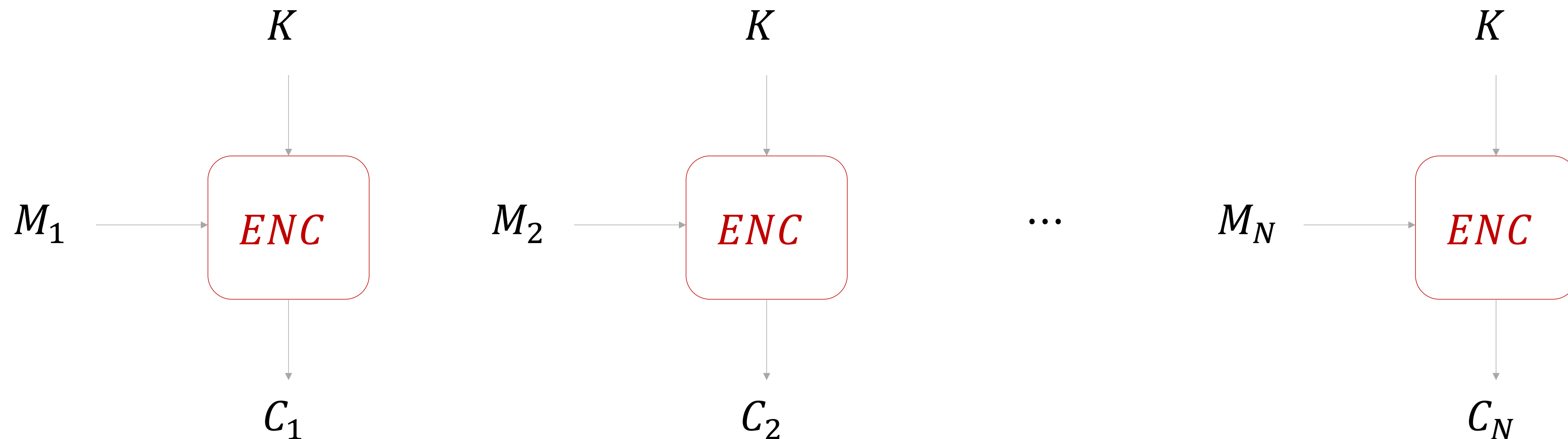
- Les systèmes par blocs permettent de traiter des petits blocs
- L'agencement des blocs va permettre des tailles quelconques de messages, et d'assurer
 - **La confidentialité** : c'est la propriété de base des systèmes par blocs, mais l'agencement des blocs va permettre des approches plus sûres
 - **L'authenticité** : est-ce que le message provient bien de son détenteur ?
 - **L'intégrité** : est-ce que le message n'a pas été modifié ?



Un système cryptographique à clé secrète
n'est rien sans ces modes opératoires

Mode ECB (Electronic CodeBook)

- Message : $M_1 || M_2 || \dots || M_N$ où chaque M_j est un bloc de taille l (dépendant de l'algorithme de chiffrement par bloc utilisé)
- Chiffrement : $\forall j, C_j = ENC(M_j, K)$
- Déchiffrement : $\forall j, M_j = DEC(C_j, K)$

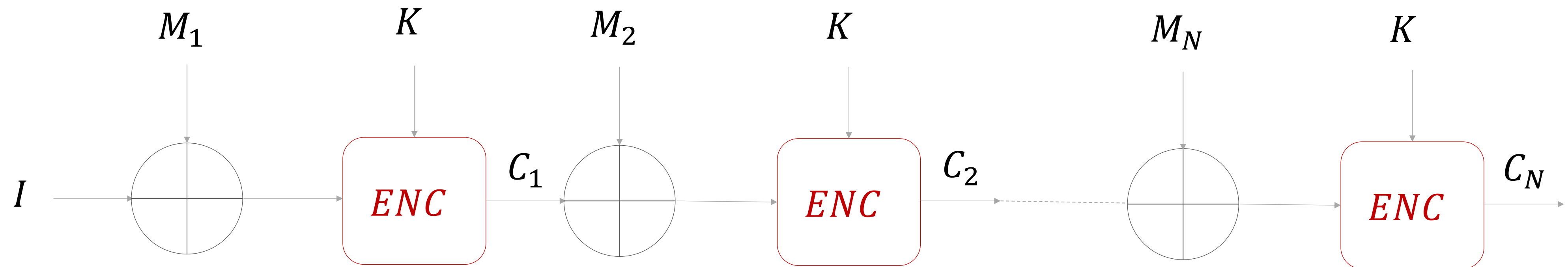


Avantages et inconvénients du mode ECB

- Avantages
 - Pas de problème de synchronisation entre les interlocuteurs
 - S'il y a un problème de transmission sur 1 bit, cela n'altère qu'un seul bloc
 - Parallélisme possible du (dé)chiffrement
- Inconvénients
 - Système déterministe
 - Un attaquant peut mélanger les blocs pour obtenir quelque chose de valide
 - Peu recommandé aujourd'hui

Mode CBC (Cipher Block Chaining)

- Message : $M_1 || M_2 || \dots || M_N$ où chaque M_j est un bloc de taille l (dépendant de l'algorithme de chiffrement par bloc utilisé)
- Chiffrement : $\forall j, C_j = ENC(M_j \oplus C_{j-1}, K)$ avec $C_0 = I$ un vecteur d'initialisation
- Déchiffrement : $\forall j, M_j = DEC(C_j, K) \oplus C_{j-1}$

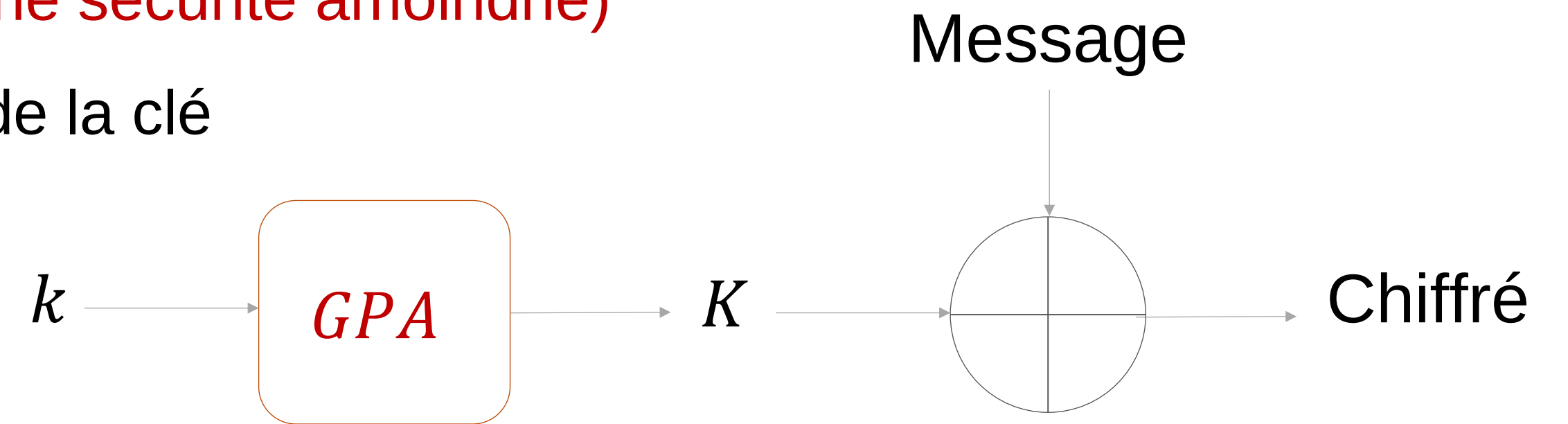


Mode *CBC-MAC*

- Le mode CBC n'est pas préconisé pour la confidentialité
- Mais il donne un **MAC (Message Authentication Code)** : *Mode CBC-MAC*
 - Alice calcule $\Sigma = C_N = \text{CBC-MAC}(M, K)$ et envoie (Σ, M) à Bob
 - Bob calcule $\text{CBC-MAC}(M, K)$ et **vérifie** qu'il obtient la valeur Σ
- Que nous apporte ce mode ?
 - Bob sait que le message n'a pas été modifié $\Rightarrow$ **intégrité**
 - Bob sait qu'il provient bien d'Alice, puisque ce n'est pas lui qui l'a généré, c'est donc elle $\Rightarrow$ **authenticité**
 - Mais il n'y a a priori **aucune confidentialité** sur le message
- Standards: ANSI X9.9, ANSI X9.19, ISO 8731-1, ISO/IEC 9797-1

Principe général du chiffrement par flot

- Objectif : rendre le OTP pratique (mais avec une sécurité amoindrie)
- Fonction $GPA: \{0,1\}^t \rightarrow \{0,1\}^*$ où t est la taille de la clé
 - k est appelée clé ou graine
 - K est appelée suite chiffrante
 - K peut dépendre du message
- Un (vrai) générateur **aléatoire** doit être non déterministe : il ne doit pas pouvoir être reproduit de manière fiable
- Un générateur **pseudo-aléatoire** doit produire une suite
 - **Déterministe** : il peut/doit être reproduit
 - Qui a l'air aléatoire (elle doit satisfaire des **tests probabilistes**, voir <https://csrc.nist.gov/projects/random-bit-generation/>)
 - **Imprédictible**...



Notion d'imprédictibilité

- Soit GPA un générateur pseudo aléatoire
 - A chaque tour d'horloge, GPA sort un bit noté GPA_j
- Notion de **prédictibilité**
 - Il existe un algorithme efficace A tel qu'il existe i et $\varepsilon > 0$ tel que
$$Pr \left[A \left(\{GPA_j(k)\}_{j \in [1, i]} \right) = GPA_{j+1} \right] > \frac{1}{2} + \varepsilon$$
- GPA est **imprédictible** si et seulement si il n'est pas prédictible !
 - Pour tout j , il n'existe pas d'adversaire efficace capable de prédire avec un avantage ε le bit $j + 1$ en connaissant les j premiers bits, pour ε négligeable

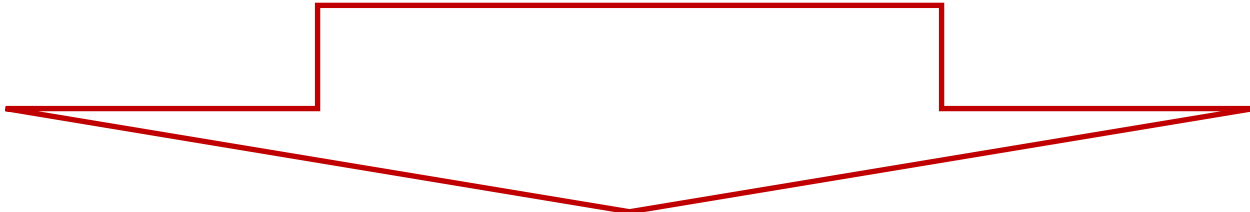
Chiffrement par bloc vs. chiffrement par flot

Algorithmes de chiffrement par blocs

- Conception très bien maîtrisée
 - Cf. les dernières compétitions du NIST
- Moins bonne efficacité

Algorithmes de chiffrement par flot

- Meilleure efficacité
- Conception moins maîtrisée
 - De nombreuses propositions ont été cryptanalysées



Chiffrement par flot peu recommandé par les instances officielles

- Aucune proposition par le BSI
- Un seul est recommandé par l'ANSSI
- ENISA en recommande plus, mais en appuyant bien sur les arguments ci-dessus

Comment construire un chiffrement par flot

1. Registres à décalage

- Principe de base peu sûr
- Mais plusieurs adaptations possibles

2. Techniques spécifiques

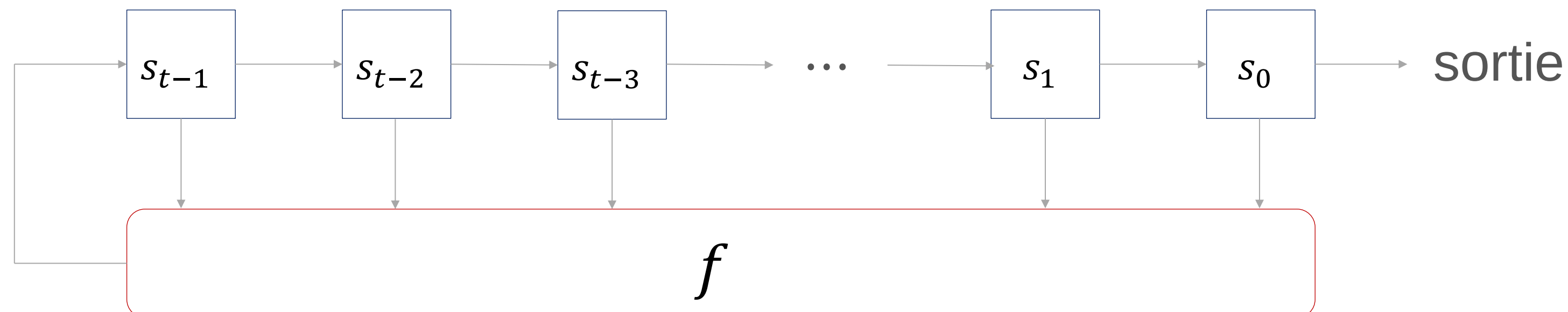
- Bonne maîtrise
- Mais les adaptations rendent les choses moins sûrs ou moins efficaces

3. Mode opératoire et chiffrement par blocs

- Excellente maîtrise de la sécurité
- Moins efficace que les autres techniques

Les registres à décalage comme base d'un *GPA*

- Description
 - Un ensemble $\{s_0, \dots, s_{t-1}\}$ de k cases mémoire
 - Une **fonction de rebouclage linéaire** $f: \{0,1\}^t \rightarrow \{0,1\}$
- **Fonctionnement (boucle itérée)**
 - Initialisation du registre avec un état initial $\{i_0, \dots, i_{t-1}\}$
 - Le contenu des cases mémoires est envoyé en entrée de f qui donne une valeur s_t
 - Le contenu du registre est décalé d'une case vers la droite
 - Le contenu de la case la plus à droite sort du registre à décalage $\Rightarrow$ sortie du *GPA*
 - Le contenu de la case la plus à gauche (laissée vide) est alimentée par s_t



Fonction de rebouclage

- La fonction f est **linéaire** : il va s'agir d'un **XOR entre le contenu de certaines cases mémoire**
- La fonction f va donc être paramétrée par $\{a_0, \dots, a_{t-1}\} \in \{0,1\}^t$

$$f_{a_0, \dots, a_{t-1}}(s_0, \dots, s_{t-1}) = \sum_{j=0}^{t-1} a_j \cdot s_j = s_t$$

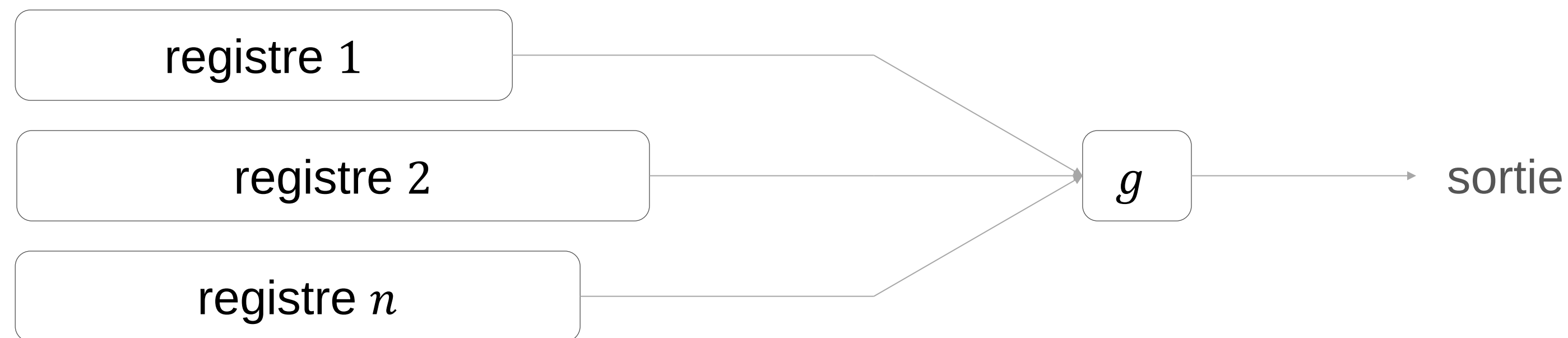
- Polynôme caractéristique d'un registre à décalage

$$P(X) = a_0 + a_1X + \dots + a_{t-1}X^{t-1} + X^t$$

- L'état '0' va poser problème
- Il existe forcément une période maximale
- **Linéaire $\Rightarrow$ Cassable** : connaissant $2t + 2$ bits, un adversaire peut retrouver l'ensemble des a_i !!
- Comment réparer ?

Rajouter une fonction non-linéaire

- Casser le mauvais et garder le bon
- Utiliser plusieurs registres à décalage et les associer via une fonction g non linéaire

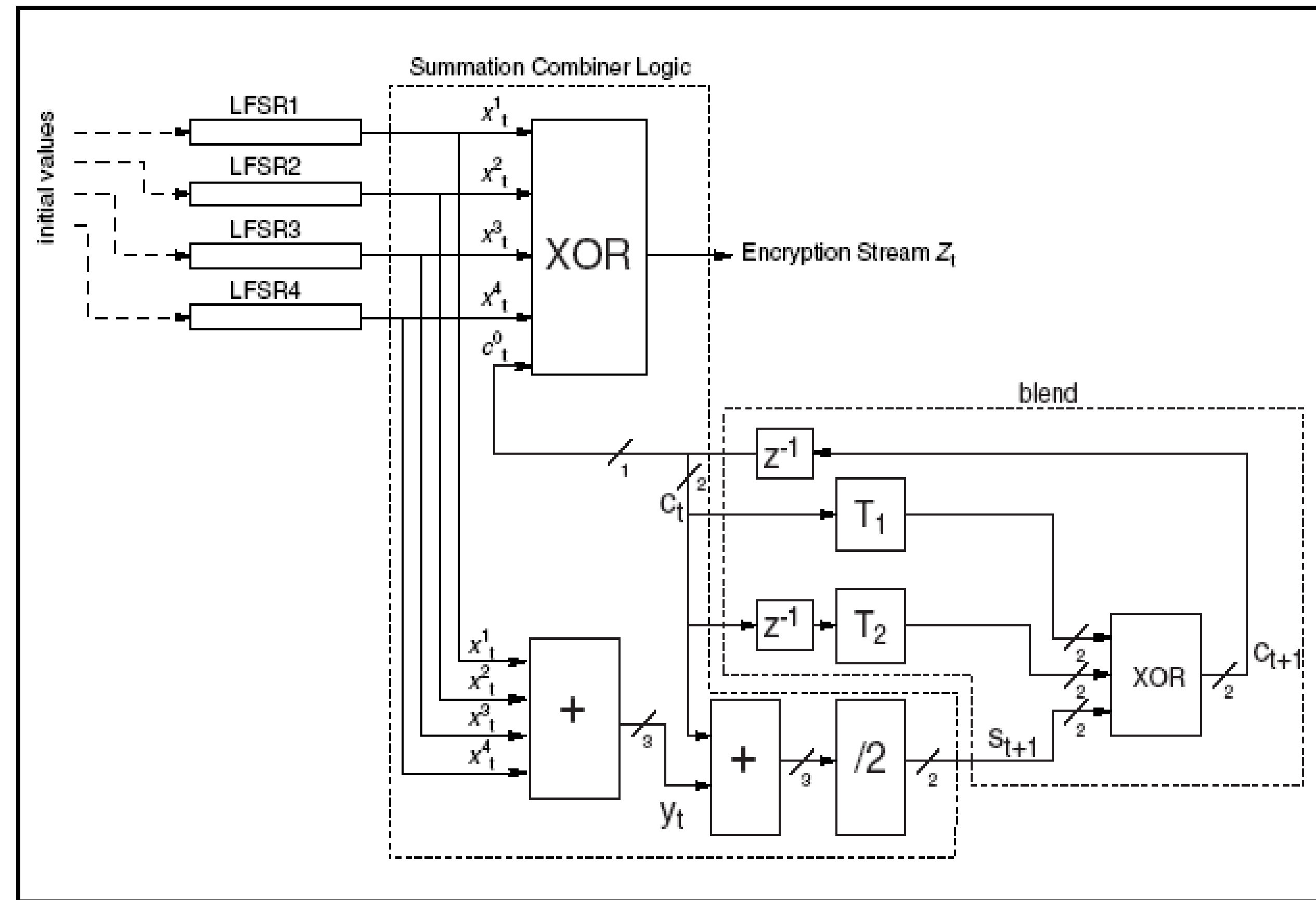


- Utilisation d'une fonction g booléenne
 - Haut degré algébrique pour éviter les attaques utilisant une structure algébrique
 - Non-linéaire, c'est-à-dire le plus éloignée possible d'une fonction affine
 - Équilibrée : la sortie doit être uniformément distribuée (mauvais exemple du générateur de Geffe $g(x_1, x_2, x_3) = x_1x_2 \oplus x_1x_3 \oplus x_2$ pour lequel 75% de la sortie vient du registre 2)
 - ...

Chiffrement $E0$ pour Bluetooth

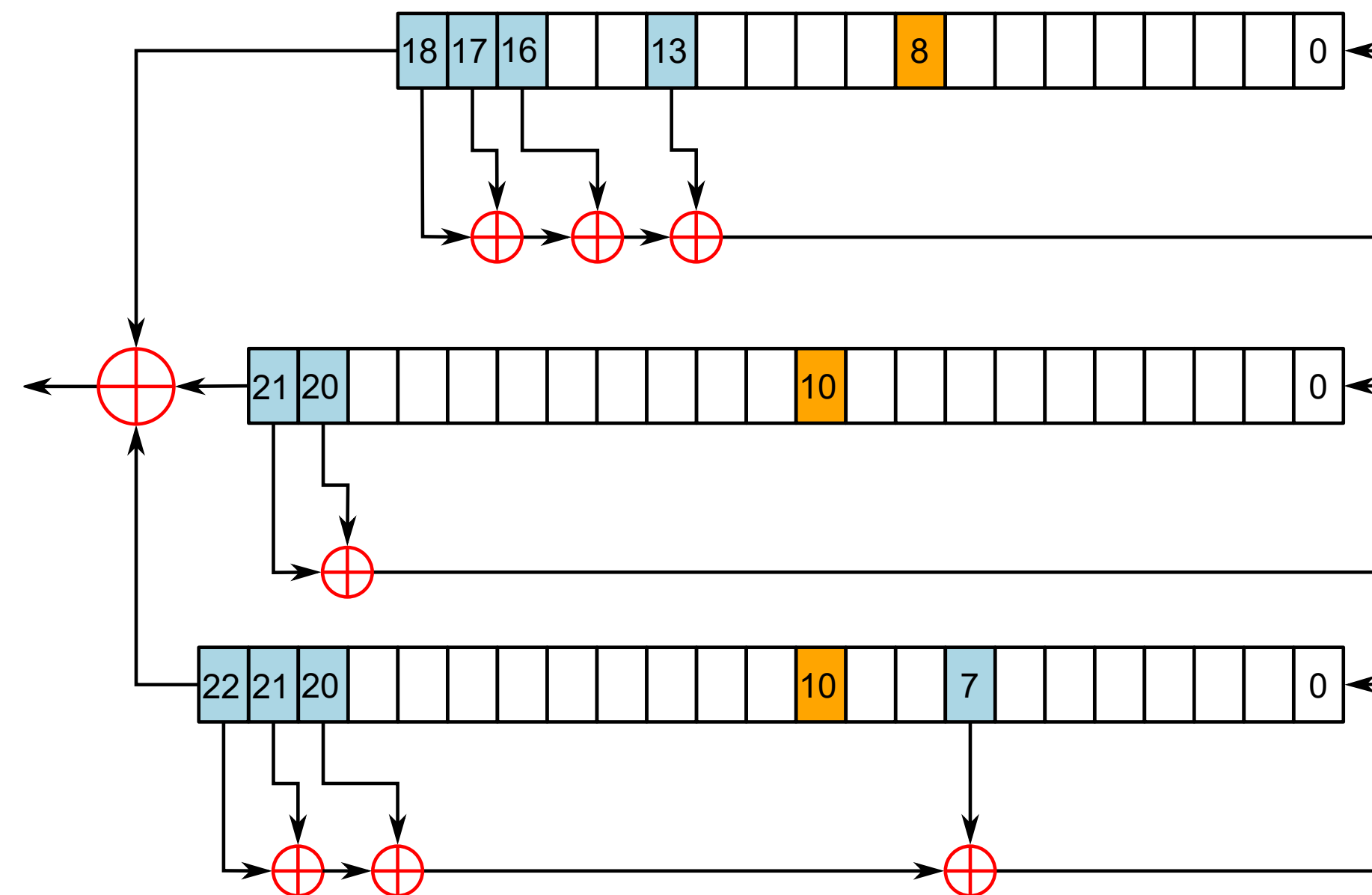
! Urgence

- 4 registres à décalages
- $P_1(X) = X^{25} + X^{20} + X^{12} + X^8 + 1$
- $P_2(X) = X^{31} + X^{24} + X^{16} + X^{12} + 1$
- $P_3(X) = X^{33} + X^{28} + X^{24} + X^4 + 1$
- $P_4(X) = X^{39} + X^{36} + X^{28} + X^4 + 1$
- Non linéarité assurée par un état interne (voir c_t)
- Attaque à texte clair connu en 2^{38} opérations
- Toujours utilisé mais **recommandé**
par aucune autorité de sécurité



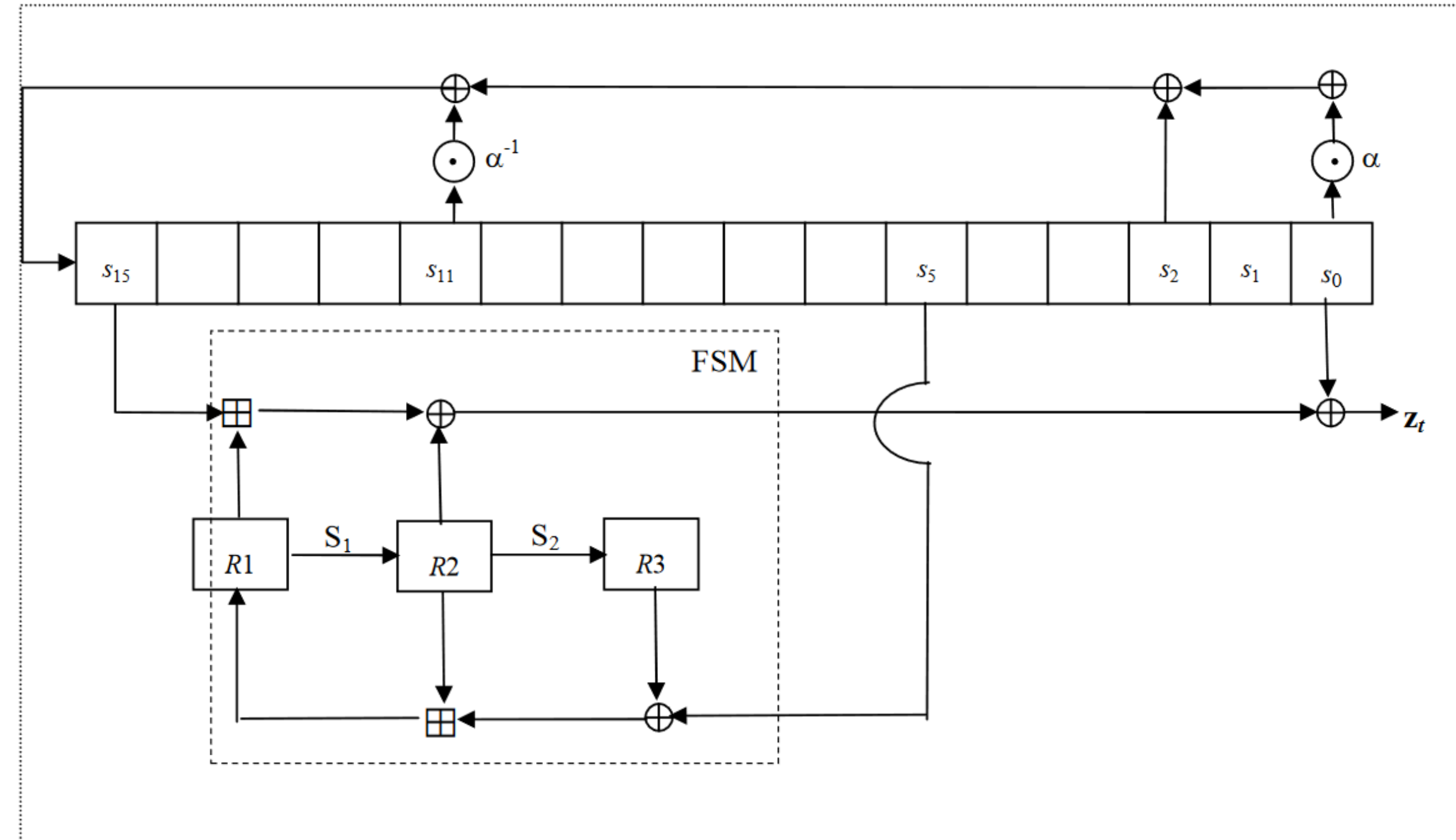
Chiffrement A5/1 pour la 2G

- Chargement de K_C (64 bits), puis du vecteur d'initialisation (22 bits) dans les 3 registres
- 100 « tours à vide »
- Production de 2×114 bits chiffnants (seuls les registres R_i tels que $C_i = m$ avancent)
- Attaque à texte clair connu en 2^{40} opérations : considéré comme non sûr



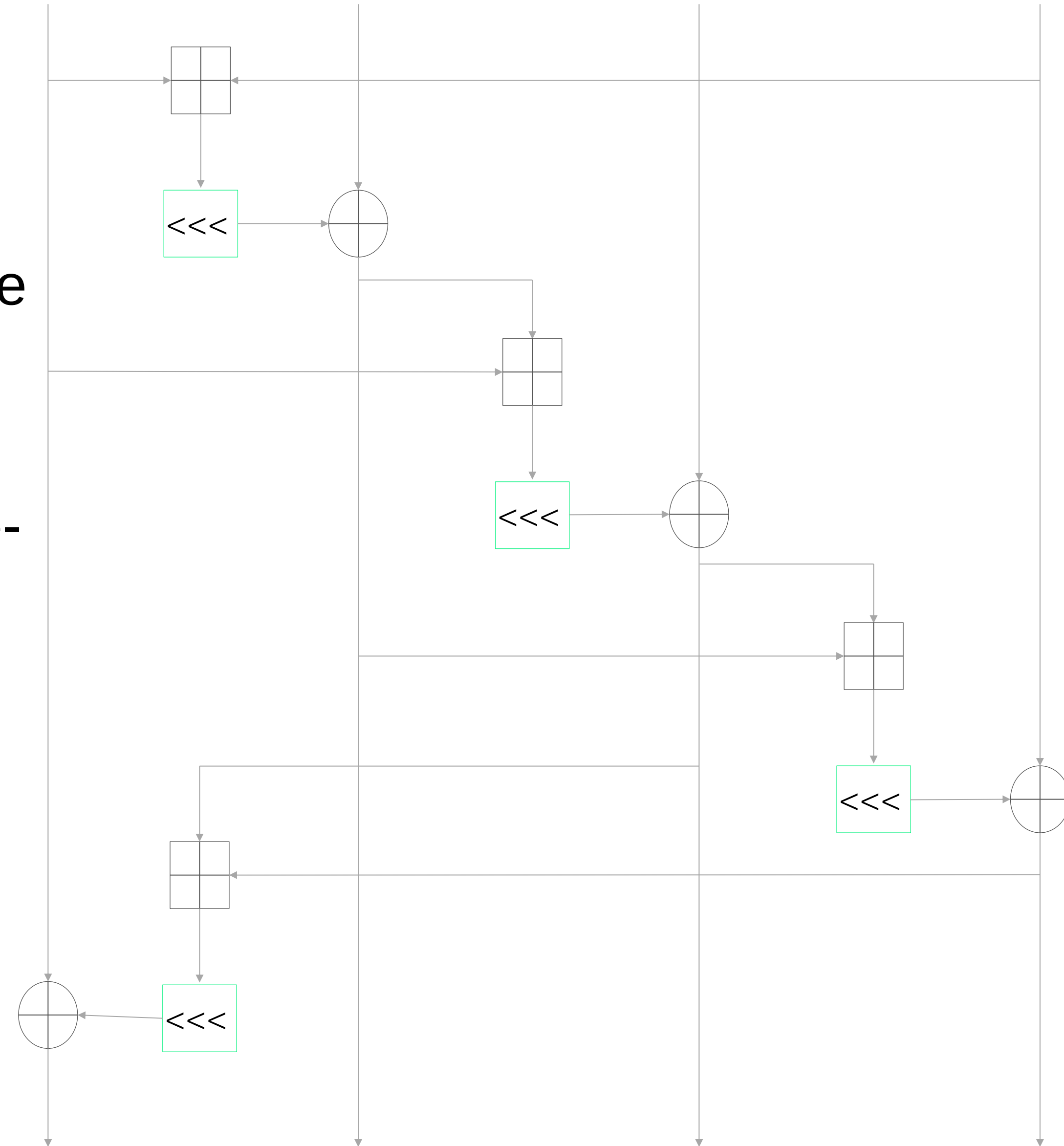
Chiffrement SNOW depuis les réseaux 3G

- Basé sur une machine à état fini, opération non linéaire
- Les fonction S_1 et S_2 sont des fonctions de substitutions
- **Systeme aujourd'hui considéré comme sûr**
- Basé sur un standard ISO/IEC 18033-4
- **Algorithme cœur des réseaux 3G, 4G et 5G (avec AES)**



GPA à base de techniques de chiffrement par blocs

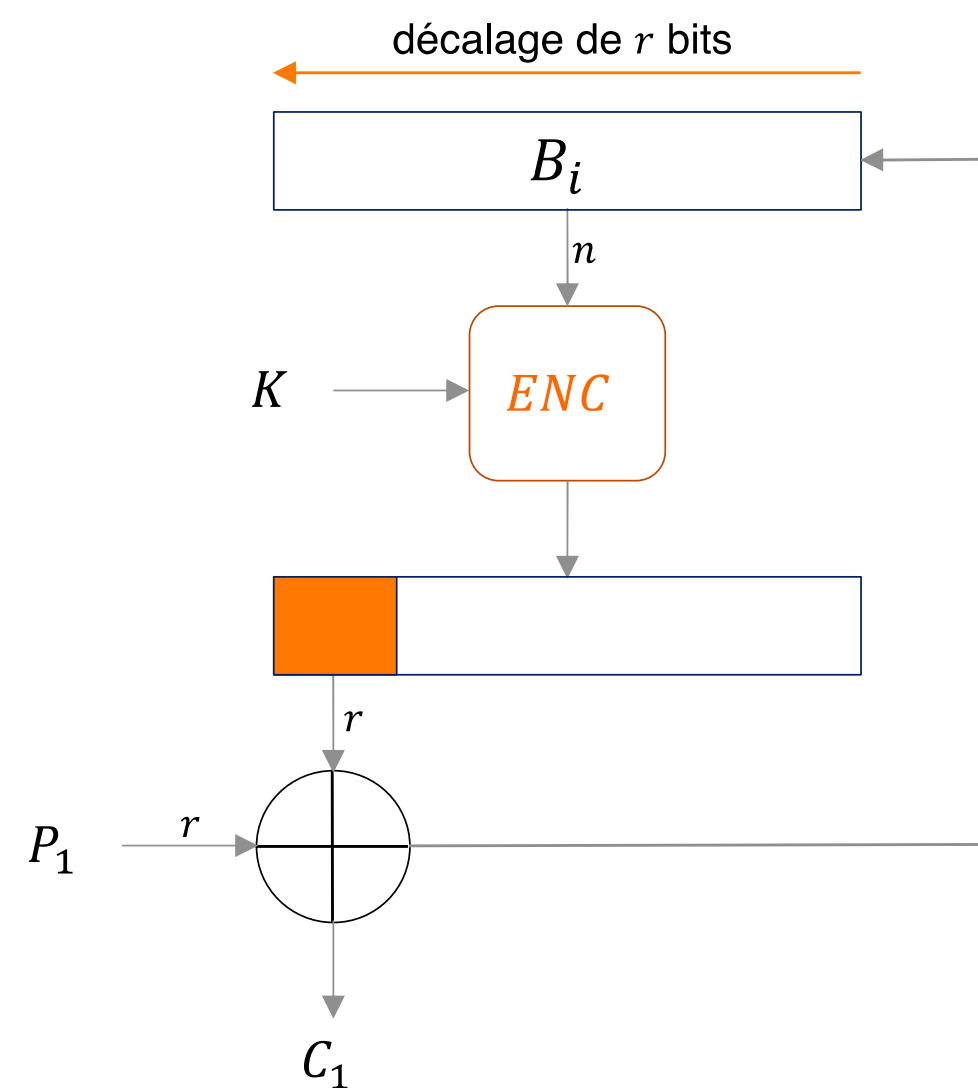
- Exemple de **Salsa2020**
 - Proposé comme standard de fait lors de la compétition eSTREAM
 - Exécution de 20 tours
 - Basé sur le principe ARX = Add-Rotate-XOR
 - Seule attaque pertinente connue en 2^{109}
- Variante ChaCha préconisée par ANSSI et ENISA



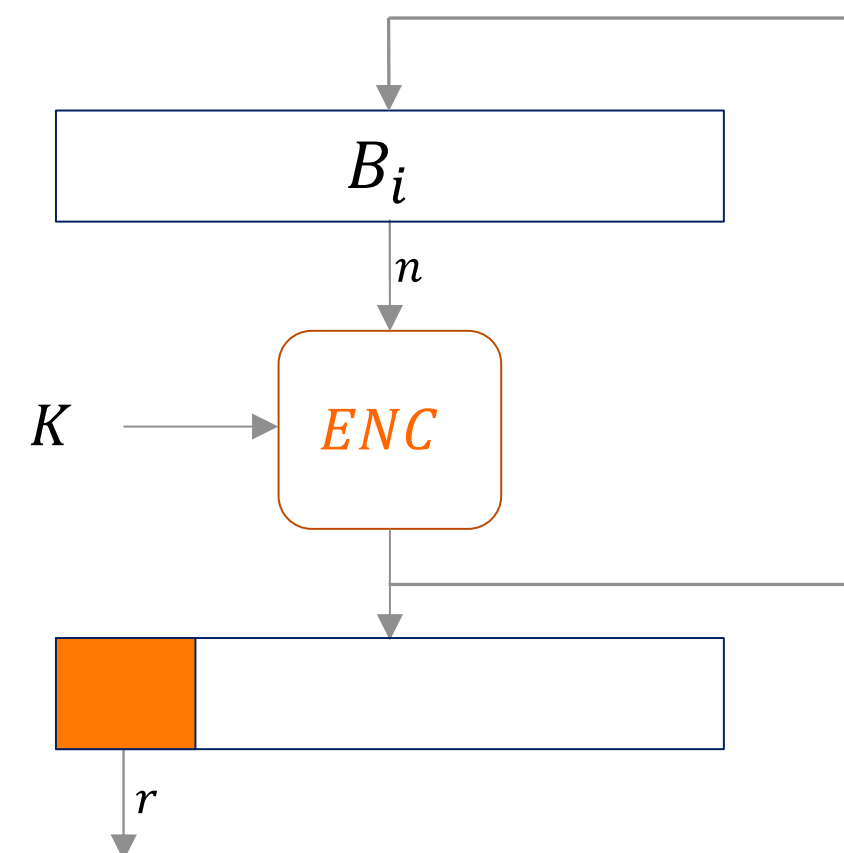
GPA et modes opératoires

- Il est possible d'utiliser un mode opératoire et un système de chiffrement par blocs pour générer une clé de façon pseudo-aléatoire, puis utiliser un OTP
- Trois possibilités sont couramment avancées

Mode **CFB**



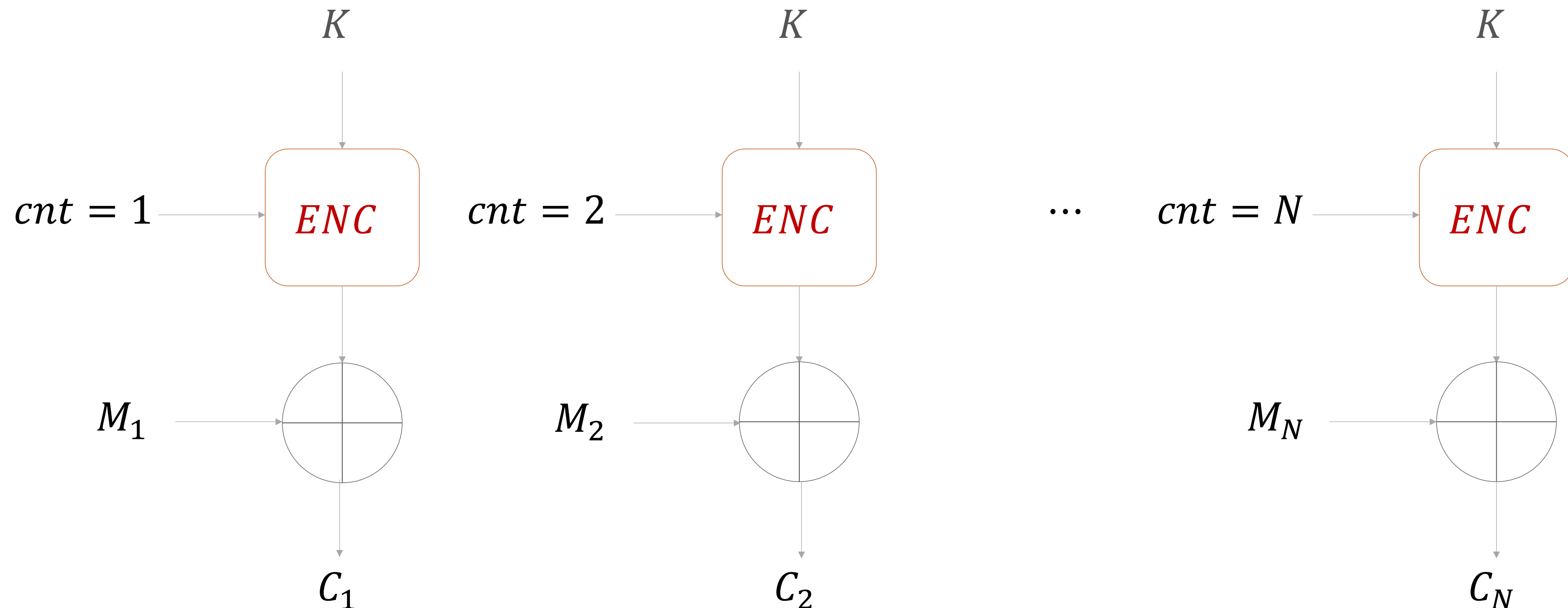
Mode **OFB**



Mode CTR...

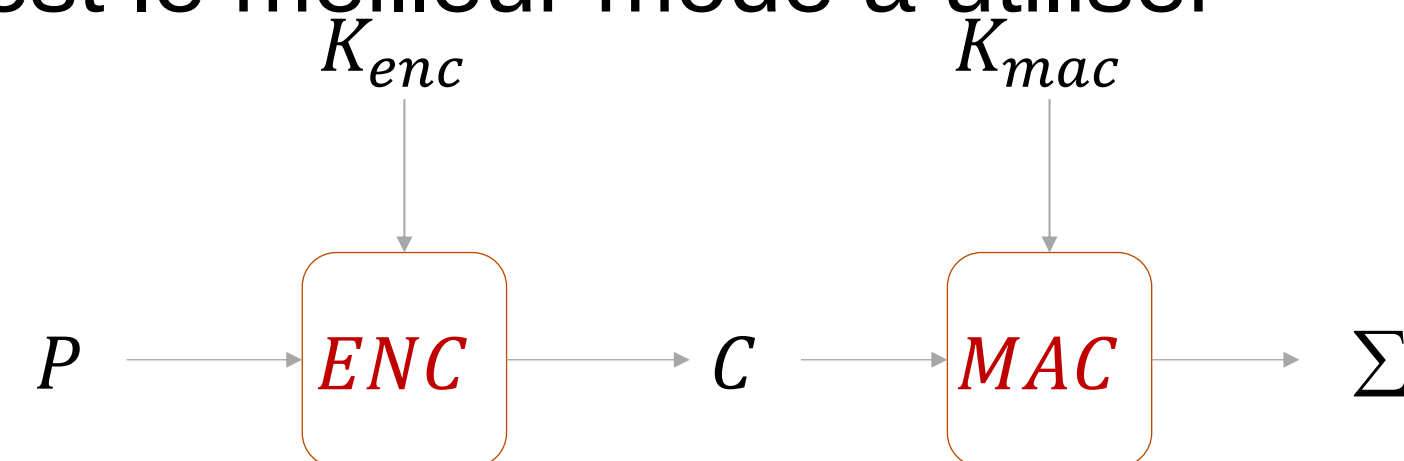
Mode opératoire et chiffrement par blocs: CTR

- Message : $M_1 || M_2 || \dots || M_N$ où chaque M_N est un bloc de taille l (dépendant de l'algorithme de chiffrement par bloc utilisé)
- Chiffrement : $\forall j, C_j = ENC(j, K) \oplus M_j$, Déchiffrement : $\forall j, M_j = ENC(j, K) \oplus C_j$
- Pré-calculs possibles, système parallélisable



Chiffrement authentifié

- L'objectif est d'assurer à la fois la **confidentialité** et le **contrôle d'intégrité**, en un seul processus + authentification
- En s'appuyant sur ce que nous avons vu jusque-là : un système de chiffrement par bloc, des modes opératoires
- Méthodes génériques
- **Encrypt-then-MAC** est le meilleur mode à utiliser



- Mac-then-Encrypt est moins préconisé, sauf si d'autres éléments sont rajoutés (exemple de TLS)
- MAC-and-Encrypt est moins préconisé, sauf si d'autres éléments sont rajoutés (exemple de SSH)

Utilisation de modes opératoires spécifiques

- Mode **CCM**
 - Idée générale très simple = exécuter les modes CTR et CBC
 - Utilisé dans de nombreux contextes : IEEE 802.11i, IPsec, TLS 1.2, Bluetooth 4.0
 - Alternative similaire : EAX
- **Offset Codebook (OCB) et Galois/Counter Mode (GCM)**
 - Un seul mode opératoire pour assurer les deux propriétés
 - **Permet de meilleures performances**
 - Utilisés dans de nombreux contextes: IEEE 802.11, IPsec, TLS 1.2, TLS 1.3
 - Standardisés à l'ISO/IEC 19772 (OCB), nombreuses RFC (GCM)

A retenir jusque-là



- Les principes de Shannon et le principe de substitution/permutation d'AES
- Le schéma de Feistel
- Il ne faut pas utiliser un double-X (exemple du double-DES)
- Accélération des calculs par des tables de correspondance
- L'importance des modes opératoires, ceux qui sont préconisés (CBC, CTR, CCM, OCB, GCM) et les propriétés qu'ils permettent d'obtenir

6. Échange de clé

Problématique de l'échange de clé

- L'échange de clé doit assurer la **confidentialité de la clé échangée**
- Mais ce n'est pas suffisant : Il faut **partager la clé avec le bon interlocuteur**
 - **Nécessité d'une authentification** qui va différer d'une technique à une autre
 - Notion d'accord de clé authentifié (*Authenticated Key Agreement AKA*)

1. Utiliser une clé pré-partagée

- Clé partagée dans un environnement sécurisé
- Usure de la clé
- **Clé partagée ne doit pas être utilisée pour protéger les communications**

2. Utiliser la cryptographie à clé publique

- Le chiffrement à clé publique permet de protéger la clé échangée
- On parle d'encapsulation de clé

3. Utiliser une tierce partie de confiance

- L'échange de clé passe par cette entité
- Souvent appelée le « Centre de Distribution de Clé »

Fraicheur de l'échange de clé

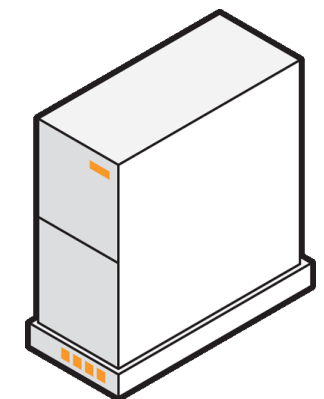
- Il est essentiel de s'assurer que la clé (l'authentification) est **bien une nouvelle clé**, et **pas un rejeu**
- Différentes méthodes
 - **Horloges synchronisées** : entre les différentes parties, avec une fenêtre d'acceptation
 - **Compteurs synchronisés** : incrémentation du compteur après chaque échange de clé valide, resynchronisation à prévoir
 - **Mécanisme de challenge-réponse** : un aléa est utilisé et vérifié pour s'assurer que ce n'est pas un rejeu

Clé pré-partagée – Exemple de l'UMTS-AKA

- UMTS Authenticated Key Agreement
 - Défini pour les réseaux 3G
 - Toujours utilisé pour les réseaux 4G et 5G (avec quelques modifications « mineures »)
- Propriétés recherchées
 - **Authentification mutuelle** entre l'utilisateur et le réseau
 - **Génération de clés de chiffrement CK et d'intégrité IK** pour les prochains échanges
 - Protection contre le rejeu via **un aléa $RAND$** (128 bits), un **compteur synchronisé SQN** (48 bits) et un champ de contrôle AMF (16 bits)
- Basé sur un algorithme ($F1, F2, F3, F4, F5$)
 - **La clé K (128 bits) pré-partagée entre la carte SIM et l'opérateur télécom** est utilisée comme entrée de cet algorithme (i.e., en entrée de chacune des fonctions)
 - Instanciation via Milenage (basé sur AES) ou TUAK (basé sur une fonction de hachage)

Clé pré-partagée – Exemple de l'UMTS-AKA

- $F1(K, RAND, AMF, SQN) = MAC_A$
- $F2(K, RAND) = RES$
- $F3(K, RAND) = CK$
- $F4(K, RAND) = IK$
- $F5(K, RAND) = AK$



$RAND, AMF, SQN \oplus AK, MAC_A$

$AK \oplus (SQN \oplus AK) = SQN$
vérifie $SQN \Rightarrow$ resynchronisation ?
vérifie MAC_A

RES

vérifie RES

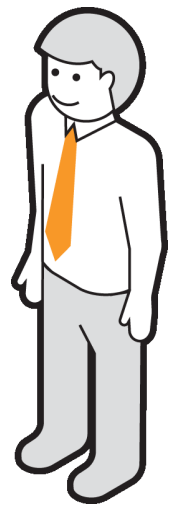
Clé publique par encapsulation

- Nécessite un système de chiffrement à clé publique (*KeyGen*, *Enc*, *Dec*)...
- ... Et un système de signature numérique (*KeyGen*, *Sign*, *Verif*)
- Principe du *Encrypt-then-Sign*
- Utilisé dans de nombreux systèmes : **TLS 1.2, SSH**

Alice



Bob



1. Génère une clé K
2. Chiffre K : $C = \text{Enc}(K, \text{pk}_{\text{Bob}})$
3. Signe C : $\sigma = \text{Sign}(C, \text{sk}_{\text{Alice}})$

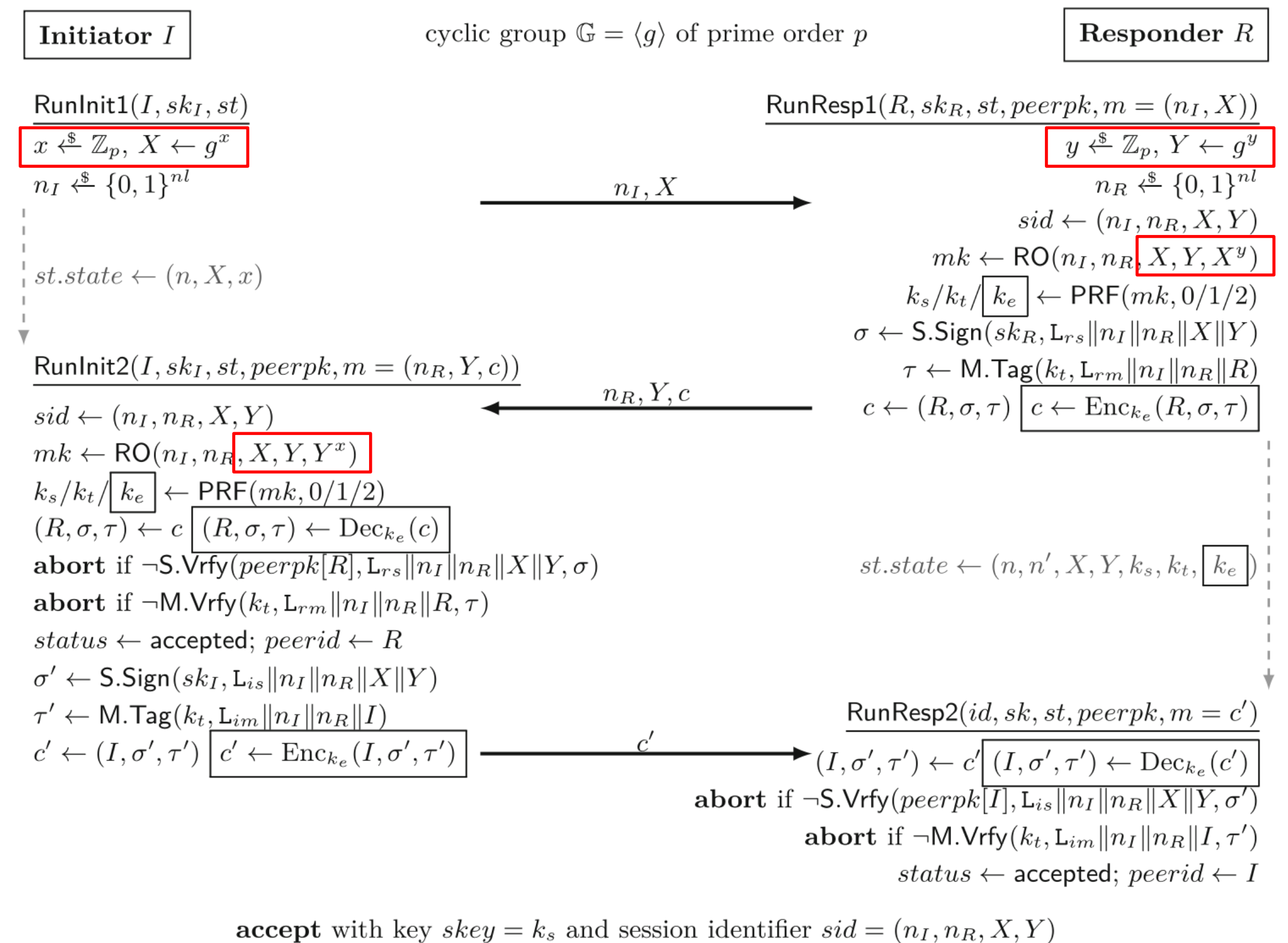
4. Envoie (C, σ)



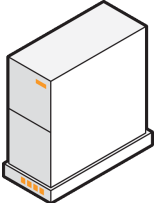
5. Vérifie σ : $\text{Verif}(C, \sigma, \text{pk}_{\text{Alice}}) = 1 ?$
6. Déchiffre K : $K = \text{Dec}(C, \text{sk}_{\text{Bob}})$

Clé publique par Diffie-Hellman – Exemple de Sigma

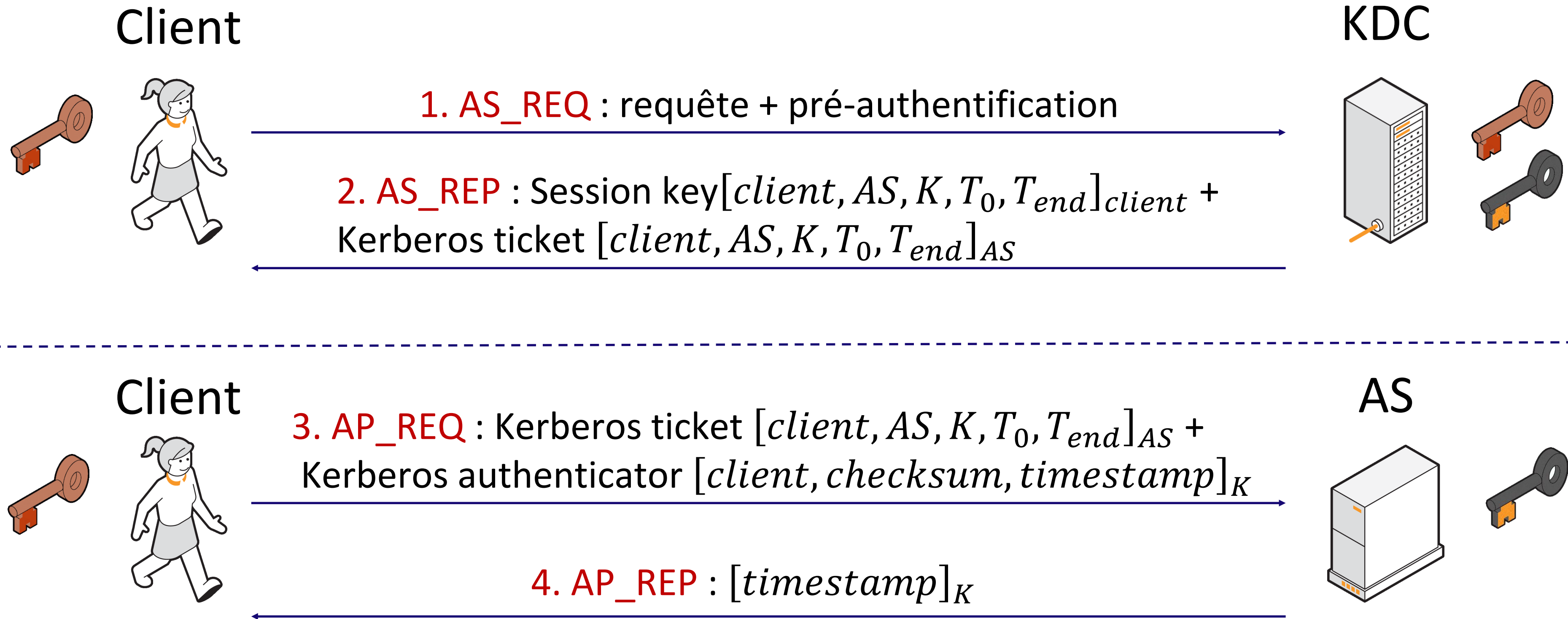
- Vous verrez une autre solution dans la partie « cryptographie à clé publique »
- « How to share a secret » par Diffie et Hellman
- Utilisé dans le protocole Sigma (base notamment de TLS 1.3, IPsec)



Tierce partie – Exemple de Kerberos

- Développé par le MIT fin des années 80, et supporté par la plupart des O.S. (Windows et Unix-like), standardisé à l'IETF (plusieurs RFC)
- Entités impliquées
 - Un **client** 
 - Permet de communiquer de façon sécurisée avec le KDC
 - Un **serveur d'application** 
 - Permet de communiquer de façon sécurisée avec le KDC
 - Un **centre de distribution de clé** (*Key Distribution Center* KDC) 
- Objectif de Kerberos
 - Client et serveur d'application partagent une clé secrète commune
 - Authentification du client, authentification mutuelle
 - Nécessite d'avoir des horloges synchronisées pour éviter le rejeu

Exemple de Kerberos – Les échanges



7. Fonctions de hachage

Concept de fonction de hachage (rappel)

Fonction de hachage

- Génération du condensé *Hash*
 - Entrées : message *m*
 - Sortie : condensé *h*
 - Vérification du condensé *HashVerif*
 - Entrées : message *m*, condensé *h*
 - Sortie : bit *0/1*
- Assure l'intégrité
 - N'assure pas l'authentification de la source
 - Utilisé dans les signatures numériques
 - Toujours déterministe
 - Sécurité
 - Résistance à la pré-image
 - Résistance à la seconde pré-image
 - Résistance aux collisions

Les fonctions de hachage

- Définition initiale
 - Fonction $H: \{0,1\}^* \rightarrow \{0,1\}^k$ pour k fixe
 - Connaissant M , le calcul de $H(M)$ doit être facile
- Fonction de hachage **cryptographiquement sûre**
 - **Résistance à la pré-image**
 - Infaisable de calculer M connaissant $H(M)$
 - **Résistance à la seconde pré-image**
 - Connaissant M et $H(M)$, infaisable de trouver M' tel que $H(M) = H(M')$ et $M \neq M'$
 - **Résistance aux collisions**
 - Infaisable de trouver M et M' tels que $H(M) = H(M')$

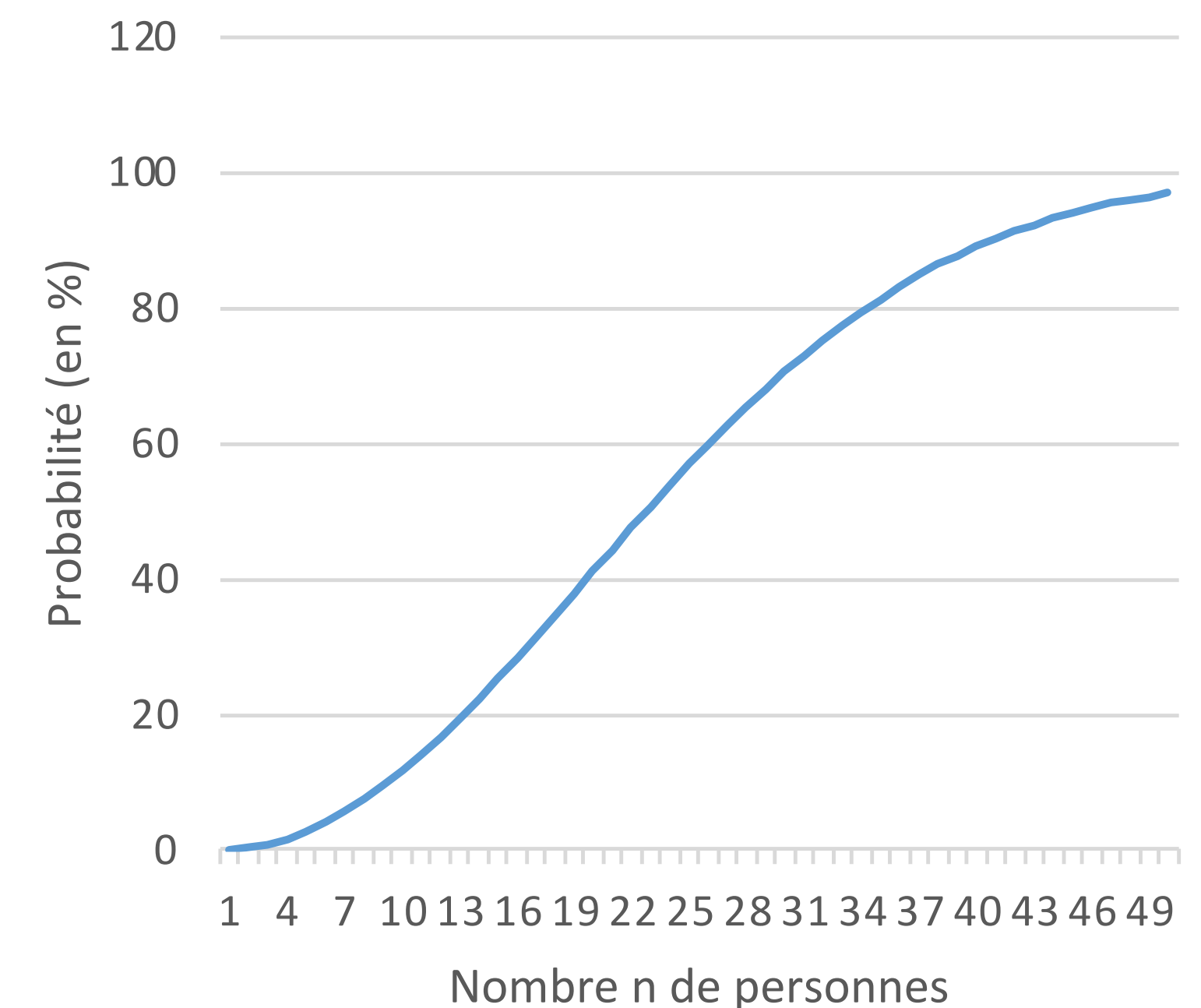
Résistance aux collisions et anniversaire

Idée générale

- Choisir M et M' au hasard et espérer que $H(M) = H(M')$
- Choisir deux personnes au hasard et espérer qu'elles aient la même date d'anniversaire
- Quelle probabilité ? Intuition = très faible

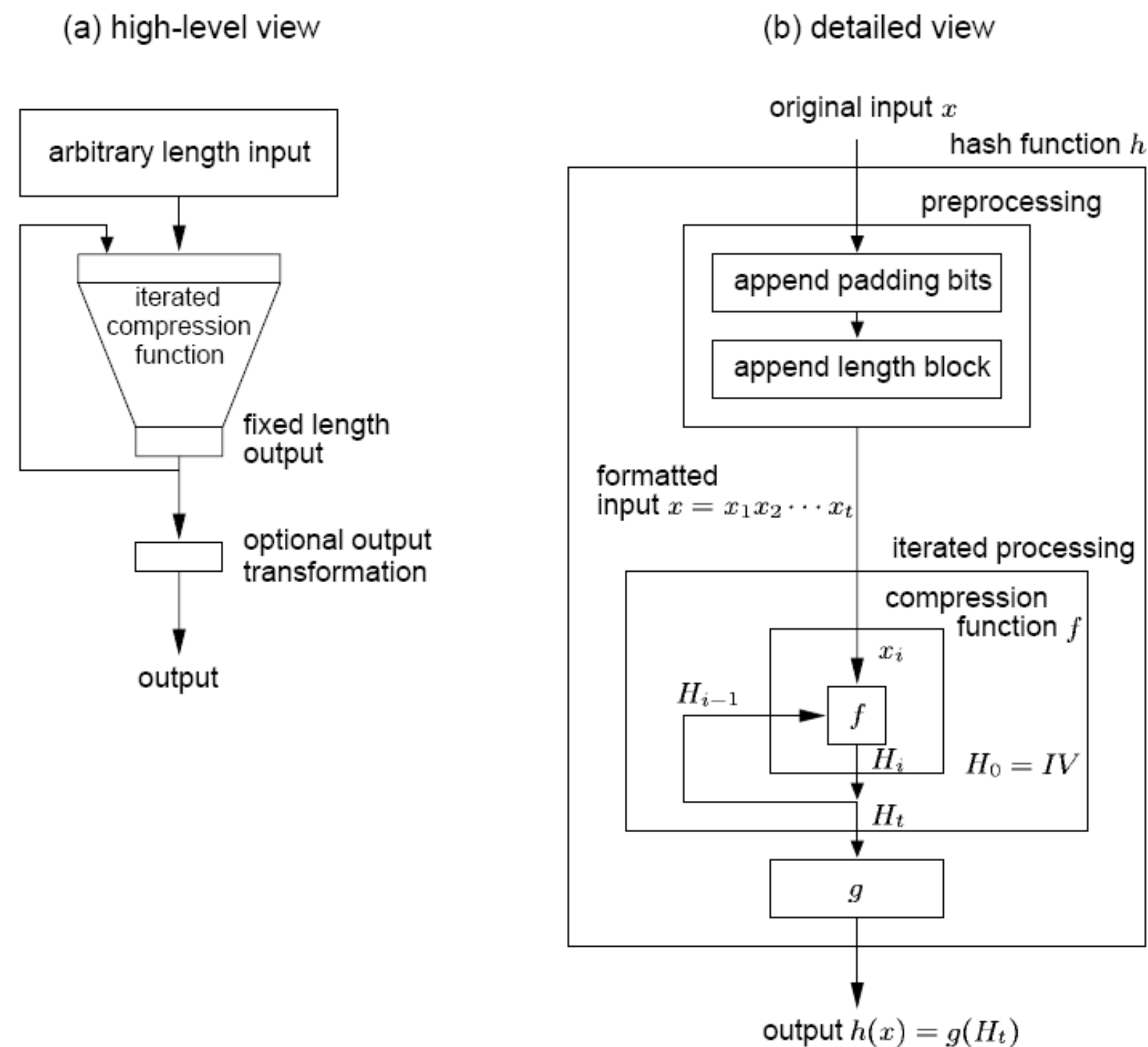
- Calculons cette probabilité $Pr(A)$ (pour 365 jours)
 - Probabilité $Pr(A')$ que dans un groupe de n personnes toutes aient un jour d'anniversaire différent
 - $Pr(A) = 1 - Pr(A')$
 - Evènement n : la personne n n'a pas la même date d'anniversaire que les personnes $n - 1, \dots, 2, 1$
 - $Pr(1) = \frac{365}{365}, Pr(2) = \frac{364}{365}, Pr(3) = \frac{363}{365}, \dots, Pr(n) = \frac{365-(n-1)}{365}$
 - Probabilités conditionnelles
 - $Pr(A') = \frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \dots \times \frac{365-(n-1)}{365} = \frac{365!}{365^n(365-n)!}$
- C'est « paradoxal » !

Paradoxe des anniversaires



Généralités sur les fonctions de hachage

- Schéma général

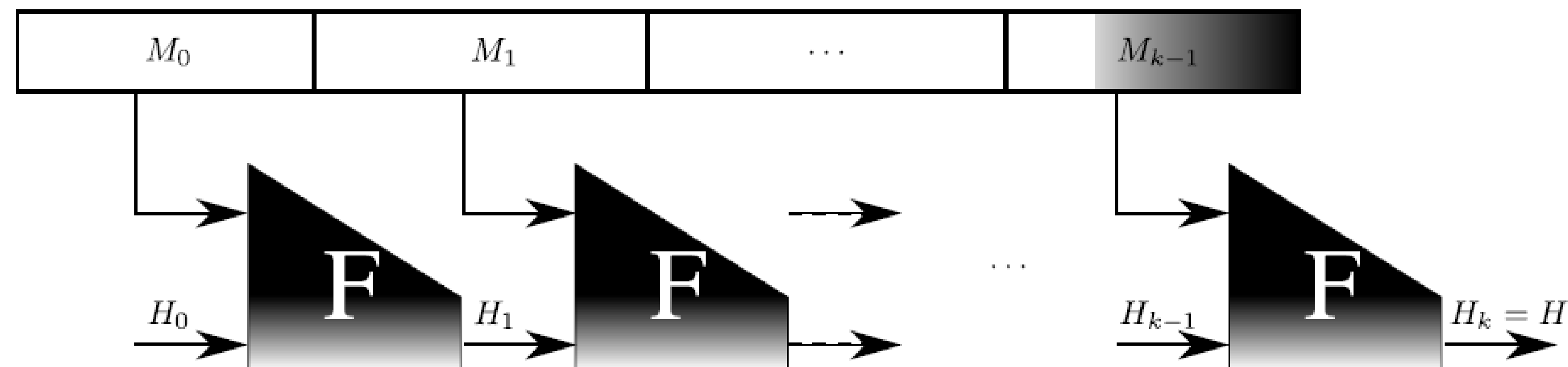


RègleHash

1. La taille minimale des empreintes produites par une fonction de hachage est de 256 bits.
2. La meilleure attaque connue permettant de trouver des collisions doit nécessiter de l'ordre de $2^{h/2}$ calculs d'empreintes, où h désigne la taille en bits des empreintes.

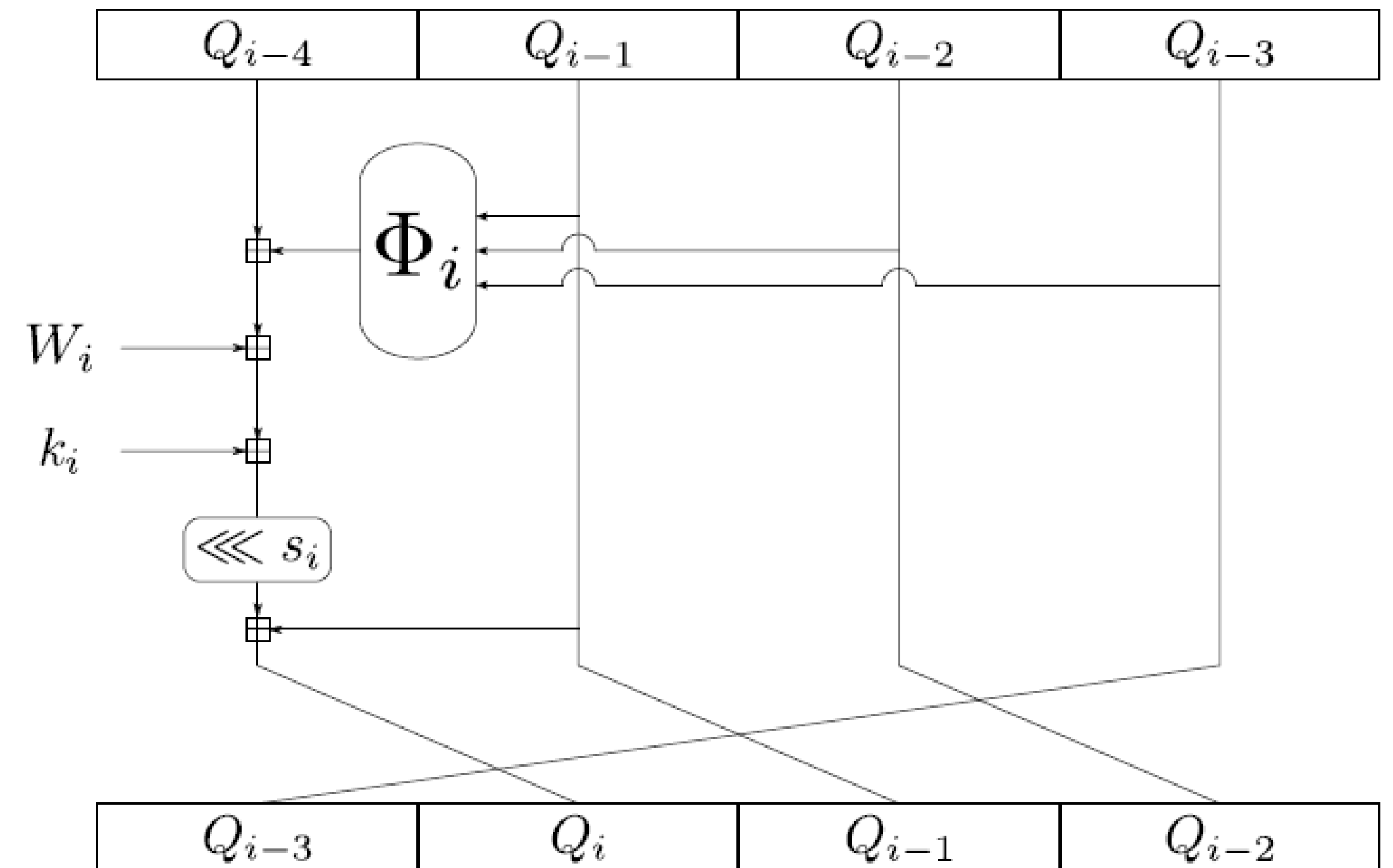
Un premier type de construction

- Principe de **Merkle Damgard**
 - Usage d'une fonction itérée
 - Fonction $F: \{0,1\}^n \times \{0,1\}^m \rightarrow \{0,1\}^n$
 - Découpage du message en plusieurs blocs
 - Vecteur d'initialisation $IV = H_0$



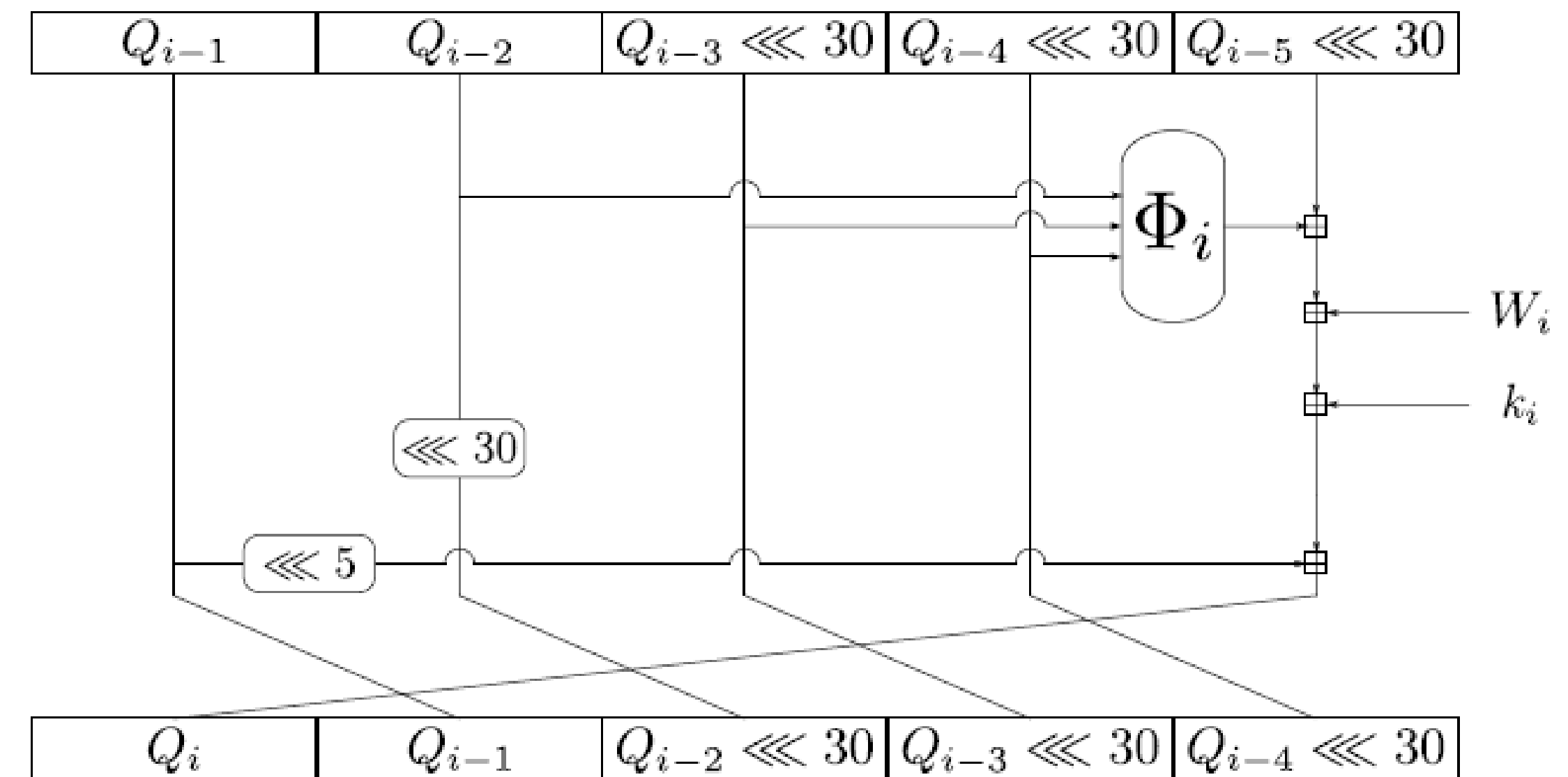
Exemple de MD5

- 4 mots de 32 bits, 4 tours
- La fonction Φ_i est une fonction booléenne, W_i est un mot du message et k_i est une constante dépendant du tour
- $\boxplus$ est l'addition modulo 2^w et $\lll s_i$ est un décalage de s_i bits



Exemple de SHA-1

- 16 mots de 32 bits, 80 tours
- Φ_i est une fonction booléenne, W_i est un mot du message (taille w) et k_i est une constante dépendant du tour
- $\boxplus$ est l'addition modulo 2^w et $\lll s_i$ est un décalage de s_i bits
- Padding
 - La taille du message est codée sur 64 bits et est rajoutée à la fin du message
 - Si l'entrée n'est pas un multiple de 512 bits, alors rajout du bit 1 puis de bits 0



Attaques sur les fonctions de hachage

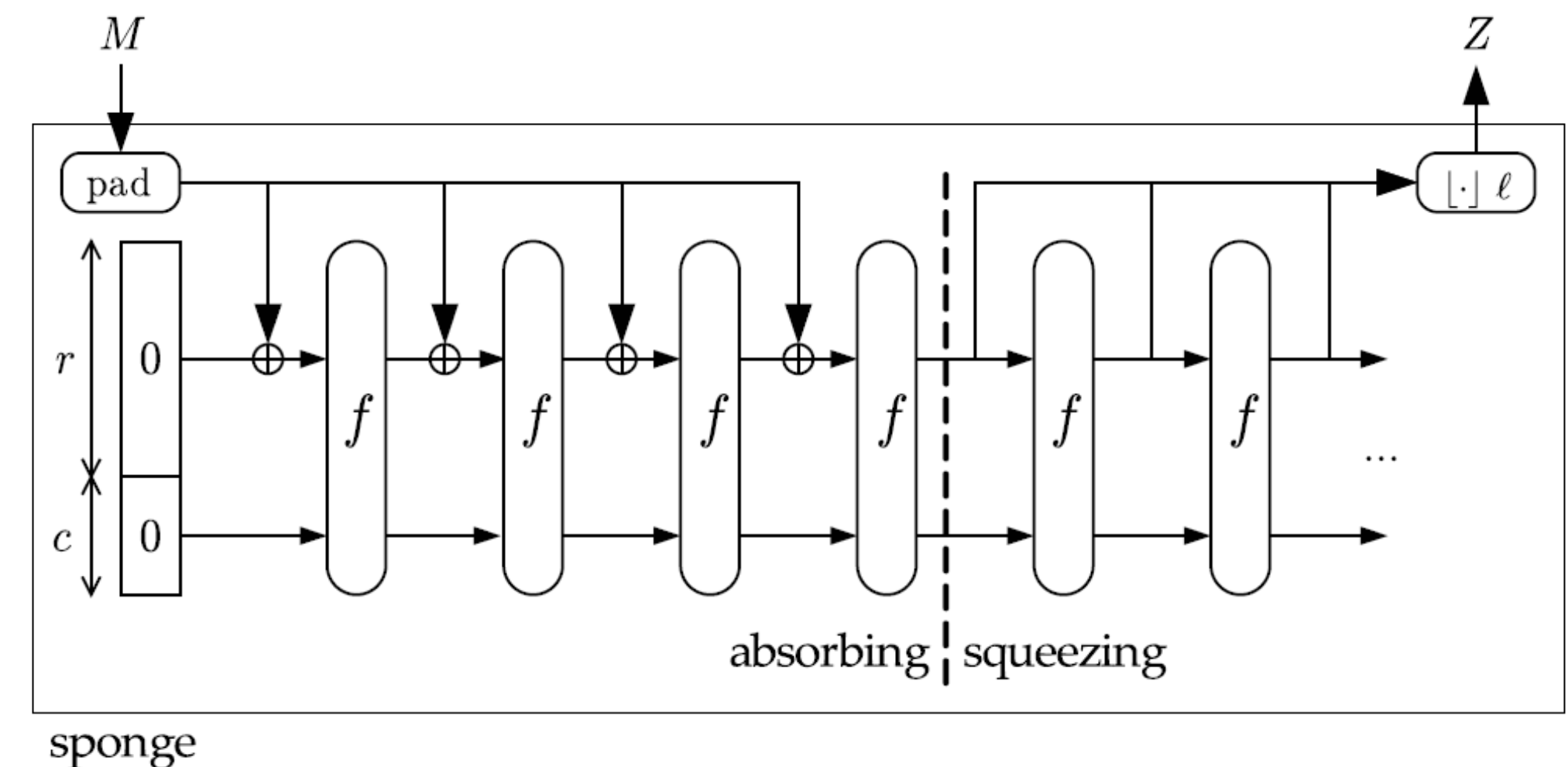
- Attaques sur la résistance aux collisions
 - Attaque en 2^{39} opérations de **SHA-0** (février 2005)
 - Attaque en 2^{63} opérations de **SHA-1** (août 2005)
 - **MD5 cassé en quelques heures** sur un système parallélisé (été 2004)
 - **Collision de SHA-1** trouvée en 2017
- Que reste-t-il ?
 - **Famille SHA2 : SHA-256, SHA-512** qui utilisent aussi le même genre de structure, mais aucune attaque connue $\Rightarrow$ **Préconisé par les autorités**
 - Nouvelle proposition...

Nouvel appel d'offre du NIST

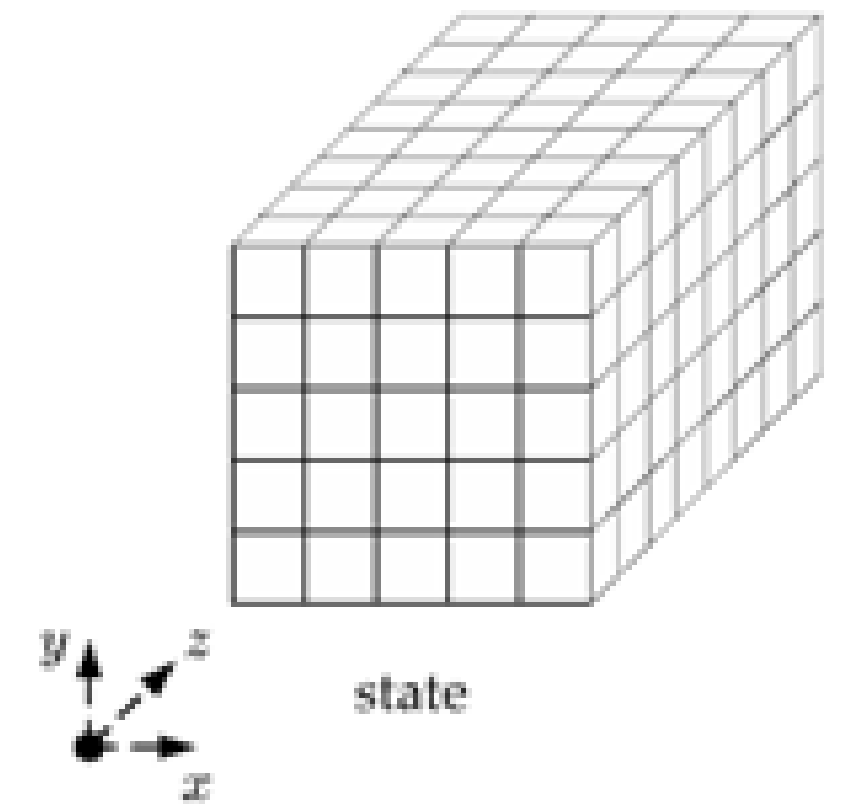
- Appel lancé en 2008 : **64 propositions**
 - 1er round : 51 restants fin 2008
 - 2ème round (juillet 2009) : 12 candidats restants (dont 3 français)
 - 3ème round (décembre 2010) : 5 finalistes
 - BLAKE (universitaires suisses), Grøstl (universitaires danois) basé sur AES, JH (universitaires de Singapour), Keccak (STMicroelectronics et NXP) et Skein (universitaires américains)
- **Septembre 2012 : vainqueur = Keccak**
 - La nouvelle fonction s'appelle **SHA-3**, ajoutée au **FIPS 180-2 Secure Hash Standard**

Keccak est une « fonction éponge »

- Basée sur une permutation f sur b bits (b = la largeur)
 - $b = r + c$ avec r = débit binaire (bitrate) et c = la capacité
- Phase d'absorption
 - Les r bits du message « xor » r bits de l'état
 - application de la permutation f
- Phase de pression
 - Les r bits de l'état sont donnés en sortie
 - application de la permutation f



Renseignements généraux sur Keccak



- Caractéristiques $Keccak[b]$
 - Etat = cube de $5 \times 5 \times 2^n$
 - $b = 25 \times 2^n$ ($n \in [0,6]$)
 - Sécurité 256 bits : $r = 1088$ et $c = 512$, sécurité 80 bits : $r = 40$ et $c = 160$
 - (Padding simple 10*), padding multi-vitesse 10*1
- Nombre de tours
 - Nombre de tours : $n_r = 12 + 2n$
 - $Keccak[25] = 12$ tours, $Keccak[1600] = 24$ tours
- Efficacité similaire à SHA-256 (grande possibilité de parallélisme)

Fonction de permutation f

- Chaque tour de la fonction f est une itération 5 fonctions de base

Fonction ι

- Cassage de la symétrie
- XOR au niveau des lignes avec des constantes dépendantes du tour
- Sans cette opération, tous les tours auraient la même susceptibilité aux attaques par glissement de la structure

Fonction π

- Opération permettant de diffuser horizontalement et verticalement

Fonction ρ

- Opération de diffusion en profondeur
- Décalage cyclique de chaque ligne

Fonction χ

- Fonction non linéaire
- Inverser le bit si les voisins présentent le modèle 01
- Fonctionne en parallèle sur des lignes de 5 bits
- Degré algébrique 2, propriétés de propagation de cryptanalyses linéaires et différentielles faciles à décrire et à analyser

Fonction θ

- Etape de mélange des bits
- Calculer la parité $c_{x,z}$ de chaque colonne
- Ajoutez à chaque cellule la parité des colonnes voisines

Fonction de hachage et MAC

Alternative au AES-CBC-MAC, on utilise SHA2/SHA3 plutôt que AES -> intégrité + authenticité = CBC-MAC != Chiffrement authentifié

- Il y a des similitudes entre une fonction de hachage et un MAC
- Construction par préfixe $MAC_K(M) = H(K \parallel M)$ **n'est pas sûr** (extension du message)
- Construction par suffixe $MAC_K(M) = H(M \parallel K)$ **est mieux**, mais peut poser problème si H est faible à la résistance aux collisions
- Construction par enveloppe $MAC_K(M) = H(K_1 \parallel M \parallel K_2)$ **semble ok**, mais aucune preuve
- Construction de type « HMAC » $MAC_K(M) = H(K_1 \parallel H(K_2 \parallel M))$ **est prouvée sûre**
 - Intuition : il est difficile de contrôler $H(K_2 \parallel M)$ sans connaître K_1 , difficile de trouver une collision pour le H interne

Détails de la fonction *HMAC*

- Pour une clé K et un message M
- Clé $K' = \begin{cases} H(K) & \text{si } K \text{ est supérieur au bloc} \\ K & \text{sinon} \end{cases}$
- Fonction $HMAC(M, K) = H\left((K' \oplus opad) || H((K' \oplus ipad) || M)\right)$
- Fonction H est une fonction de hachage
- Chaîne *opad* est un padding “externe” (“outer”) répétant l’octet 0x5C
- Chaîne *ipad* est un padding “interne” (“inner”) répétant l’octet 0x36

De multiples applications aux fonctions de hachage

- Vérification de l'intégrité d'un message
 - Sans clé : avec une signature numérique
 - Avec clé : sans signature numérique (MAC)
- Code de détection de modification (pour des logiciels téléchargés)
- Fonction de dérivation d'une clé (*Hash-based Key Derivation Function HKDF*)
 - A partir d'une autre clé (cf. 4G/5G)
 - A partir d'un mot de passe
- Fonction de génération forcée d'un aléa

Merci

